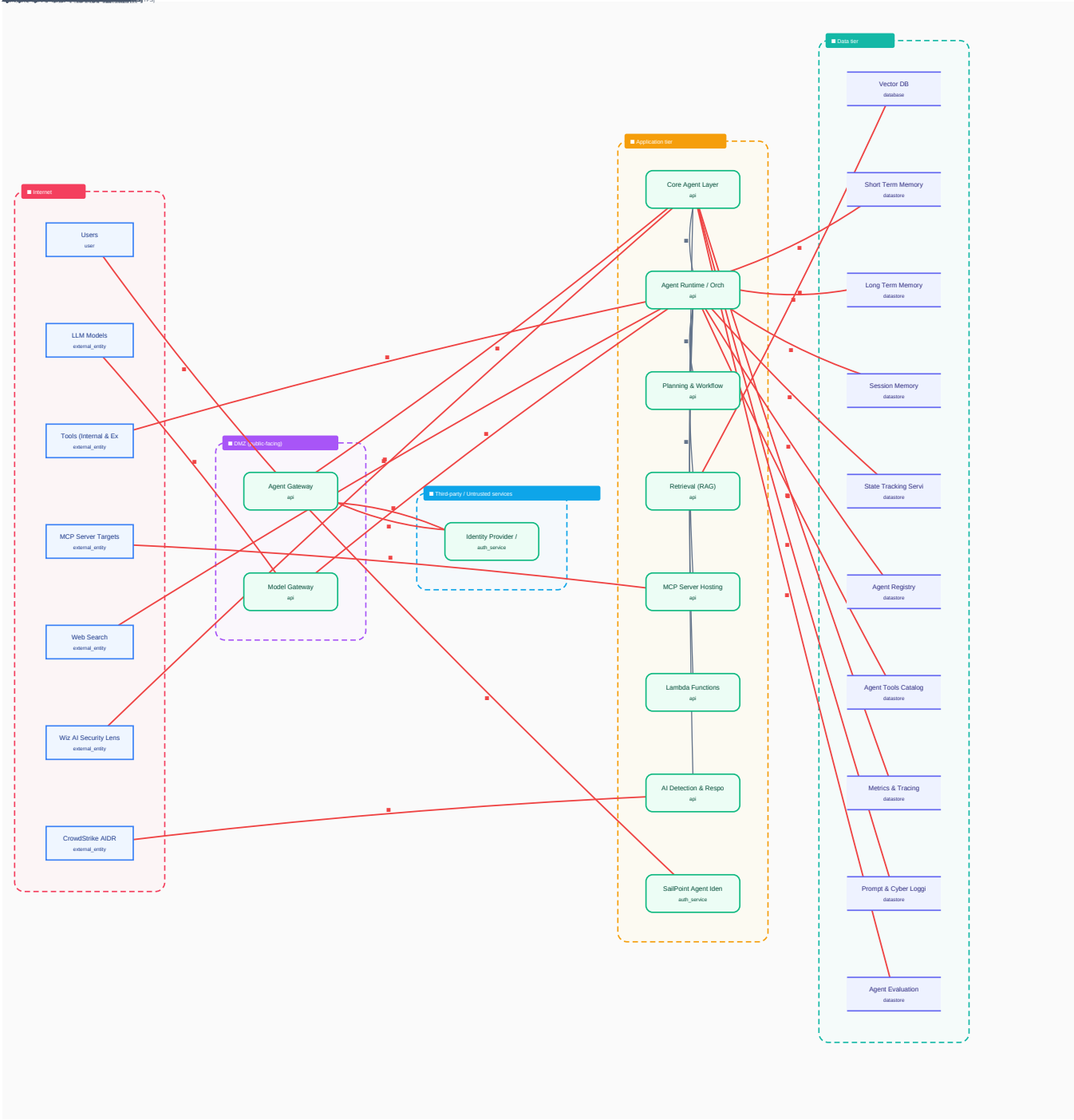


Threat Model Report: AEGIS

Generated 2026-07-18 03:11 UTC | Methodologies: STRIDE | LLM-enhanced: No (not requested)

Data Flow Diagram



Executive Summary

Total threats: **386** (rule-based: 386, LLM: 0, cross-boundary: 88)

Severity	Count
----------	-------

Critical	56
High	257
Medium	73
Low	0
Info	0

Components

Name	Type	Description
Users	user	Declared component (user)
Agent Gateway	api	Declared component (api)
Identity Provider / Okta MCP	auth_service	Declared component (auth_service)
Core Agent Layer	api	Declared component (api)
Agent Runtime / Orchestration	api	Declared component (api)
Planning & Workflow	api	Declared component (api)
Model Gateway	api	Declared component (api)
LLM Models	external_entity	Declared component (external_entity)
Vector DB	database	Declared component (database)
Short Term Memory	datastore	Declared component (datastore)
Long Term Memory	datastore	Declared component (datastore)
Session Memory	datastore	Declared component (datastore)
State Tracking Services	datastore	Declared component (datastore)
Retrieval (RAG)	api	Declared component (api)
Tools (Internal & External)	external_entity	Declared component (external_entity)
MCP Server Hosting	api	Declared component (api)
MCP Server Targets	external_entity	Declared component (external_entity)
Agent Registry	datastore	Declared component (datastore)
Agent Tools Catalog	datastore	Declared component (datastore)
Web Search	external_entity	Declared component (external_entity)
Lambda Functions	api	Declared component (api)
AI Detection & Response	api	Declared component (api)
Metrics & Tracing	datastore	Declared component (datastore)
Prompt & Cyber Logging	datastore	Declared component (datastore)
Agent Evaluation	datastore	Declared component (datastore)
SailPoint Agent Identity	auth_service	Declared component (auth_service)
Wiz AI Security Lens	external_entity	Declared component (external_entity)
CrowdStrike AIDR	external_entity	Declared component (external_entity)

■ Untrusted-Input Boundary Crossings

Flows where untrusted (or less-trusted) input crosses into an internal trust zone. These are the highest-priority validation points in the system — every byte that enters here must be treated as hostile until proven otherwise.

Agent Gateway → Identity Provider / Okta MCP *[Critical]*

Zones: DMZ (public-facing) → Third-party / Untrusted services

Flow: *Agent Gateway → Identity Provider / Okta MCP* · Protocol: HTTPS · Auth: bearer · Encryption: encrypted · 4 threats on this flow

■ Validation requirements at the receiver:

- Define an explicit allow-list schema (JSON Schema, OpenAPI, Protobuf) for accepted payloads — reject anything that doesn't match.
- Validate every field's type, length, charset, and value range BEFORE business logic runs.
- Canonicalize input (URL-decode, Unicode NFC, path-resolve) BEFORE validation to defeat bypass tricks.

Trust Boundary Analysis

■ **Internet**

Contains: Users, LLM Models, Tools (Internal & External), MCP Server Targets, Web Search, Wiz AI Security Lens, CrowdStrike AIDR

Ingress:

- Model Gateway → LLM Models (Model Gateway → LLM Models) — HTTPS, auth: api_key, encrypted: yes
- MCP Server Hosting → MCP Server Targets (MCP Server Hosting → MCP Server Targets) — HTTPS, auth: mtls, encrypted: yes
- Agent Runtime / Orchestrator → Tools (Internal & External) (Agent Runtime / Orchestrator → Tools (Internal & External)) — HTTPS, auth: api_key, encrypted: yes
- Agent Runtime / Orchestrator → Web Search (Agent Runtime / Orchestrator → Web Search) — HTTPS, auth: api_key, encrypted: yes
- AI Detection & Response → CrowdStrike AIDR (AI Detection & Response → CrowdStrike AIDR) — HTTPS, auth: api_key, encrypted: yes
- Core Agent Layer → Wiz AI Security Lens (Core Agent Layer → Wiz AI Security Lens) — HTTPS, auth: bearer, encrypted: yes

Egress:

- Users → Agent Gateway (Users → Agent Gateway) — HTTPS, auth: session, encrypted: yes

Cross-boundary threats affecting this zone: 24

■ **DMZ (public-facing)**

Contains: Agent Gateway, Model Gateway

Ingress:

- Users → Agent Gateway (Users → Agent Gateway) — HTTPS, auth: session, encrypted: yes
- Identity Provider / Okta MCP → Agent Gateway (Identity Provider / Okta MCP → Agent Gateway) — HTTPS, auth: bearer, encrypted: yes
- Agent Runtime / Orchestrator → Model Gateway (Agent Runtime / Orchestrator → Model Gateway) — HTTPS, auth: bearer, encrypted: yes

Egress:

- Agent Gateway → Identity Provider / Okta MCP (Agent Gateway → Identity Provider / Okta MCP) — HTTPS, auth: bearer, encrypted: yes
- Agent Gateway → Core Agent Layer (Agent Gateway → Core Agent Layer) — HTTPS, auth: bearer, encrypted: yes
- Model Gateway → LLM Models (Model Gateway → LLM Models) — HTTPS, auth: api_key, encrypted: yes
- Agent Gateway → SailPoint Agent Identity (Agent Gateway → SailPoint Agent Identity) — HTTPS, auth: bearer, encrypted: yes

Cross-boundary threats affecting this zone: 12

■ **Third-party / Untrusted services**

Contains: Identity Provider / Okta MCP

Ingress:

- Agent Gateway → Identity Provider / Okta MCP (Agent Gateway → Identity Provider / Okta MCP) — HTTPS, auth: bearer, encrypted: yes

Egress:

- Identity Provider / Okta MCP → Agent Gateway (Identity Provider / Okta MCP → Agent Gateway) — HTTPS, auth: bearer, encrypted: yes

Cross-boundary threats affecting this zone: 4

■ **Application tier**

Contains: Core Agent Layer, Agent Runtime / Orchestrator, Planning & Workflow, Retrieval (RAG), MCP Server Hosting, Lambda Functions, AI Detection & Response, SailPoint Agent Identity

Ingress:

- Agent Gateway → Core Agent Layer (Agent Gateway → Core Agent Layer) — HTTPS, auth: bearer, encrypted: yes

- Agent Gateway → SailPoint Agent Identity (Agent Gateway → SailPoint Agent Identity) — HTTPS, auth: bearer, encrypted: yes

Egress:

- Agent Runtime / Orchestrator → Agent Registry (Agent Runtime / Orchestrator → Agent Registry) — HTTPS, auth: bearer, encrypted: yes

- Agent Runtime / Orchestrator → Agent Tools Catalog (Agent Runtime / Orchestrator → Agent Tools Catalog) — HTTPS, auth: bearer, encrypted: yes

- Retrieval (RAG) → Vector DB (Retrieval (RAG) → Vector DB) — TCP, auth: credentials, encrypted: yes

- Agent Runtime / Orchestrator → Short Term Memory (Agent Runtime / Orchestrator → Short Term Memory) — TCP, auth: credentials, encrypted: yes

- Agent Runtime / Orchestrator → Long Term Memory (Agent Runtime / Orchestrator → Long Term Memory) — TCP, auth: credentials, encrypted: yes

- Agent Runtime / Orchestrator → Session Memory (Agent Runtime / Orchestrator → Session Memory) — TCP, auth: credentials, encrypted: yes

- Agent Runtime / Orchestrator → State Tracking Services (Agent Runtime / Orchestrator → State Tracking Services) — TCP, auth: credentials, encrypted: yes

- Agent Runtime / Orchestrator → Model Gateway (Agent Runtime / Orchestrator → Model Gateway) — HTTPS, auth: bearer, encrypted: yes

- MCP Server Hosting → MCP Server Targets (MCP Server Hosting → MCP Server Targets) — HTTPS, auth: mtls, encrypted: yes

- Agent Runtime / Orchestrator → Tools (Internal & External) (Agent Runtime / Orchestrator → Tools (Internal & External)) — HTTPS, auth: api_key, encrypted: yes

- Agent Runtime / Orchestrator → Web Search (Agent Runtime / Orchestrator → Web Search) — HTTPS, auth: api_key, encrypted: yes

- AI Detection & Response → CrowdStrike AIDR (AI Detection & Response → CrowdStrike AIDR) — HTTPS, auth: api_key, encrypted: yes

- Core Agent Layer → Prompt & Cyber Logging (Core Agent Layer → Prompt & Cyber Logging) — HTTPS, auth: bearer, encrypted: yes

- Core Agent Layer → Metrics & Tracing (Core Agent Layer → Metrics & Tracing) — HTTPS, auth: bearer, encrypted: yes

- Core Agent Layer → Agent Evaluation (Core Agent Layer → Agent Evaluation) — HTTPS, auth: bearer, encrypted: yes

- Core Agent Layer → Wiz AI Security Lens (Core Agent Layer → Wiz AI Security Lens) — HTTPS, auth: bearer, encrypted: yes

Cross-boundary threats affecting this zone: 8

■ **Data tier**

Contains: Vector DB, Short Term Memory, Long Term Memory, Session Memory, State Tracking Services, Agent Registry, Agent Tools Catalog, Metrics & Tracing, Prompt & Cyber Logging, Agent Evaluation

Ingress:

- Agent Runtime / Orchestrator → Agent Registry (Agent Runtime / Orchestrator → Agent Registry) — HTTPS, auth: bearer, encrypted: yes
- Agent Runtime / Orchestrator → Agent Tools Catalog (Agent Runtime / Orchestrator → Agent Tools Catalog) — HTTPS, auth: bearer, encrypted: yes
- Retrieval (RAG) → Vector DB (Retrieval (RAG) → Vector DB) — TCP, auth: credentials, encrypted: yes
- Agent Runtime / Orchestrator → Short Term Memory (Agent Runtime / Orchestrator → Short Term Memory) — TCP, auth: credentials, encrypted: yes
- Agent Runtime / Orchestrator → Long Term Memory (Agent Runtime / Orchestrator → Long Term Memory) — TCP, auth: credentials, encrypted: yes
- Agent Runtime / Orchestrator → Session Memory (Agent Runtime / Orchestrator → Session Memory) — TCP, auth: credentials, encrypted: yes
- Agent Runtime / Orchestrator → State Tracking Services (Agent Runtime / Orchestrator → State Tracking Services) — TCP, auth: credentials, encrypted: yes
- Core Agent Layer → Prompt & Cyber Logging (Core Agent Layer → Prompt & Cyber Logging) — HTTPS, auth: bearer, encrypted: yes
- Core Agent Layer → Metrics & Tracing (Core Agent Layer → Metrics & Tracing) — HTTPS, auth: bearer, encrypted: yes
- Core Agent Layer → Agent Evaluation (Core Agent Layer → Agent Evaluation) — HTTPS, auth: bearer, encrypted: yes

Cross-boundary threats affecting this zone: 40

Identified Threats

Critical (56)

Broken access control / missing authz check

STRIDE → Elevation of Privilege | Component: **AI Detection & Response** (api) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) [CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L](#)

CVSS 4.0: 6.3 (Medium) [CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **AI Detection & Response** (`api`).

■ **Description:** Endpoints fail to verify the caller has permission for the action.

■ ■ Attack scenario:

1. Attacker authenticates to AI Detection & Response as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:9 R:10 E:9 A:10 D:9 → **Total 47/50**

■ How to mitigate:

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to AI Detection & Response should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Container / process escape

STRIDE → Elevation of Privilege | Component: **AI Detection & Response** (api) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) [CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L](#)

CVSS 4.0: 6.3 (Medium) [CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **AI Detection & Response** (`api`).

■ **Description:** Attacker breaks out of a sandbox to gain host privileges.

■ ■ Attack scenario:

1. Attacker authenticates to AI Detection & Response as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:9 R:8 E:9 A:10 D:9 → **Total 45/50**

■ How to mitigate:

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to AI Detection & Response should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

Stored data tampering via injection

STRIDE → Tampering | Component: **Agent Evaluation** (datastore) | Source: rule-based

CWE: CWE-345 — Insufficient Verification of Data Authenticity

CVSS 3.1: 7.3 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Evaluation** (`datastore`).

■ **Description:** SQL/NoSQL/command injection alters records or schema.

■■ Attack scenario:

1. Attacker reaches an input vector exposed by Agent Evaluation (request body, query string, message queue payload, persisted state).
2. Submits malformed input designed to alter the program's logic or stored data (injection, parameter tampering, deserialization gadgets).
3. Agent Evaluation processes the input without strict validation, and the malicious change takes effect.
4. Tampered data is persisted, executed, or relayed downstream — corrupting integrity beyond the original entry point.

DREAD — D:10 R:10 E:9 A:7 D:9 → **Total 45/50**

■ How to mitigate:

[preventive] **Schema-validate and parameterize all inputs** — JSON Schema / OpenAPI validation at API edges; parameterized queries / prepared statements for SQL; ORM-level validators on persisted models.

[preventive] **Sign and verify integrity-critical messages** — For commands or state mutations, attach an HMAC or signature so downstream consumers can detect tampering in transit or at rest.

[detective] **Use append-only / write-once stores for audit-critical data** — Where data must not be silently changed, write to an append-only log (e.g., immutable S3 bucket, CloudTrail-like store) and reconcile against the mutable copy.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-345 — Insufficient Verification of Data Authenticity](#)

Privilege transit across boundary: Core Agent Layer → Agent Evaluation [cross-boundary]

STRIDE → Elevation of Privilege | Component: **Agent Evaluation** (datastore) | Source: rule-based

CWE: CWE-441 — Confused Deputy

CVSS 3.1: 8.9 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:L

CVSS 4.0: 7.4 (High) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:H/SI:H/SA:L

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Core Agent Layer** → **Agent Evaluation** (label: `*Core Agent Layer → Agent Evaluation*`, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from ``Application tier`` into ``Data tier``. The receiving component is **Agent Evaluation** (`datastore`).

■ **Description:** If the receiver in 'Data tier' acts on behalf of the caller, attackers compromising 'Application tier' may inherit the receiver's privileges (confused-deputy).

■■ Attack scenario:

1. Attacker compromises a low-privilege component in zone ``Application tier``.
2. Issues requests to Agent Evaluation that get re-executed with the receiver's higher privileges (classic confused-deputy).
3. Because authorization is checked once at request entry but not per-action against the originating caller, the attacker effectively becomes a privileged actor in ``Data tier``.
4. Attacker performs actions far beyond the original principal's permissions.

DREAD — D:10 R:8 E:9 A:9 D:9 → **Total 45/50**

■ How to mitigate:

[preventive] **Use scoped, short-lived delegation tokens** — When Agent Evaluation acts on behalf of a caller, attach a delegation token with the caller's identity, scope (action allow-list), and ≤ 5-min TTL.

[preventive] **Authorize per-action against the originating caller** — Don't rely on ambient authority of the receiver. Each action checks: (a) the delegation token is valid, (b) the original caller has permission for this specific action, (c) the action is within scope.

[*detective*] **Audit each privilege transit** — Log: original caller → delegating component → action performed. Make this queryable for incident response.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-441 — Confused Deputy](#)

Broken access control / missing authz check

STRIDE → Elevation of Privilege | Component: **Agent Gateway** (api) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Gateway** (`api`).

■ **Description:** Endpoints fail to verify the caller has permission for the action.

■ ■ Attack scenario:

1. Attacker authenticates to Agent Gateway as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:9 R:10 E:10 A:10 D:10 → **Total 49/50**

■ How to mitigate:

[*preventive*] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[*preventive*] **Apply least-privilege to service identities** — The IAM/role attached to Agent Gateway should grant only the actions it actually performs. Audit IAM grants quarterly.

[*detective*] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Container / process escape

STRIDE → Elevation of Privilege | Component: **Agent Gateway** (api) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Gateway** (`api`).

■ **Description:** Attacker breaks out of a sandbox to gain host privileges.

■ ■ Attack scenario:

1. Attacker authenticates to Agent Gateway as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:9 R:8 E:10 A:10 D:10 → **Total 47/50**

■ How to mitigate:

[*preventive*] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[*preventive*] **Apply least-privilege to service identities** — The IAM/role attached to Agent Gateway should grant only the actions it actually performs. Audit IAM grants quarterly.

[*detective*] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

Privilege transit across boundary: Users → Agent Gateway [cross-boundary]

STRIDE → Elevation of Privilege | Component: **Agent Gateway** (api) | Source: rule-based

CWE: CWE-441 — Confused Deputy

CVSS 3.1: 9.9 (Critical) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:L

CVSS 4.0: 7.4 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:H/SI:H/SA:L

Boundary crossing: Internet → DMZ (public-facing)

■ **Location:** On the data flow **Users → Agent Gateway** (label: *Users → Agent Gateway*, protocol: HTTPS, auth: session, encrypted: yes), crossing the trust boundary from `Internet` into `DMZ (public-facing)`. The receiving component is **Agent Gateway** (`api`).

■ **Description:** If the receiver in 'DMZ (public-facing)' acts on behalf of the caller, attackers compromising 'Internet' may inherit the receiver's privileges (confused-deputy).

■ ■ Attack scenario:

1. Attacker compromises a low-privilege component in zone `Internet`.
2. Issues requests to Agent Gateway that get re-executed with the receiver's higher privileges (classic confused-deputy).
3. Because authorization is checked once at request entry but not per-action against the originating caller, the attacker effectively becomes a privileged actor in `DMZ (public-facing)`.
4. Attacker performs actions far beyond the original principal's permissions.

DREAD — D:9 R:8 E:10 A:10 D:10 → **Total 47/50**

■ How to mitigate:

[preventive] **Use scoped, short-lived delegation tokens** — When Agent Gateway acts on behalf of a caller, attach a delegation token with the caller's identity, scope (action allow-list), and ≤ 5-min TTL.

[preventive] **Authorize per-action against the originating caller** — Don't rely on ambient authority of the receiver. Each action checks: (a) the delegation token is valid, (b) the original caller has permission for this specific action, (c) the action is within scope.

[detective] **Audit each privilege transit** — Log: original caller → delegating component → action performed. Make this queryable for incident response.

A01:2021 — Broken Access Control | STRIDE reference | CWE-441 — Confused Deputy

Privilege transit across boundary: Identity Provider / Okta MCP → Agent Gateway [cross-boundary]

STRIDE → Elevation of Privilege | Component: **Agent Gateway** (api) | Source: rule-based

CWE: CWE-441 — Confused Deputy

CVSS 3.1: 9.9 (Critical) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:L

CVSS 4.0: 7.4 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:H/SI:H/SA:L

Boundary crossing: Third-party / Untrusted services → DMZ (public-facing)

■ **Location:** On the data flow **Identity Provider / Okta MCP → Agent Gateway** (label: *Identity Provider / Okta MCP → Agent Gateway*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `Third-party / Untrusted services` into `DMZ (public-facing)`. The receiving component is **Agent Gateway** (`api`).

■ **Description:** If the receiver in 'DMZ (public-facing)' acts on behalf of the caller, attackers compromising 'Third-party / Untrusted services' may inherit the receiver's privileges (confused-deputy).

■ ■ Attack scenario:

1. Attacker compromises a low-privilege component in zone `Third-party / Untrusted services`.
2. Issues requests to Agent Gateway that get re-executed with the receiver's higher privileges (classic confused-deputy).
3. Because authorization is checked once at request entry but not per-action against the originating caller, the attacker effectively becomes a privileged actor in `DMZ (public-facing)`.
4. Attacker performs actions far beyond the original principal's permissions.

DREAD — D:9 R:8 E:10 A:10 D:10 → **Total 47/50**

■ How to mitigate:

[preventive] **Use scoped, short-lived delegation tokens** — When Agent Gateway acts on behalf of a caller, attach a delegation token with the

caller's identity, scope (action allow-list), and ≤ 5 -min TTL.

[preventive] **Authorize per-action against the originating caller** — Don't rely on ambient authority of the receiver. Each action checks: (a) the delegation token is valid, (b) the original caller has permission for this specific action, (c) the action is within scope.

[detective] **Audit each privilege transit** — Log: original caller → delegating component → action performed. Make this queryable for incident response.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-441 — Confused Deputy](#)

Stored data tampering via injection

STRIDE → Tampering | Component: **Agent Registry** (datastore) | Source: rule-based

CWE: CWE-345 — Insufficient Verification of Data Authenticity

CVSS 3.1: 7.3 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Registry** (`datastore`).

■ **Description:** SQL/NoSQL/command injection alters records or schema.

■ ■ Attack scenario:

1. Attacker reaches an input vector exposed by Agent Registry (request body, query string, message queue payload, persisted state).
2. Submits malformed input designed to alter the program's logic or stored data (injection, parameter tampering, deserialization gadgets).
3. Agent Registry processes the input without strict validation, and the malicious change takes effect.
4. Tampered data is persisted, executed, or relayed downstream — corrupting integrity beyond the original entry point.

DREAD — D:10 R:10 E:9 A:7 D:9 → **Total 45/50**

■ How to mitigate:

[preventive] **Schema-validate and parameterize all inputs** — JSON Schema / OpenAPI validation at API edges; parameterized queries / prepared statements for SQL; ORM-level validators on persisted models.

[preventive] **Sign and verify integrity-critical messages** — For commands or state mutations, attach an HMAC or signature so downstream consumers can detect tampering in transit or at rest.

[detective] **Use append-only / write-once stores for audit-critical data** — Where data must not be silently changed, write to an append-only log (e.g., immutable S3 bucket, CloudTrail-like store) and reconcile against the mutable copy.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-345 — Insufficient Verification of Data Authenticity](#)

Privilege transit across boundary: **Agent Runtime / Orchestrator** → **Agent Registry** [cross-boundary]

STRIDE → Elevation of Privilege | Component: **Agent Registry** (datastore) | Source: rule-based

CWE: CWE-441 — Confused Deputy

CVSS 3.1: 8.9 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:L

CVSS 4.0: 7.4 (High) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:H/SI:H/SA:L

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Agent Runtime / Orchestrator** → **Agent Registry** (label: *Agent Runtime / Orchestrator → Agent Registry*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Agent Registry** (`datastore`).

■ **Description:** If the receiver in 'Data tier' acts on behalf of the caller, attackers compromising 'Application tier' may inherit the receiver's privileges (confused-deputy).

■ ■ Attack scenario:

1. Attacker compromises a low-privilege component in zone `Application tier`.
2. Issues requests to Agent Registry that get re-executed with the receiver's higher privileges (classic confused-deputy).
3. Because authorization is checked once at request entry but not per-action against the originating caller, the attacker effectively becomes a privileged actor in `Data tier`.
4. Attacker performs actions far beyond the original principal's permissions.

DREAD — D:10 R:8 E:9 A:9 D:9 → **Total 45/50**

■ How to mitigate:

[preventive] **Use scoped, short-lived delegation tokens** — When Agent Registry acts on behalf of a caller, attach a delegation token with the caller's identity, scope (action allow-list), and ≤ 5 -min TTL.

[preventive] **Authorize per-action against the originating caller** — Don't rely on ambient authority of the receiver. Each action checks: (a) the delegation token is valid, (b) the original caller has permission for this specific action, (c) the action is within scope.

[detective] **Audit each privilege transit** — Log: original caller → delegating component → action performed. Make this queryable for incident response.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-441 — Confused Deputy](#)

Broken access control / missing authz check

STRIDE → Elevation of Privilege | Component: **Agent Runtime / Orchestrator** (api) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Runtime / Orchestrator** (`api`).

■ **Description:** Endpoints fail to verify the caller has permission for the action.

■ ■ Attack scenario:

1. Attacker authenticates to Agent Runtime / Orchestrator as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:9 R:10 E:9 A:10 D:9 → **Total 47/50**

■ How to mitigate:

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to Agent Runtime / Orchestrator should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Container / process escape

STRIDE → Elevation of Privilege | Component: **Agent Runtime / Orchestrator** (api) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Runtime / Orchestrator** (`api`).

■ **Description:** Attacker breaks out of a sandbox to gain host privileges.

■ ■ Attack scenario:

1. Attacker authenticates to Agent Runtime / Orchestrator as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:9 R:8 E:9 A:10 D:9 → **Total 45/50**

■ How to mitigate:

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to Agent Runtime / Orchestrator should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Stored data tampering via injection

STRIDE → Tampering | Component: **Agent Tools Catalog** (datastore) | Source: rule-based

CWE: CWE-345 — Insufficient Verification of Data Authenticity

CVSS 3.1: 7.3 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Tools Catalog** (`datastore`).

■ **Description:** SQL/NoSQL/command injection alters records or schema.

■ ■ ■ Attack scenario:

1. Attacker reaches an input vector exposed by Agent Tools Catalog (request body, query string, message queue payload, persisted state).
2. Submits malformed input designed to alter the program's logic or stored data (injection, parameter tampering, deserialization gadgets).
3. Agent Tools Catalog processes the input without strict validation, and the malicious change takes effect.
4. Tampered data is persisted, executed, or relayed downstream — corrupting integrity beyond the original entry point.

DREAD — D:10 R:10 E:9 A:7 D:9 → **Total 45/50**

■ How to mitigate:

[preventive] **Schema-validate and parameterize all inputs** — JSON Schema / OpenAPI validation at API edges; parameterized queries / prepared statements for SQL; ORM-level validators on persisted models.

[preventive] **Sign and verify integrity-critical messages** — For commands or state mutations, attach an HMAC or signature so downstream consumers can detect tampering in transit or at rest.

[detective] **Use append-only / write-once stores for audit-critical data** — Where data must not be silently changed, write to an append-only log (e.g., immutable S3 bucket, CloudTrail-like store) and reconcile against the mutable copy.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-345 — Insufficient Verification of Data Authenticity](#)

Privilege transit across boundary: Agent Runtime / Orchestrator → Agent Tools Catalog [cross-boundary]

STRIDE → Elevation of Privilege | Component: **Agent Tools Catalog** (datastore) | Source: rule-based

CWE: CWE-441 — Confused Deputy

CVSS 3.1: 8.9 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:L

CVSS 4.0: 7.4 (High) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:H/SI:H/SA:L

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Agent Runtime / Orchestrator** → **Agent Tools Catalog** (label: *Agent Runtime / Orchestrator → Agent Tools Catalog*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Agent Tools Catalog** (`datastore`).

■ **Description:** If the receiver in 'Data tier' acts on behalf of the caller, attackers compromising 'Application tier' may inherit the receiver's privileges (confused-deputy).

■ ■ ■ Attack scenario:

1. Attacker compromises a low-privilege component in zone `Application tier`.
2. Issues requests to Agent Tools Catalog that get re-executed with the receiver's higher privileges (classic confused-deputy).
3. Because authorization is checked once at request entry but not per-action against the originating caller, the attacker effectively becomes a privileged actor in `Data tier`.
4. Attacker performs actions far beyond the original principal's permissions.

DREAD — D:10 R:8 E:9 A:9 D:9 → **Total 45/50**

■ How to mitigate:

[preventive] **Use scoped, short-lived delegation tokens** — When Agent Tools Catalog acts on behalf of a caller, attach a delegation token with the caller's identity, scope (action allow-list), and ≤ 5-min TTL.

[preventive] **Authorize per-action against the originating caller** — Don't rely on ambient authority of the receiver. Each action checks: (a) the delegation token is valid, (b) the original caller has permission for this specific action, (c) the action is within scope.

[detective] **Audit each privilege transit** — Log: original caller → delegating component → action performed. Make this queryable for incident response.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-441 — Confused Deputy](#)

Broken access control / missing authz check

STRIDE → Elevation of Privilege | Component: **Core Agent Layer** (api) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **Core Agent Layer** (`api`).

■ **Description:** Endpoints fail to verify the caller has permission for the action.

■ ■ Attack scenario:

1. Attacker authenticates to Core Agent Layer as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:9 R:10 E:10 A:10 D:10 → **Total 49/50**

■ How to mitigate:

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to Core Agent Layer should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Container / process escape

STRIDE → Elevation of Privilege | Component: **Core Agent Layer** (api) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **Core Agent Layer** (`api`).

■ **Description:** Attacker breaks out of a sandbox to gain host privileges.

■ ■ Attack scenario:

1. Attacker authenticates to Core Agent Layer as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:9 R:8 E:10 A:10 D:10 → **Total 47/50**

■ How to mitigate:

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to Core Agent Layer should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Privilege transit across boundary: Agent Gateway → Core Agent Layer [cross-boundary]

STRIDE → Elevation of Privilege | Component: **Core Agent Layer** (api) | Source: rule-based

CWE: CWE-441 — Confused Deputy

CVSS 3.1: 9.9 (Critical) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:L

CVSS 4.0: 7.4 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:H/SI:H/SA:L

Boundary crossing: DMZ (public-facing) → Application tier

■ **Location:** On the data flow **Agent Gateway → Core Agent Layer** (label: *Agent Gateway → Core Agent Layer*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `DMZ (public-facing)` into `Application tier`. The receiving component is **Core Agent Layer** (`api`).

■ **Description:** If the receiver in 'Application tier' acts on behalf of the caller, attackers compromising 'DMZ (public-facing)' may inherit the receiver's privileges (confused-deputy).

■ Attack scenario:

1. Attacker compromises a low-privilege component in zone `DMZ (public-facing)`.
2. Issues requests to Core Agent Layer that get re-executed with the receiver's higher privileges (classic confused-deputy).
3. Because authorization is checked once at request entry but not per-action against the originating caller, the attacker effectively becomes a privileged actor in `Application tier`.
4. Attacker performs actions far beyond the original principal's permissions.

DREAD — D:9 R:8 E:10 A:10 D:10 → **Total 47/50**

■ How to mitigate:

[preventive] **Use scoped, short-lived delegation tokens** — When Core Agent Layer acts on behalf of a caller, attach a delegation token with the caller's identity, scope (action allow-list), and ≤ 5-min TTL.

[preventive] **Authorize per-action against the originating caller** — Don't rely on ambient authority of the receiver. Each action checks: (a) the delegation token is valid, (b) the original caller has permission for this specific action, (c) the action is within scope.

[detective] **Audit each privilege transit** — Log: original caller → delegating component → action performed. Make this queryable for incident response.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-441 — Confused Deputy](#)

Privilege transit across boundary: AI Detection & Response → CrowdStrike AIDR [cross-boundary]

STRIDE → Elevation of Privilege | Component: **CrowdStrike AIDR** (external_entity) | Source: rule-based

CWE: CWE-441 — Confused Deputy

CVSS 3.1: 9.9 (Critical) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:L

CVSS 4.0: 7.4 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:H/SI:H/SA:L

Boundary crossing: Application tier → Internet

■ **Location:** On the data flow **AI Detection & Response → CrowdStrike AIDR** (label: *AI Detection & Response → CrowdStrike AIDR*, protocol: HTTPS, auth: api_key, encrypted: yes), crossing the trust boundary from `Application tier` into `Internet`. The receiving component is **CrowdStrike AIDR** (`external\_entity`).

■ **Description:** If the receiver in 'Internet' acts on behalf of the caller, attackers compromising 'Application tier' may inherit the receiver's privileges (confused-deputy).

■ Attack scenario:

1. Attacker compromises a low-privilege component in zone `Application tier`.
2. Issues requests to CrowdStrike AIDR that get re-executed with the receiver's higher privileges (classic confused-deputy).
3. Because authorization is checked once at request entry but not per-action against the originating caller, the attacker effectively

becomes a privileged actor in `Internet`.

4. Attacker performs actions far beyond the original principal's permissions.

DREAD — D:9 R:8 E:10 A:9 D:10 → **Total 46/50**

■ How to mitigate:

[preventive] **Use scoped, short-lived delegation tokens** — When CrowdStrike AIDR acts on behalf of a caller, attach a delegation token with the caller's identity, scope (action allow-list), and ≤ 5-min TTL.

[preventive] **Authorize per-action against the originating caller** — Don't rely on ambient authority of the receiver. Each action checks: (a) the delegation token is valid, (b) the original caller has permission for this specific action, (c) the action is within scope.

[detective] **Audit each privilege transit** — Log: original caller → delegating component → action performed. Make this queryable for incident response.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-441 — Confused Deputy](#)

Broken access control / missing authz check

STRIDE → Elevation of Privilege | Component: **Identity Provider / Okta MCP** (auth_service) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **Identity Provider / Okta MCP** (`auth\_service`).

■ **Description:** Endpoints fail to verify the caller has permission for the action.

■ Attack scenario:

1. Attacker authenticates to Identity Provider / Okta MCP as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:10 R:10 E:10 A:10 D:10 → **Total 50/50**

■ How to mitigate:

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to Identity Provider / Okta MCP should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Container / process escape

STRIDE → Elevation of Privilege | Component: **Identity Provider / Okta MCP** (auth_service) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **Identity Provider / Okta MCP** (`auth\_service`).

■ **Description:** Attacker breaks out of a sandbox to gain host privileges.

■ Attack scenario:

1. Attacker authenticates to Identity Provider / Okta MCP as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:10 R:8 E:10 A:10 D:10 → **Total 48/50**

■ How to mitigate:

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to Identity Provider / Okta MCP should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Privilege transit across boundary: Agent Gateway → Identity Provider / Okta MCP [cross-boundary]

STRIDE → Elevation of Privilege | Component: **Identity Provider / Okta MCP** (auth_service) | Source: rule-based

CWE: CWE-441 — Confused Deputy

CVSS 3.1: 9.9 (Critical) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:L

CVSS 4.0: 7.4 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:H/SI:H/SA:L

Boundary crossing: DMZ (public-facing) → Third-party / Untrusted services

■ **Location:** On the data flow **Agent Gateway → Identity Provider / Okta MCP** (label: *Agent Gateway → Identity Provider / Okta MCP*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `DMZ (public-facing)` into `Third-party / Untrusted services`. The receiving component is **Identity Provider / Okta MCP** (`auth\_service`).

■ **Description:** If the receiver in 'Third-party / Untrusted services' acts on behalf of the caller, attackers compromising 'DMZ (public-facing)' may inherit the receiver's privileges (confused-deputy).

■■ Attack scenario:

1. Attacker compromises a low-privilege component in zone `DMZ (public-facing)`.
2. Issues requests to Identity Provider / Okta MCP that get re-executed with the receiver's higher privileges (classic confused-deputy).
3. Because authorization is checked once at request entry but not per-action against the originating caller, the attacker effectively becomes a privileged actor in `Third-party / Untrusted services`.
4. Attacker performs actions far beyond the original principal's permissions.

DREAD — D:10 R:8 E:10 A:10 D:10 → **Total 48/50**

■ How to mitigate:

[preventive] **Use scoped, short-lived delegation tokens** — When Identity Provider / Okta MCP acts on behalf of a caller, attach a delegation token with the caller's identity, scope (action allow-list), and ≤ 5-min TTL.

[preventive] **Authorize per-action against the originating caller** — Don't rely on ambient authority of the receiver. Each action checks: (a) the delegation token is valid, (b) the original caller has permission for this specific action, (c) the action is within scope.

[detective] **Audit each privilege transit** — Log: original caller → delegating component → action performed. Make this queryable for incident response.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-441 — Confused Deputy](#)

Privilege transit across boundary: Model Gateway → LLM Models [cross-boundary]

STRIDE → Elevation of Privilege | Component: **LLM Models** (external_entity) | Source: rule-based

CWE: CWE-441 — Confused Deputy

CVSS 3.1: 9.9 (Critical) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:L

CVSS 4.0: 7.4 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:H/SI:H/SA:L

Boundary crossing: DMZ (public-facing) → Internet

■ **Location:** On the data flow **Model Gateway → LLM Models** (label: *Model Gateway → LLM Models*, protocol: HTTPS, auth: api_key, encrypted: yes), crossing the trust boundary from `DMZ (public-facing)` into `Internet`. The receiving component is **LLM Models** (`external\_entity`).

■ **Description:** If the receiver in 'Internet' acts on behalf of the caller, attackers compromising 'DMZ (public-facing)' may inherit the receiver's privileges (confused-deputy).

■ Attack scenario:

1. Attacker compromises a low-privilege component in zone `DMZ (public-facing)`.
2. Issues requests to LLM Models that get re-executed with the receiver's higher privileges (classic confused-deputy).
3. Because authorization is checked once at request entry but not per-action against the originating caller, the attacker effectively becomes a privileged actor in `Internet`.
4. Attacker performs actions far beyond the original principal's permissions.

DREAD — D:9 R:8 E:10 A:9 D:10 → **Total 46/50**

■ How to mitigate:

[preventive] **Use scoped, short-lived delegation tokens** — When LLM Models acts on behalf of a caller, attach a delegation token with the caller's identity, scope (action allow-list), and ≤ 5-min TTL.

[preventive] **Authorize per-action against the originating caller** — Don't rely on ambient authority of the receiver. Each action checks: (a) the delegation token is valid, (b) the original caller has permission for this specific action, (c) the action is within scope.

[detective] **Audit each privilege transit** — Log: original caller → delegating component → action performed. Make this queryable for incident response.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-441 — Confused Deputy](#)

Broken access control / missing authz check

STRIDE → Elevation of Privilege | Component: **Lambda Functions** (api) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **Lambda Functions** (`api`).

■ **Description:** Endpoints fail to verify the caller has permission for the action.

■ Attack scenario:

1. Attacker authenticates to Lambda Functions as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:9 R:10 E:9 A:9 D:9 → **Total 46/50**

■ How to mitigate:

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to Lambda Functions should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Container / process escape

STRIDE → Elevation of Privilege | Component: **Lambda Functions** (api) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **Lambda Functions** (`api`).

■ **Description:** Attacker breaks out of a sandbox to gain host privileges.

■ Attack scenario:

1. Attacker authenticates to Lambda Functions as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with

weak gating.

3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:9 R:8 E:9 A:9 D:9 → **Total 44/50**

■ How to mitigate:

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to Lambda Functions should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Stored data tampering via injection

STRIDE → Tampering | Component: **Long Term Memory** (datastore) | Source: rule-based

CWE: CWE-345 — Insufficient Verification of Data Authenticity

CVSS 3.1: 7.3 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Long Term Memory** (`datastore`).

■ **Description:** SQL/NoSQL/command injection alters records or schema.

■ Attack scenario:

1. Attacker reaches an input vector exposed by Long Term Memory (request body, query string, message queue payload, persisted state).
2. Submits malformed input designed to alter the program's logic or stored data (injection, parameter tampering, deserialization gadgets).
3. Long Term Memory processes the input without strict validation, and the malicious change takes effect.
4. Tampered data is persisted, executed, or relayed downstream — corrupting integrity beyond the original entry point.

DREAD — D:10 R:10 E:9 A:7 D:9 → **Total 45/50**

■ How to mitigate:

[preventive] **Schema-validate and parameterize all inputs** — JSON Schema / OpenAPI validation at API edges; parameterized queries / prepared statements for SQL; ORM-level validators on persisted models.

[preventive] **Sign and verify integrity-critical messages** — For commands or state mutations, attach an HMAC or signature so downstream consumers can detect tampering in transit or at rest.

[detective] **Use append-only / write-once stores for audit-critical data** — Where data must not be silently changed, write to an append-only log (e.g., immutable S3 bucket, CloudTrail-like store) and reconcile against the mutable copy.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-345 — Insufficient Verification of Data Authenticity](#)

Privilege transit across boundary: Agent Runtime / Orchestrator → Long Term Memory [cross-boundary]

STRIDE → Elevation of Privilege | Component: **Long Term Memory** (datastore) | Source: rule-based

CWE: CWE-441 — Confused Deputy

CVSS 3.1: 8.9 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:L

CVSS 4.0: 7.4 (High) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:H/SI:H/SA:L

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Agent Runtime / Orchestrator** → **Long Term Memory** (label: *Agent Runtime / Orchestrator → Long Term Memory*, protocol: TCP, auth: credentials, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Long Term Memory** (`datastore`).

■ **Description:** If the receiver in 'Data tier' acts on behalf of the caller, attackers compromising 'Application tier' may inherit the receiver's privileges (confused-deputy).

■ Attack scenario:

1. Attacker compromises a low-privilege component in zone `Application tier`.
2. Issues requests to Long Term Memory that get re-executed with the receiver's higher privileges (classic confused-deputy).
3. Because authorization is checked once at request entry but not per-action against the originating caller, the attacker effectively becomes a privileged actor in `Data tier`.
4. Attacker performs actions far beyond the original principal's permissions.

DREAD — D:10 R:8 E:9 A:9 D:9 → **Total 45/50**

■ How to mitigate:

[preventive] **Use scoped, short-lived delegation tokens** — When Long Term Memory acts on behalf of a caller, attach a delegation token with the caller's identity, scope (action allow-list), and ≤ 5-min TTL.

[preventive] **Authorize per-action against the originating caller** — Don't rely on ambient authority of the receiver. Each action checks: (a) the delegation token is valid, (b) the original caller has permission for this specific action, (c) the action is within scope.

[detective] **Audit each privilege transit** — Log: original caller → delegating component → action performed. Make this queryable for incident response.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-441 — Confused Deputy](#)

Broken access control / missing authz check

STRIDE → Elevation of Privilege | Component: **MCP Server Hosting** (api) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **MCP Server Hosting** (`api`).

■ **Description:** Endpoints fail to verify the caller has permission for the action.

■ Attack scenario:

1. Attacker authenticates to MCP Server Hosting as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:9 R:10 E:9 A:10 D:9 → **Total 47/50**

■ How to mitigate:

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to MCP Server Hosting should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Container / process escape

STRIDE → Elevation of Privilege | Component: **MCP Server Hosting** (api) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **MCP Server Hosting** (`api`).

■ **Description:** Attacker breaks out of a sandbox to gain host privileges.

■ Attack scenario:

1. Attacker authenticates to MCP Server Hosting as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with

weak gating.

3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:9 R:8 E:9 A:10 D:9 → **Total 45/50**

■ How to mitigate:

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to MCP Server Hosting should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Privilege transit across boundary: MCP Server Hosting → MCP Server Targets [cross-boundary]

STRIDE → Elevation of Privilege | Component: **MCP Server Targets** (external_entity) | Source: rule-based

CWE: CWE-441 — Confused Deputy

CVSS 3.1: 9.9 (Critical) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:L

CVSS 4.0: 7.4 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:H/SI:H/SA:L

Boundary crossing: Application tier → Internet

■ **Location:** On the data flow **MCP Server Hosting** → **MCP Server Targets** (label: *MCP Server Hosting → MCP Server Targets*, protocol: HTTPS, auth: mtls, encrypted: yes), crossing the trust boundary from 'Application tier' into 'Internet'. The receiving component is **MCP Server Targets** ('external_entity').

■ **Description:** If the receiver in 'Internet' acts on behalf of the caller, attackers compromising 'Application tier' may inherit the receiver's privileges (confused-deputy).

■■ Attack scenario:

1. Attacker compromises a low-privilege component in zone 'Application tier'.
2. Issues requests to MCP Server Targets that get re-executed with the receiver's higher privileges (classic confused-deputy).
3. Because authorization is checked once at request entry but not per-action against the originating caller, the attacker effectively becomes a privileged actor in 'Internet'.
4. Attacker performs actions far beyond the original principal's permissions.

DREAD — D:9 R:8 E:9 A:9 D:10 → **Total 45/50**

■ How to mitigate:

[preventive] **Use scoped, short-lived delegation tokens** — When MCP Server Targets acts on behalf of a caller, attach a delegation token with the caller's identity, scope (action allow-list), and ≤ 5-min TTL.

[preventive] **Authorize per-action against the originating caller** — Don't rely on ambient authority of the receiver. Each action checks: (a) the delegation token is valid, (b) the original caller has permission for this specific action, (c) the action is within scope.

[detective] **Audit each privilege transit** — Log: original caller → delegating component → action performed. Make this queryable for incident response.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-441 — Confused Deputy](#)

Stored data tampering via injection

STRIDE → Tampering | Component: **Metrics & Tracing** (datastore) | Source: rule-based

CWE: CWE-345 — Insufficient Verification of Data Authenticity

CVSS 3.1: 7.3 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Metrics & Tracing** ('datastore').

■ **Description:** SQL/NoSQL/command injection alters records or schema.

■■ Attack scenario:

1. Attacker reaches an input vector exposed by Metrics & Tracing (request body, query string, message queue payload, persisted

state).

2. Submits malformed input designed to alter the program's logic or stored data (injection, parameter tampering, deserialization gadgets).
3. Metrics & Tracing processes the input without strict validation, and the malicious change takes effect.
4. Tampered data is persisted, executed, or relayed downstream — corrupting integrity beyond the original entry point.

DREAD — D:10 R:10 E:9 A:7 D:9 → **Total 45/50**

■ How to mitigate:

[preventive] **Schema-validate and parameterize all inputs** — JSON Schema / OpenAPI validation at API edges; parameterized queries / prepared statements for SQL; ORM-level validators on persisted models.

[preventive] **Sign and verify integrity-critical messages** — For commands or state mutations, attach an HMAC or signature so downstream consumers can detect tampering in transit or at rest.

[detective] **Use append-only / write-once stores for audit-critical data** — Where data must not be silently changed, write to an append-only log (e.g., immutable S3 bucket, CloudTrail-like store) and reconcile against the mutable copy.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-345 — Insufficient Verification of Data Authenticity](#)

Privilege transit across boundary: Core Agent Layer → Metrics & Tracing [cross-boundary]

STRIDE → Elevation of Privilege | Component: **Metrics & Tracing** (datastore) | Source: rule-based

CWE: CWE-441 — Confused Deputy

CVSS 3.1: 8.9 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:L

CVSS 4.0: 7.4 (High) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:H/SI:H/SA:L

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Core Agent Layer** → **Metrics & Tracing** (label: *Core Agent Layer → Metrics & Tracing*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Metrics & Tracing** (`datastore`).

■ **Description:** If the receiver in 'Data tier' acts on behalf of the caller, attackers compromising 'Application tier' may inherit the receiver's privileges (confused-deputy).

■■ Attack scenario:

1. Attacker compromises a low-privilege component in zone `Application tier`.
2. Issues requests to Metrics & Tracing that get re-executed with the receiver's higher privileges (classic confused-deputy).
3. Because authorization is checked once at request entry but not per-action against the originating caller, the attacker effectively becomes a privileged actor in `Data tier`.
4. Attacker performs actions far beyond the original principal's permissions.

DREAD — D:10 R:8 E:9 A:9 D:9 → **Total 45/50**

■ How to mitigate:

[preventive] **Use scoped, short-lived delegation tokens** — When Metrics & Tracing acts on behalf of a caller, attach a delegation token with the caller's identity, scope (action allow-list), and ≤ 5-min TTL.

[preventive] **Authorize per-action against the originating caller** — Don't rely on ambient authority of the receiver. Each action checks: (a) the delegation token is valid, (b) the original caller has permission for this specific action, (c) the action is within scope.

[detective] **Audit each privilege transit** — Log: original caller → delegating component → action performed. Make this queryable for incident response.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-441 — Confused Deputy](#)

Broken access control / missing authz check

STRIDE → Elevation of Privilege | Component: **Model Gateway** (api) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **Model Gateway** (`api`).

■ **Description:** Endpoints fail to verify the caller has permission for the action.

■ Attack scenario:

1. Attacker authenticates to Model Gateway as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:9 R:10 E:10 A:10 D:10 → **Total 49/50**

■ How to mitigate:

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to Model Gateway should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Container / process escape

STRIDE → Elevation of Privilege | Component: **Model Gateway** (api) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **Model Gateway** (`api`).

■ **Description:** Attacker breaks out of a sandbox to gain host privileges.

■ Attack scenario:

1. Attacker authenticates to Model Gateway as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:9 R:8 E:10 A:10 D:10 → **Total 47/50**

■ How to mitigate:

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to Model Gateway should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Privilege transit across boundary: Agent Runtime / Orchestrator → Model Gateway [cross-boundary]

STRIDE → Elevation of Privilege | Component: **Model Gateway** (api) | Source: rule-based

CWE: CWE-441 — Confused Deputy

CVSS 3.1: 9.9 (Critical) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:L

CVSS 4.0: 7.4 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:H/SI:H/SA:L

Boundary crossing: Application tier → DMZ (public-facing)

■ **Location:** On the data flow **Agent Runtime / Orchestrator** → **Model Gateway** (label: *Agent Runtime / Orchestrator → Model Gateway*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `Application tier` into `DMZ (public-facing)`. The receiving component is **Model Gateway** (`api`).

■ **Description:** If the receiver in 'DMZ (public-facing)' acts on behalf of the caller, attackers compromising 'Application tier' may inherit the receiver's privileges (confused-deputy).

■ ■ **Attack scenario:**

1. Attacker compromises a low-privilege component in zone 'Application tier'.
2. Issues requests to Model Gateway that get re-executed with the receiver's higher privileges (classic confused-deputy).
3. Because authorization is checked once at request entry but not per-action against the originating caller, the attacker effectively becomes a privileged actor in 'DMZ (public-facing)'.
4. Attacker performs actions far beyond the original principal's permissions.

DREAD — D:9 R:8 E:10 A:10 D:10 → **Total 47/50**

■ **How to mitigate:**

[preventive] **Use scoped, short-lived delegation tokens** — When Model Gateway acts on behalf of a caller, attach a delegation token with the caller's identity, scope (action allow-list), and ≤ 5-min TTL.

[preventive] **Authorize per-action against the originating caller** — Don't rely on ambient authority of the receiver. Each action checks: (a) the delegation token is valid, (b) the original caller has permission for this specific action, (c) the action is within scope.

[detective] **Audit each privilege transit** — Log: original caller → delegating component → action performed. Make this queryable for incident response.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-441 — Confused Deputy](#)

Broken access control / missing authz check

STRIDE → Elevation of Privilege | Component: **Planning & Workflow** (api) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **Planning & Workflow** ('api').

■ **Description:** Endpoints fail to verify the caller has permission for the action.

■ ■ **Attack scenario:**

1. Attacker authenticates to Planning & Workflow as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:9 R:10 E:9 A:9 D:9 → **Total 46/50**

■ **How to mitigate:**

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to Planning & Workflow should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Container / process escape

STRIDE → Elevation of Privilege | Component: **Planning & Workflow** (api) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **Planning & Workflow** ('api').

■ **Description:** Attacker breaks out of a sandbox to gain host privileges.

■ Attack scenario:

1. Attacker authenticates to Planning & Workflow as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:9 R:8 E:9 A:9 D:9 → **Total 44/50**

■ How to mitigate:

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to Planning & Workflow should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Stored data tampering via injection

STRIDE → Tampering | Component: **Prompt & Cyber Logging** (datastore) | Source: rule-based

CWE: CWE-345 — Insufficient Verification of Data Authenticity

CVSS 3.1: 7.3 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Prompt & Cyber Logging** (`datastore`).

■ **Description:** SQL/NoSQL/command injection alters records or schema.

■ Attack scenario:

1. Attacker reaches an input vector exposed by Prompt & Cyber Logging (request body, query string, message queue payload, persisted state).
2. Submits malformed input designed to alter the program's logic or stored data (injection, parameter tampering, deserialization gadgets).
3. Prompt & Cyber Logging processes the input without strict validation, and the malicious change takes effect.
4. Tampered data is persisted, executed, or relayed downstream — corrupting integrity beyond the original entry point.

DREAD — D:10 R:10 E:9 A:7 D:9 → **Total 45/50**

■ How to mitigate:

[preventive] **Schema-validate and parameterize all inputs** — JSON Schema / OpenAPI validation at API edges; parameterized queries / prepared statements for SQL; ORM-level validators on persisted models.

[preventive] **Sign and verify integrity-critical messages** — For commands or state mutations, attach an HMAC or signature so downstream consumers can detect tampering in transit or at rest.

[detective] **Use append-only / write-once stores for audit-critical data** — Where data must not be silently changed, write to an append-only log (e.g., immutable S3 bucket, CloudTrail-like store) and reconcile against the mutable copy.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-345 — Insufficient Verification of Data Authenticity](#)

Privilege transit across boundary: Core Agent Layer → Prompt & Cyber Logging [cross-boundary]

STRIDE → Elevation of Privilege | Component: **Prompt & Cyber Logging** (datastore) | Source: rule-based

CWE: CWE-441 — Confused Deputy

CVSS 3.1: 8.9 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:L

CVSS 4.0: 7.4 (High) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:H/SI:H/SA:L

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Core Agent Layer** → **Prompt & Cyber Logging** (label: *Core Agent Layer → Prompt & Cyber Logging*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Prompt & Cyber Logging** (`datastore`).

■ **Description:** If the receiver in 'Data tier' acts on behalf of the caller, attackers compromising 'Application tier' may inherit the receiver's privileges (confused-deputy).

■ ■ **Attack scenario:**

1. Attacker compromises a low-privilege component in zone `Application tier`.
2. Issues requests to Prompt & Cyber Logging that get re-executed with the receiver's higher privileges (classic confused-deputy).
3. Because authorization is checked once at request entry but not per-action against the originating caller, the attacker effectively becomes a privileged actor in `Data tier`.
4. Attacker performs actions far beyond the original principal's permissions.

DREAD — D:10 R:8 E:9 A:9 D:9 → **Total 45/50**

■ **How to mitigate:**

[preventive] **Use scoped, short-lived delegation tokens** — When Prompt & Cyber Logging acts on behalf of a caller, attach a delegation token with the caller's identity, scope (action allow-list), and ≤ 5-min TTL.

[preventive] **Authorize per-action against the originating caller** — Don't rely on ambient authority of the receiver. Each action checks: (a) the delegation token is valid, (b) the original caller has permission for this specific action, (c) the action is within scope.

[detective] **Audit each privilege transit** — Log: original caller → delegating component → action performed. Make this queryable for incident response.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-441 — Confused Deputy](#)

Broken access control / missing authz check

STRIDE → Elevation of Privilege | Component: **Retrieval (RAG)** (api) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **Retrieval (RAG)** (`api`).

■ **Description:** Endpoints fail to verify the caller has permission for the action.

■ ■ **Attack scenario:**

1. Attacker authenticates to Retrieval (RAG) as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:9 R:10 E:9 A:10 D:9 → **Total 47/50**

■ **How to mitigate:**

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to Retrieval (RAG) should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Container / process escape

STRIDE → Elevation of Privilege | Component: **Retrieval (RAG)** (api) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **Retrieval (RAG)** (`api`).

■ **Description:** Attacker breaks out of a sandbox to gain host privileges.

■ Attack scenario:

1. Attacker authenticates to Retrieval (RAG) as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:9 R:8 E:9 A:10 D:9 → **Total 45/50**

■ How to mitigate:

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to Retrieval (RAG) should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Broken access control / missing authz check

STRIDE → Elevation of Privilege | Component: **SailPoint Agent Identity** (auth_service) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **SailPoint Agent Identity** (auth_service).

■ **Description:** Endpoints fail to verify the caller has permission for the action.

■ Attack scenario:

1. Attacker authenticates to SailPoint Agent Identity as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:10 R:10 E:10 A:9 D:10 → **Total 49/50**

■ How to mitigate:

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to SailPoint Agent Identity should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Container / process escape

STRIDE → Elevation of Privilege | Component: **SailPoint Agent Identity** (auth_service) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **SailPoint Agent Identity** (auth_service).

■ **Description:** Attacker breaks out of a sandbox to gain host privileges.

■ Attack scenario:

1. Attacker authenticates to SailPoint Agent Identity as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with

weak gating.

3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:10 R:8 E:10 A:9 D:10 → **Total 47/50**

■ How to mitigate:

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to SailPoint Agent Identity should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Privilege transit across boundary: Agent Gateway → SailPoint Agent Identity [cross-boundary]

STRIDE → Elevation of Privilege | Component: **SailPoint Agent Identity** (auth_service) | Source: rule-based

CWE: CWE-441 — Confused Deputy

CVSS 3.1: 9.9 (Critical) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:L

CVSS 4.0: 7.4 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:H/SI:H/SA:L

Boundary crossing: **DMZ (public-facing) → Application tier**

■ **Location:** On the data flow **Agent Gateway → SailPoint Agent Identity** (label: *Agent Gateway → SailPoint Agent Identity*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `DMZ (public-facing)` into `Application tier`. The receiving component is **SailPoint Agent Identity** (`auth\_service`).

■ **Description:** If the receiver in 'Application tier' acts on behalf of the caller, attackers compromising 'DMZ (public-facing)' may inherit the receiver's privileges (confused-deputy).

■■ Attack scenario:

1. Attacker compromises a low-privilege component in zone `DMZ (public-facing)`.
2. Issues requests to SailPoint Agent Identity that get re-executed with the receiver's higher privileges (classic confused-deputy).
3. Because authorization is checked once at request entry but not per-action against the originating caller, the attacker effectively becomes a privileged actor in `Application tier`.
4. Attacker performs actions far beyond the original principal's permissions.

DREAD — D:10 R:8 E:10 A:10 D:10 → **Total 48/50**

■ How to mitigate:

[preventive] **Use scoped, short-lived delegation tokens** — When SailPoint Agent Identity acts on behalf of a caller, attach a delegation token with the caller's identity, scope (action allow-list), and ≤ 5-min TTL.

[preventive] **Authorize per-action against the originating caller** — Don't rely on ambient authority of the receiver. Each action checks: (a) the delegation token is valid, (b) the original caller has permission for this specific action, (c) the action is within scope.

[detective] **Audit each privilege transit** — Log: original caller → delegating component → action performed. Make this queryable for incident response.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-441 — Confused Deputy](#)

Stored data tampering via injection

STRIDE → Tampering | Component: **Session Memory** (datastore) | Source: rule-based

CWE: CWE-345 — Insufficient Verification of Data Authenticity

CVSS 3.1: 7.3 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Session Memory** (`datastore`).

■ **Description:** SQL/NoSQL/command injection alters records or schema.

■■ Attack scenario:

1. Attacker reaches an input vector exposed by Session Memory (request body, query string, message queue payload, persisted

state).

2. Submits malformed input designed to alter the program's logic or stored data (injection, parameter tampering, deserialization gadgets).
3. Session Memory processes the input without strict validation, and the malicious change takes effect.
4. Tampered data is persisted, executed, or relayed downstream — corrupting integrity beyond the original entry point.

DREAD — D:10 R:10 E:9 A:7 D:9 → **Total 45/50**

■ How to mitigate:

[preventive] **Schema-validate and parameterize all inputs** — JSON Schema / OpenAPI validation at API edges; parameterized queries / prepared statements for SQL; ORM-level validators on persisted models.

[preventive] **Sign and verify integrity-critical messages** — For commands or state mutations, attach an HMAC or signature so downstream consumers can detect tampering in transit or at rest.

[detective] **Use append-only / write-once stores for audit-critical data** — Where data must not be silently changed, write to an append-only log (e.g., immutable S3 bucket, CloudTrail-like store) and reconcile against the mutable copy.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-345 — Insufficient Verification of Data Authenticity](#)

Privilege transit across boundary: Agent Runtime / Orchestrator → Session Memory [cross-boundary]

STRIDE → Elevation of Privilege | Component: **Session Memory** (datastore) | Source: rule-based

CWE: CWE-441 — Confused Deputy

CVSS 3.1: 8.9 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:L

CVSS 4.0: 7.4 (High) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:H/SI:H/SA:L

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Agent Runtime / Orchestrator** → **Session Memory** (label: *Agent Runtime / Orchestrator → Session Memory*, protocol: TCP, auth: credentials, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Session Memory** (`datastore`).

■ **Description:** If the receiver in 'Data tier' acts on behalf of the caller, attackers compromising 'Application tier' may inherit the receiver's privileges (confused-deputy).

■■ Attack scenario:

1. Attacker compromises a low-privilege component in zone `Application tier`.
2. Issues requests to Session Memory that get re-executed with the receiver's higher privileges (classic confused-deputy).
3. Because authorization is checked once at request entry but not per-action against the originating caller, the attacker effectively becomes a privileged actor in `Data tier`.
4. Attacker performs actions far beyond the original principal's permissions.

DREAD — D:10 R:8 E:9 A:9 D:9 → **Total 45/50**

■ How to mitigate:

[preventive] **Use scoped, short-lived delegation tokens** — When Session Memory acts on behalf of a caller, attach a delegation token with the caller's identity, scope (action allow-list), and ≤ 5-min TTL.

[preventive] **Authorize per-action against the originating caller** — Don't rely on ambient authority of the receiver. Each action checks: (a) the delegation token is valid, (b) the original caller has permission for this specific action, (c) the action is within scope.

[detective] **Audit each privilege transit** — Log: original caller → delegating component → action performed. Make this queryable for incident response.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-441 — Confused Deputy](#)

Stored data tampering via injection

STRIDE → Tampering | Component: **Short Term Memory** (datastore) | Source: rule-based

CWE: CWE-345 — Insufficient Verification of Data Authenticity

CVSS 3.1: 7.3 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Short Term Memory** (`datastore`).

■ **Description:** SQL/NoSQL/command injection alters records or schema.

■ Attack scenario:

1. Attacker reaches an input vector exposed by Short Term Memory (request body, query string, message queue payload, persisted state).
2. Submits malformed input designed to alter the program's logic or stored data (injection, parameter tampering, deserialization gadgets).
3. Short Term Memory processes the input without strict validation, and the malicious change takes effect.
4. Tampered data is persisted, executed, or relayed downstream — corrupting integrity beyond the original entry point.

DREAD — D:10 R:10 E:9 A:7 D:9 → **Total 45/50**

■ How to mitigate:

[preventive] **Schema-validate and parameterize all inputs** — JSON Schema / OpenAPI validation at API edges; parameterized queries / prepared statements for SQL; ORM-level validators on persisted models.

[preventive] **Sign and verify integrity-critical messages** — For commands or state mutations, attach an HMAC or signature so downstream consumers can detect tampering in transit or at rest.

[detective] **Use append-only / write-once stores for audit-critical data** — Where data must not be silently changed, write to an append-only log (e.g., immutable S3 bucket, CloudTrail-like store) and reconcile against the mutable copy.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-345 — Insufficient Verification of Data Authenticity](#)

Privilege transit across boundary: Agent Runtime / Orchestrator → Short Term Memory [cross-boundary]

STRIDE → Elevation of Privilege | Component: **Short Term Memory** (datastore) | Source: rule-based

CWE: CWE-441 — Confused Deputy

CVSS 3.1: 8.9 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:L

CVSS 4.0: 7.4 (High) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:H/SI:H/SA:L

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Agent Runtime / Orchestrator** → **Short Term Memory** (label: *Agent Runtime / Orchestrator → Short Term Memory*, protocol: TCP, auth: credentials, encrypted: yes), crossing the trust boundary from 'Application tier' into 'Data tier'. The receiving component is **Short Term Memory** ('datastore').

■ **Description:** If the receiver in 'Data tier' acts on behalf of the caller, attackers compromising 'Application tier' may inherit the receiver's privileges (confused-deputy).

■ Attack scenario:

1. Attacker compromises a low-privilege component in zone 'Application tier'.
2. Issues requests to Short Term Memory that get re-executed with the receiver's higher privileges (classic confused-deputy).
3. Because authorization is checked once at request entry but not per-action against the originating caller, the attacker effectively becomes a privileged actor in 'Data tier'.
4. Attacker performs actions far beyond the original principal's permissions.

DREAD — D:10 R:8 E:9 A:9 D:9 → **Total 45/50**

■ How to mitigate:

[preventive] **Use scoped, short-lived delegation tokens** — When Short Term Memory acts on behalf of a caller, attach a delegation token with the caller's identity, scope (action allow-list), and ≤ 5-min TTL.

[preventive] **Authorize per-action against the originating caller** — Don't rely on ambient authority of the receiver. Each action checks: (a) the delegation token is valid, (b) the original caller has permission for this specific action, (c) the action is within scope.

[detective] **Audit each privilege transit** — Log: original caller → delegating component → action performed. Make this queryable for incident response.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-441 — Confused Deputy](#)

Stored data tampering via injection

STRIDE → Tampering | Component: **State Tracking Services** (datastore) | Source: rule-based

CWE: CWE-345 — Insufficient Verification of Data Authenticity

CVSS 3.1: 7.3 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **State Tracking Services** ('datastore').

■ **Description:** SQL/NoSQL/command injection alters records or schema.

■ ■ **Attack scenario:**

1. Attacker reaches an input vector exposed by State Tracking Services (request body, query string, message queue payload, persisted state).
2. Submits malformed input designed to alter the program's logic or stored data (injection, parameter tampering, deserialization gadgets).
3. State Tracking Services processes the input without strict validation, and the malicious change takes effect.
4. Tampered data is persisted, executed, or relayed downstream — corrupting integrity beyond the original entry point.

DREAD — D:10 R:10 E:9 A:7 D:9 → **Total 45/50**

■ **How to mitigate:**

[preventive] **Schema-validate and parameterize all inputs** — JSON Schema / OpenAPI validation at API edges; parameterized queries / prepared statements for SQL; ORM-level validators on persisted models.

[preventive] **Sign and verify integrity-critical messages** — For commands or state mutations, attach an HMAC or signature so downstream consumers can detect tampering in transit or at rest.

[detective] **Use append-only / write-once stores for audit-critical data** — Where data must not be silently changed, write to an append-only log (e.g., immutable S3 bucket, CloudTrail-like store) and reconcile against the mutable copy.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-345 — Insufficient Verification of Data Authenticity](#)

Privilege transit across boundary: Agent Runtime / Orchestrator → State Tracking Services [cross-boundary]

STRIDE → Elevation of Privilege | Component: **State Tracking Services** (datastore) | Source: rule-based

CWE: CWE-441 — Confused Deputy

CVSS 3.1: 8.9 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:L

CVSS 4.0: 7.4 (High) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:H/SI:H/SA:L

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Agent Runtime / Orchestrator → State Tracking Services** (label: *Agent Runtime / Orchestrator → State Tracking Services*, protocol: TCP, auth: credentials, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **State Tracking Services** (`datastore`).

■ **Description:** If the receiver in 'Data tier' acts on behalf of the caller, attackers compromising 'Application tier' may inherit the receiver's privileges (confused-deputy).

■ ■ **Attack scenario:**

1. Attacker compromises a low-privilege component in zone `Application tier`.
2. Issues requests to State Tracking Services that get re-executed with the receiver's higher privileges (classic confused-deputy).
3. Because authorization is checked once at request entry but not per-action against the originating caller, the attacker effectively becomes a privileged actor in `Data tier`.
4. Attacker performs actions far beyond the original principal's permissions.

DREAD — D:10 R:8 E:9 A:9 D:9 → **Total 45/50**

■ **How to mitigate:**

[preventive] **Use scoped, short-lived delegation tokens** — When State Tracking Services acts on behalf of a caller, attach a delegation token with the caller's identity, scope (action allow-list), and ≤ 5-min TTL.

[preventive] **Authorize per-action against the originating caller** — Don't rely on ambient authority of the receiver. Each action checks: (a) the delegation token is valid, (b) the original caller has permission for this specific action, (c) the action is within scope.

[detective] **Audit each privilege transit** — Log: original caller → delegating component → action performed. Make this queryable for incident response.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-441 — Confused Deputy](#)

Privilege transit across boundary: Agent Runtime / Orchestrator → Tools (Internal & External)
[cross-boundary]

STRIDE → Elevation of Privilege | Component: **Tools (Internal & External)** (external_entity) | Source: rule-based

CWE: CWE-441 — Confused Deputy

CVSS 3.1: 9.9 (Critical) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:L

CVSS 4.0: 7.4 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:H/SI:H/SA:L](#)

Boundary crossing: Application tier → Internet

■ **Location:** On the data flow **Agent Runtime / Orchestrator** → **Tools (Internal & External)** (label: *Agent Runtime / Orchestrator → Tools (Internal & External)*, protocol: HTTPS, auth: api_key, encrypted: yes), crossing the trust boundary from `Application tier` into `Internet`. The receiving component is **Tools (Internal & External)** (`external\_entity`).

■ **Description:** If the receiver in 'Internet' acts on behalf of the caller, attackers compromising 'Application tier' may inherit the receiver's privileges (confused-deputy).

■ ■ **Attack scenario:**

1. Attacker compromises a low-privilege component in zone `Application tier`.
2. Issues requests to Tools (Internal & External) that get re-executed with the receiver's higher privileges (classic confused-deputy).
3. Because authorization is checked once at request entry but not per-action against the originating caller, the attacker effectively becomes a privileged actor in `Internet`.
4. Attacker performs actions far beyond the original principal's permissions.

DREAD — D:9 R:8 E:10 A:9 D:10 → **Total 46/50**

■ **How to mitigate:**

[preventive] **Use scoped, short-lived delegation tokens** — When Tools (Internal & External) acts on behalf of a caller, attach a delegation token with the caller's identity, scope (action allow-list), and ≤ 5-min TTL.

[preventive] **Authorize per-action against the originating caller** — Don't rely on ambient authority of the receiver. Each action checks: (a) the delegation token is valid, (b) the original caller has permission for this specific action, (c) the action is within scope.

[detective] **Audit each privilege transit** — Log: original caller → delegating component → action performed. Make this queryable for incident response.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-441 — Confused Deputy](#)

Stored data tampering via injection

[STRIDE](#) → Tampering | Component: **Vector DB** (database) | Source: rule-based

CWE: CWE-345 — Insufficient Verification of Data Authenticity

CVSS 3.1: 7.3 (High) [CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N](#)

CVSS 4.0: 5.2 (Medium) [CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Vector DB** (`database`).

■ **Description:** SQL/NoSQL/command injection alters records or schema.

■ ■ **Attack scenario:**

1. Attacker reaches an input vector exposed by Vector DB (request body, query string, message queue payload, persisted state).
2. Submits malformed input designed to alter the program's logic or stored data (injection, parameter tampering, deserialization gadgets).
3. Vector DB processes the input without strict validation, and the malicious change takes effect.
4. Tampered data is persisted, executed, or relayed downstream — corrupting integrity beyond the original entry point.

DREAD — D:10 R:10 E:9 A:9 D:9 → **Total 47/50**

■ **How to mitigate:**

[preventive] **Schema-validate and parameterize all inputs** — JSON Schema / OpenAPI validation at API edges; parameterized queries / prepared statements for SQL; ORM-level validators on persisted models.

[preventive] **Sign and verify integrity-critical messages** — For commands or state mutations, attach an HMAC or signature so downstream consumers can detect tampering in transit or at rest.

[detective] **Use append-only / write-once stores for audit-critical data** — Where data must not be silently changed, write to an append-only log (e.g., immutable S3 bucket, CloudTrail-like store) and reconcile against the mutable copy.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-345 — Insufficient Verification of Data Authenticity](#)

Broken access control / missing authz check

STRIDE → Elevation of Privilege | Component: **Vector DB** (database) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 7.6 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **Vector DB** (`database`).

■ **Description:** Endpoints fail to verify the caller has permission for the action.

■ ■ **Attack scenario:**

1. Attacker authenticates to Vector DB as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:10 R:10 E:9 A:9 D:9 → **Total 47/50**

■ **How to mitigate:**

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to Vector DB should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Container / process escape

STRIDE → Elevation of Privilege | Component: **Vector DB** (database) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 7.6 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **Vector DB** (`database`).

■ **Description:** Attacker breaks out of a sandbox to gain host privileges.

■ ■ **Attack scenario:**

1. Attacker authenticates to Vector DB as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:10 R:8 E:9 A:9 D:9 → **Total 45/50**

■ **How to mitigate:**

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to Vector DB should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Privilege transit across boundary: Retrieval (RAG) → Vector DB [cross-boundary]

STRIDE → Elevation of Privilege | Component: **Vector DB** (database) | Source: rule-based

CWE: CWE-441 — Confused Deputy

CVSS 3.1: 8.9 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:L

CVSS 4.0: 7.4 (High) [CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:H/SI:H/SA:L](#)

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Retrieval (RAG) → Vector DB** (label: *Retrieval (RAG) → Vector DB*, protocol: TCP, auth: credentials, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Vector DB** (`database`).

■ **Description:** If the receiver in 'Data tier' acts on behalf of the caller, attackers compromising 'Application tier' may inherit the receiver's privileges (confused-deputy).

■ ■ **Attack scenario:**

1. Attacker compromises a low-privilege component in zone `Application tier`.
2. Issues requests to Vector DB that get re-executed with the receiver's higher privileges (classic confused-deputy).
3. Because authorization is checked once at request entry but not per-action against the originating caller, the attacker effectively becomes a privileged actor in `Data tier`.
4. Attacker performs actions far beyond the original principal's permissions.

DREAD — D:10 R:8 E:9 A:10 D:9 → **Total 46/50**

■ **How to mitigate:**

[preventive] **Use scoped, short-lived delegation tokens** — When Vector DB acts on behalf of a caller, attach a delegation token with the caller's identity, scope (action allow-list), and ≤ 5-min TTL.

[preventive] **Authorize per-action against the originating caller** — Don't rely on ambient authority of the receiver. Each action checks: (a) the delegation token is valid, (b) the original caller has permission for this specific action, (c) the action is within scope.

[detective] **Audit each privilege transit** — Log: original caller → delegating component → action performed. Make this queryable for incident response.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-441 — Confused Deputy](#)

Privilege transit across boundary: Agent Runtime / Orchestrator → Web Search [cross-boundary]

STRIDE → Elevation of Privilege | Component: **Web Search** (external_entity) | Source: rule-based

CWE: CWE-441 — Confused Deputy

CVSS 3.1: 9.9 (Critical) [CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:L](#)

CVSS 4.0: 7.4 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:H/SI:H/SA:L](#)

Boundary crossing: Application tier → Internet

■ **Location:** On the data flow **Agent Runtime / Orchestrator → Web Search** (label: *Agent Runtime / Orchestrator → Web Search*, protocol: HTTPS, auth: api_key, encrypted: yes), crossing the trust boundary from `Application tier` into `Internet`. The receiving component is **Web Search** (`external\_entity`).

■ **Description:** If the receiver in 'Internet' acts on behalf of the caller, attackers compromising 'Application tier' may inherit the receiver's privileges (confused-deputy).

■ ■ **Attack scenario:**

1. Attacker compromises a low-privilege component in zone `Application tier`.
2. Issues requests to Web Search that get re-executed with the receiver's higher privileges (classic confused-deputy).
3. Because authorization is checked once at request entry but not per-action against the originating caller, the attacker effectively becomes a privileged actor in `Internet`.
4. Attacker performs actions far beyond the original principal's permissions.

DREAD — D:9 R:8 E:10 A:9 D:10 → **Total 46/50**

■ **How to mitigate:**

[preventive] **Use scoped, short-lived delegation tokens** — When Web Search acts on behalf of a caller, attach a delegation token with the caller's identity, scope (action allow-list), and ≤ 5-min TTL.

[preventive] **Authorize per-action against the originating caller** — Don't rely on ambient authority of the receiver. Each action checks: (a) the delegation token is valid, (b) the original caller has permission for this specific action, (c) the action is within scope.

[detective] **Audit each privilege transit** — Log: original caller → delegating component → action performed. Make this queryable for incident response.

Privilege transit across boundary: Core Agent Layer → Wiz AI Security Lens [cross-boundary]

STRIDE → Elevation of Privilege | Component: **Wiz AI Security Lens** (external_entity) | Source: rule-based

CWE: CWE-441 — Confused Deputy

CVSS 3.1: 9.9 (Critical) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:L

CVSS 4.0: 7.4 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:H/SI:H/SA:L

Boundary crossing: Application tier → Internet

■ **Location:** On the data flow **Core Agent Layer** → **Wiz AI Security Lens** (label: *Core Agent Layer → Wiz AI Security Lens*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `Application tier` into `Internet`. The receiving component is **Wiz AI Security Lens** (`external\_entity`).

■ **Description:** If the receiver in 'Internet' acts on behalf of the caller, attackers compromising 'Application tier' may inherit the receiver's privileges (confused-deputy).

■ ■ Attack scenario:

1. Attacker compromises a low-privilege component in zone `Application tier`.
2. Issues requests to Wiz AI Security Lens that get re-executed with the receiver's higher privileges (classic confused-deputy).
3. Because authorization is checked once at request entry but not per-action against the originating caller, the attacker effectively becomes a privileged actor in `Internet`.
4. Attacker performs actions far beyond the original principal's permissions.

DREAD — D:9 R:8 E:10 A:9 D:10 → **Total 46/50**

■ How to mitigate:

[preventive] **Use scoped, short-lived delegation tokens** — When Wiz AI Security Lens acts on behalf of a caller, attach a delegation token with the caller's identity, scope (action allow-list), and ≤ 5-min TTL.

[preventive] **Authorize per-action against the originating caller** — Don't rely on ambient authority of the receiver. Each action checks: (a) the delegation token is valid, (b) the original caller has permission for this specific action, (c) the action is within scope.

[detective] **Audit each privilege transit** — Log: original caller → delegating component → action performed. Make this queryable for incident response.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-441 — Confused Deputy](#)

High (257)

Authentication bypass via credential stuffing

STRIDE → Spoofing | Component: **AI Detection & Response** (api) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **AI Detection & Response** (`api`).

■ **Description:** Attackers reuse leaked credentials to impersonate legitimate users.

■ ■ Attack scenario:

1. Attacker locates AI Detection & Response's identity claim mechanism (token format, header name, signing key handling).
2. Forges or replays a token to assert another principal's identity.
3. AI Detection & Response accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
4. Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:7 R:6 E:7 A:8 D:7 → **Total 35/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Session token hijacking

STRIDE → Spoofing | Component: **AI Detection & Response** (api) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **AI Detection & Response** (`api`).

■ **Description:** Attacker steals or predicts a session token to impersonate a user.

■ ■ Attack scenario:

1. Attacker locates AI Detection & Response's identity claim mechanism (token format, header name, signing key handling).
2. Forges or replays a token to assert another principal's identity.
3. AI Detection & Response accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
4. Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:7 R:6 E:7 A:8 D:7 → **Total 35/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

API key / service identity spoofing

STRIDE → Spoofing | Component: **AI Detection & Response** (api) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **AI Detection & Response** (`api`).

■ **Description:** An attacker obtains or forges an API key to act as a trusted service.

■ ■ Attack scenario:

1. Attacker locates AI Detection & Response's identity claim mechanism (token format, header name, signing key handling).
2. Forges or replays a token to assert another principal's identity.
3. AI Detection & Response accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
4. Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:7 R:6 E:7 A:8 D:7 → **Total 35/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Log tampering

STRIDE → Repudiation | Component: **AI Detection & Response** (api) | Source: rule-based

CWE: CWE-778 — Insufficient Logging

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:H/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **AI Detection & Response** (`api`).

■ **Description:** An attacker with access modifies or deletes audit trails.

■ ■ Attack scenario:

1. User performs a sensitive action through AI Detection & Response (a transfer, a permission change, a deletion).
2. Audit logs are absent, incomplete, or modifiable by the same principal who performed the action.
3. User later denies having performed the action, or an attacker covers their tracks.
4. Without tamper-evident logs, neither responsible party can be identified — leading to fraud, dispute, or regulatory exposure.

DREAD — D:7 R:6 E:7 A:8 D:7 → **Total 35/50**

■ How to mitigate:

[*detective*] **Emit tamper-evident audit logs** — Sign log entries (HMAC chain, hash-linked) so insertion or deletion is detectable. Forward to a separate, append-only system the action's principal cannot administer.

[*detective*] **Capture sufficient detail per audit event** — Who (authenticated principal, not just session ID), what (specific action and target), when (UTC, monotonic-clock-corroborated), where (source IP, request ID), why (correlated to the upstream business event).

[*detective*] **Periodic log integrity verification** — Schedule daily integrity checks on the audit log chain. Alert on any gap.

[A09:2021 — Security Logging and Monitoring Failures](#) | [STRIDE reference](#) | [CWE-778 — Insufficient Logging](#)

Sensitive data exposure in transit

STRIDE → Information Disclosure | Component: **AI Detection & Response** (api) | Source: rule-based

CWE: CWE-319 — Cleartext Transmission of Sensitive Information

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **AI Detection & Response** (`api`).

■ **Description:** PII, secrets, or tokens transmitted over unencrypted channels.

■ ■ Attack scenario:

1. Attacker reaches AI Detection & Response via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:6 E:7 A:8 D:8 → **Total 36/50**

■ How to mitigate:

[*preventive*] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[*preventive*] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[*preventive*] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-319 — Cleartext Transmission of Sensitive Information](#)

Sensitive data exposure at rest

STRIDE → Information Disclosure | Component: **AI Detection & Response** (api) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **AI Detection & Response** (`api`).

■ **Description:** Stored PII / secrets accessible without proper authorization.

■■ **Attack scenario:**

1. Attacker reaches AI Detection & Response via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:6 E:7 A:8 D:8 → **Total 36/50**

■ **How to mitigate:**

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Insecure direct object reference (IDOR)

STRIDE → Information Disclosure | Component: **AI Detection & Response** (api) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **AI Detection & Response** (`api`).

■ **Description:** User can access another user's resources by guessing IDs.

■■ **Attack scenario:**

1. Attacker reaches AI Detection & Response via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:8 E:7 A:8 D:8 → **Total 38/50**

■ **How to mitigate:**

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Application-layer DDoS

STRIDE → Denial of Service | Component: **AI Detection & Response** (api) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **AI Detection & Response** (`api`).

■ **Description:** Attacker floods expensive endpoints (e.g., search, login).

■ ■ **Attack scenario:**

1. Attacker identifies a costly operation in AI Detection & Response (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. AI Detection & Response's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach AI Detection & Response; cascading failure may take down dependents.

DREAD — D:7 R:6 E:7 A:8 D:7 → **Total 35/50**

■ **How to mitigate:**

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Privilege escalation via mass assignment

STRIDE → Elevation of Privilege | Component: **AI Detection & Response** (api) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **AI Detection & Response** (`api`).

■ **Description:** User submits extra fields (e.g., role=admin) and they bind to the model.

■ ■ **Attack scenario:**

1. Attacker authenticates to AI Detection & Response as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:7 R:8 E:7 A:8 D:7 → **Total 37/50**

■ **How to mitigate:**

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to AI Detection & Response should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Data-in-transit modification

STRIDE → Tampering | Component: **Agent Evaluation** (datastore) | Source: rule-based

CWE: CWE-345 — Insufficient Verification of Data Authenticity

CVSS 3.1: 7.3 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Evaluation** (`datastore`).

■ **Description:** Attacker on the network path alters request/response payloads.

■ Attack scenario:

1. Attacker reaches an input vector exposed by Agent Evaluation (request body, query string, message queue payload, persisted state).
2. Submits malformed input designed to alter the program's logic or stored data (injection, parameter tampering, deserialization gadgets).
3. Agent Evaluation processes the input without strict validation, and the malicious change takes effect.
4. Tampered data is persisted, executed, or relayed downstream — corrupting integrity beyond the original entry point.

DREAD — D:8 R:6 E:7 A:5 D:7 → **Total 33/50**

■ How to mitigate:

[preventive] **Schema-validate and parameterize all inputs** — JSON Schema / OpenAPI validation at API edges; parameterized queries / prepared statements for SQL; ORM-level validators on persisted models.

[preventive] **Sign and verify integrity-critical messages** — For commands or state mutations, attach an HMAC or signature so downstream consumers can detect tampering in transit or at rest.

[detective] **Use append-only / write-once stores for audit-critical data** — Where data must not be silently changed, write to an append-only log (e.g., immutable S3 bucket, CloudTrail-like store) and reconcile against the mutable copy.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-345 — Insufficient Verification of Data Authenticity](#)

Config / secret tampering

STRIDE → Tampering | Component: **Agent Evaluation** (datastore) | Source: rule-based

CWE: CWE-345 — Insufficient Verification of Data Authenticity

CVSS 3.1: 7.3 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Evaluation** (`datastore`).

■ **Description:** Unauthorized changes to runtime config alter security behavior.

■ Attack scenario:

1. Attacker reaches an input vector exposed by Agent Evaluation (request body, query string, message queue payload, persisted state).
2. Submits malformed input designed to alter the program's logic or stored data (injection, parameter tampering, deserialization gadgets).
3. Agent Evaluation processes the input without strict validation, and the malicious change takes effect.
4. Tampered data is persisted, executed, or relayed downstream — corrupting integrity beyond the original entry point.

DREAD — D:8 R:6 E:7 A:5 D:7 → **Total 33/50**

■ How to mitigate:

[preventive] **Schema-validate and parameterize all inputs** — JSON Schema / OpenAPI validation at API edges; parameterized queries / prepared statements for SQL; ORM-level validators on persisted models.

[preventive] **Sign and verify integrity-critical messages** — For commands or state mutations, attach an HMAC or signature so downstream consumers can detect tampering in transit or at rest.

[detective] **Use append-only / write-once stores for audit-critical data** — Where data must not be silently changed, write to an append-only log (e.g., immutable S3 bucket, CloudTrail-like store) and reconcile against the mutable copy.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-345 — Insufficient Verification of Data Authenticity](#)

Sensitive data exposure in transit

STRIDE → Information Disclosure | Component: **Agent Evaluation** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Evaluation** (`datastore`).

■ **Description:** PII, secrets, or tokens transmitted over unencrypted channels.

■ ■ Attack scenario:

1. Attacker reaches Agent Evaluation via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:8 R:6 E:7 A:5 D:8 → **Total 34/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Sensitive data exposure at rest

STRIDE → Information Disclosure | Component: **Agent Evaluation** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Evaluation** (`datastore`).

■ **Description:** Stored PII / secrets accessible without proper authorization.

■ ■ Attack scenario:

1. Attacker reaches Agent Evaluation via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:8 R:6 E:7 A:5 D:8 → **Total 34/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Insecure direct object reference (IDOR)

STRIDE → Information Disclosure | Component: **Agent Evaluation** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Evaluation** (`datastore`).

■ **Description:** User can access another user's resources by guessing IDs.

■ ■ Attack scenario:

1. Attacker reaches Agent Evaluation via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs,

debug data, full PII fields).

3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:8 R:8 E:7 A:5 D:8 → **Total 36/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Trust-boundary crossing without strong authn: Core Agent Layer → Agent Evaluation [cross-boundary]

STRIDE → Spoofing | Component: **Agent Evaluation** (datastore) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 6.8 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Core Agent Layer** → **Agent Evaluation** (label: *Core Agent Layer → Agent Evaluation*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Agent Evaluation** (datastore).

■ **Description:** Flow 'Core Agent Layer → Agent Evaluation' crosses trust boundary 'Application tier' → 'Data tier'. Caller identity must be re-verified at the boundary; existing trust does not transit.

■■ Attack scenario:

1. Attacker positions themselves on the network path between source and destination, or compromises an upstream component in the source zone.
2. Crafts requests with forged or replayed identity claims (cookies, tokens, IP-based trust).
3. Agent Evaluation accepts the request because identity is not re-verified at the boundary, treating the upstream zone as trusted.
4. Attacker can now perform actions as the impersonated principal across the boundary.

DREAD — D:8 R:6 E:7 A:7 D:7 → **Total 35/50**

■ How to mitigate:

[preventive] **Re-authenticate at the boundary** — Require a fresh, audience-bound credential at the receiver. Do not infer identity from network position, source IP, or upstream session.

[preventive] **Use mutual TLS or signed tokens with audience claims** — For service-to-service calls into Agent Evaluation, terminate mTLS at the boundary and reject any caller without a valid client cert. For human-driven flows, use OAuth2 / OIDC with audience and issuer validation.

[detective] **Log every cross-boundary auth decision** — Emit an authentication event (success and failure) tagged with both source and destination zone. Forward to SIEM with retention ≥ 90 days.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Cross-boundary input not validated: Core Agent Layer → Agent Evaluation [cross-boundary]

STRIDE → Tampering | Component: **Agent Evaluation** (datastore) | Source: rule-based

CWE: CWE-20 — Improper Input Validation

CVSS 3.1: 8.7 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:L/SI:H/SA:N

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Core Agent Layer** → **Agent Evaluation** (label: *Core Agent Layer → Agent Evaluation*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Agent Evaluation** (datastore).

■ **Description:** Data crossing the trust boundary into 'Data tier' must be treated as untrusted, even if the source is internal. Implicit trust is the most common cause of injection / SSRF / deserialization bugs.

■ ■ **Attack scenario:**

1. Attacker compromises a less-trusted upstream zone or directly sends malformed input to Agent Evaluation.
2. Sends payloads with unexpected types, lengths, or special characters (e.g. SQL meta-chars, NUL bytes, control sequences).
3. Agent Evaluation processes the input under the assumption that the upstream zone has already validated it — but the boundary changes the security context.
4. Injection, deserialization flaws, or buffer issues trigger; attacker pivots into Agent Evaluation or the zone behind it.

DREAD — D:8 R:6 E:7 A:7 D:7 → **Total 35/50**

■ **How to mitigate:**

[preventive] **Validate against an allow-list schema at the boundary** — Define an explicit schema (JSON Schema, Protobuf, OpenAPI) for every accepted message at this boundary. Reject anything that doesn't match — type, length, charset, enum values, nested depth.

[preventive] **Canonicalize before validation** — Decode URL-encoding, Unicode normalization (NFC), trim whitespace, and resolve relative paths before validating. Avoid double-decoding bypasses.

[preventive] **Apply context-specific output encoding** — Whatever Agent Evaluation does with the input — SQL: parameterized queries, HTML: contextual escaping, OS commands: avoid shell, use exec arrays, LDAP: escape filter chars.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-20 — Improper Input Validation](#)

Cross-boundary data exposure risk: Core Agent Layer → Agent Evaluation [cross-boundary]

STRIDE → Information Disclosure | Component: **Agent Evaluation** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 6.8 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Core Agent Layer** → **Agent Evaluation** (label: *Core Agent Layer → Agent Evaluation*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Agent Evaluation** (`datastore`).

■ **Description:** Information leaving 'Application tier' into 'Data tier' may include data the receiving zone is not authorized to see. Cross-boundary egress is a common data-leak surface.

■ ■ **Attack scenario:**

1. Attacker observes traffic leaving the source zone, or compromises a component in the destination zone.
2. The destination zone receives more data than it strictly needs (over-fetching, verbose error responses, full record dumps).
3. Attacker harvests sensitive fields (PII, secrets, internal IDs) that should never have left the source zone.
4. Data is exfiltrated, sold, or used to plan a deeper attack on the originating zone.

DREAD — D:8 R:6 E:7 A:7 D:8 → **Total 36/50**

■ **How to mitigate:**

[preventive] **Enforce minimum-data-needed at the egress point** — Whitelist exactly which fields leave the source zone. Strip everything else server-side before serialization.

[preventive] **Tokenize or redact sensitive fields** — Replace PII / PCI / PHI fields with reversible tokens or one-way hashes when the destination zone doesn't need the cleartext value.

[detective] **Log cross-boundary data flows for review** — Sample-log the field set crossing this boundary (not the values) so DLP and privacy reviews can audit what's egressing.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Authentication bypass via credential stuffing

STRIDE → Spoofing | Component: **Agent Gateway** (api) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Gateway** (`api`).

■ **Description:** Attackers reuse leaked credentials to impersonate legitimate users.

■ ■ **Attack scenario:**

1. Attacker locates Agent Gateway's identity claim mechanism (token format, header name, signing key handling).
2. Forges or replays a token to assert another principal's identity.
3. Agent Gateway accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
4. Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:7 R:6 E:9 A:9 D:9 → **Total 40/50**

■ **How to mitigate:**

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Session token hijacking

STRIDE → Spoofing | Component: **Agent Gateway** (api) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Gateway** (`api`).

■ **Description:** Attacker steals or predicts a session token to impersonate a user.

■ ■ **Attack scenario:**

1. Attacker locates Agent Gateway's identity claim mechanism (token format, header name, signing key handling).
2. Forges or replays a token to assert another principal's identity.
3. Agent Gateway accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
4. Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:7 R:6 E:9 A:9 D:9 → **Total 40/50**

■ **How to mitigate:**

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

API key / service identity spoofing

STRIDE → Spoofing | Component: **Agent Gateway** (api) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Gateway** (`api`).

■ **Description:** An attacker obtains or forges an API key to act as a trusted service.

■ ■ **Attack scenario:**

1. Attacker locates Agent Gateway's identity claim mechanism (token format, header name, signing key handling).

2. Forges or replays a token to assert another principal's identity.
3. Agent Gateway accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
4. Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:7 R:6 E:9 A:9 D:9 → **Total 40/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Log tampering

STRIDE → Repudiation | Component: **Agent Gateway** (api) | Source: rule-based

CWE: CWE-778 — Insufficient Logging

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Gateway** (`api`).

■ **Description:** An attacker with access modifies or deletes audit trails.

■■ Attack scenario:

1. User performs a sensitive action through Agent Gateway (a transfer, a permission change, a deletion).
2. Audit logs are absent, incomplete, or modifiable by the same principal who performed the action.
3. User later denies having performed the action, or an attacker covers their tracks.
4. Without tamper-evident logs, neither responsible party can be identified — leading to fraud, dispute, or regulatory exposure.

DREAD — D:7 R:6 E:9 A:9 D:9 → **Total 40/50**

■ How to mitigate:

[detective] **Emit tamper-evident audit logs** — Sign log entries (HMAC chain, hash-linked) so insertion or deletion is detectable. Forward to a separate, append-only system the action's principal cannot administer.

[detective] **Capture sufficient detail per audit event** — Who (authenticated principal, not just session ID), what (specific action and target), when (UTC, monotonic-clock-corroborated), where (source IP, request ID), why (correlated to the upstream business event).

[detective] **Periodic log integrity verification** — Schedule daily integrity checks on the audit log chain. Alert on any gap.

[A09:2021 — Security Logging and Monitoring Failures](#) | [STRIDE reference](#) | [CWE-778 — Insufficient Logging](#)

Sensitive data exposure in transit

STRIDE → Information Disclosure | Component: **Agent Gateway** (api) | Source: rule-based

CWE: CWE-319 — Cleartext Transmission of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Gateway** (`api`).

■ **Description:** PII, secrets, or tokens transmitted over unencrypted channels.

■■ Attack scenario:

1. Attacker reaches Agent Gateway via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:6 E:9 A:9 D:10 → **Total 41/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-319 — Cleartext Transmission of Sensitive Information](#)

Sensitive data exposure at rest

STRIDE → Information Disclosure | Component: **Agent Gateway** (api) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Gateway** (`api`).

■ **Description:** Stored PII / secrets accessible without proper authorization.

■■ Attack scenario:

1. Attacker reaches Agent Gateway via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:6 E:9 A:9 D:10 → **Total 41/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Insecure direct object reference (IDOR)

STRIDE → Information Disclosure | Component: **Agent Gateway** (api) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Gateway** (`api`).

■ **Description:** User can access another user's resources by guessing IDs.

■■ Attack scenario:

1. Attacker reaches Agent Gateway via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:8 E:9 A:9 D:10 → **Total 43/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Application-layer DDoS

STRIDE → Denial of Service | Component: **Agent Gateway** (api) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Gateway** (`api`).

■ **Description:** Attacker floods expensive endpoints (e.g., search, login).

■ ■ ■ Attack scenario:

1. Attacker identifies a costly operation in Agent Gateway (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Agent Gateway's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Agent Gateway; cascading failure may take down dependents.

DREAD — D:7 R:6 E:9 A:9 D:9 → **Total 40/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Privilege escalation via mass assignment

STRIDE → Elevation of Privilege | Component: **Agent Gateway** (api) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Gateway** (`api`).

■ **Description:** User submits extra fields (e.g., role=admin) and they bind to the model.

■ ■ ■ Attack scenario:

1. Attacker authenticates to Agent Gateway as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:7 R:8 E:9 A:9 D:9 → **Total 42/50**

■ How to mitigate:

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to Agent Gateway should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when

admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Trust-boundary crossing without strong authn: Users → Agent Gateway [cross-boundary]

STRIDE → Spoofing | Component: **Agent Gateway** (api) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Internet → DMZ (public-facing)

■ **Location:** On the data flow **Users → Agent Gateway** (label: *Users → Agent Gateway*, protocol: HTTPS, auth: session, encrypted: yes), crossing the trust boundary from `Internet` into `DMZ (public-facing)`. The receiving component is **Agent Gateway** (`api`).

■ **Description:** Flow 'Users → Agent Gateway' crosses trust boundary 'Internet' → 'DMZ (public-facing)'. Caller identity must be re-verified at the boundary; existing trust does not transit.

■ ■ Attack scenario:

1. Attacker positions themselves on the network path between source and destination, or compromises an upstream component in the source zone.
2. Crafts requests with forged or replayed identity claims (cookies, tokens, IP-based trust).
3. Agent Gateway accepts the request because identity is not re-verified at the boundary, treating the upstream zone as trusted.
4. Attacker can now perform actions as the impersonated principal across the boundary.

DREAD — D:7 R:6 E:9 A:10 D:9 → **Total 41/50**

■ How to mitigate:

[preventive] **Re-authenticate at the boundary** — Require a fresh, audience-bound credential at the receiver. Do not infer identity from network position, source IP, or upstream session.

[preventive] **Use mutual TLS or signed tokens with audience claims** — For service-to-service calls into Agent Gateway, terminate mTLS at the boundary and reject any caller without a valid client cert. For human-driven flows, use OAuth2 / OIDC with audience and issuer validation.

[detective] **Log every cross-boundary auth decision** — Emit an authentication event (success and failure) tagged with both source and destination zone. Forward to SIEM with retention ≥ 90 days.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Cross-boundary input not validated: Users → Agent Gateway [cross-boundary]

STRIDE → Tampering | Component: **Agent Gateway** (api) | Source: rule-based

CWE: CWE-20 — Improper Input Validation

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:N/I:H/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:N/VI:H/VA:N/SC:L/SI:H/SA:N

Boundary crossing: Internet → DMZ (public-facing)

■ **Location:** On the data flow **Users → Agent Gateway** (label: *Users → Agent Gateway*, protocol: HTTPS, auth: session, encrypted: yes), crossing the trust boundary from `Internet` into `DMZ (public-facing)`. The receiving component is **Agent Gateway** (`api`).

■ **Description:** Data crossing the trust boundary into 'DMZ (public-facing)' must be treated as untrusted, even if the source is internal. Implicit trust is the most common cause of injection / SSRF / deserialization bugs.

■ ■ Attack scenario:

1. Attacker compromises a less-trusted upstream zone or directly sends malformed input to Agent Gateway.
2. Sends payloads with unexpected types, lengths, or special characters (e.g. SQL meta-chars, NUL bytes, control sequences).
3. Agent Gateway processes the input under the assumption that the upstream zone has already validated it — but the boundary changes the security context.
4. Injection, deserialization flaws, or buffer issues trigger; attacker pivots into Agent Gateway or the zone behind it.

DREAD — D:7 R:6 E:9 A:10 D:9 → **Total 41/50**

■ How to mitigate:

[preventive] **Validate against an allow-list schema at the boundary** — Define an explicit schema (JSON Schema, Protobuf, OpenAPI) for every accepted message at this boundary. Reject anything that doesn't match — type, length, charset, enum values, nested depth.

[preventive] **Canonicalize before validation** — Decode URL-encoding, Unicode normalization (NFC), trim whitespace, and resolve relative paths before validating. Avoid double-decoding bypasses.

[preventive] **Apply context-specific output encoding** — Whatever Agent Gateway does with the input — SQL: parameterized queries, HTML: contextual escaping, OS commands: avoid shell, use exec arrays, LDAP: escape filter chars.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-20 — Improper Input Validation](#)

Cross-boundary data exposure risk: Users → Agent Gateway [cross-boundary]

STRIDE → Information Disclosure | Component: **Agent Gateway** (api) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Internet → DMZ (public-facing)

■ **Location:** On the data flow **Users → Agent Gateway** (label: *Users → Agent Gateway*, protocol: HTTPS, auth: session, encrypted: yes), crossing the trust boundary from `Internet` into `DMZ (public-facing)`. The receiving component is **Agent Gateway** (`api`).

■ **Description:** Information leaving 'Internet' into 'DMZ (public-facing)' may include data the receiving zone is not authorized to see. Cross-boundary egress is a common data-leak surface.

■■ Attack scenario:

1. Attacker observes traffic leaving the source zone, or compromises a component in the destination zone.
2. The destination zone receives more data than it strictly needs (over-fetching, verbose error responses, full record dumps).
3. Attacker harvests sensitive fields (PII, secrets, internal IDs) that should never have left the source zone.
4. Data is exfiltrated, sold, or used to plan a deeper attack on the originating zone.

DREAD — D:7 R:6 E:9 A:10 D:10 → **Total 42/50**

■ How to mitigate:

[preventive] **Enforce minimum-data-needed at the egress point** — Whitelist exactly which fields leave the source zone. Strip everything else server-side before serialization.

[preventive] **Tokenize or redact sensitive fields** — Replace PII / PCI / PHI fields with reversible tokens or one-way hashes when the destination zone doesn't need the cleartext value.

[detective] **Log cross-boundary data flows for review** — Sample-log the field set crossing this boundary (not the values) so DLP and privacy reviews can audit what's egressing.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Trust-boundary crossing without strong authn: Identity Provider / Okta MCP → Agent Gateway [cross-boundary]

STRIDE → Spoofing | Component: **Agent Gateway** (api) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Third-party / Untrusted services → DMZ (public-facing)

■ **Location:** On the data flow **Identity Provider / Okta MCP → Agent Gateway** (label: *Identity Provider / Okta MCP → Agent Gateway*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `Third-party / Untrusted services` into `DMZ (public-facing)`. The receiving component is **Agent Gateway** (`api`).

■ **Description:** Flow 'Identity Provider / Okta MCP → Agent Gateway' crosses trust boundary 'Third-party / Untrusted services' → 'DMZ (public-facing)'. Caller identity must be re-verified at the boundary; existing trust does not transit.

■■ Attack scenario:

1. Attacker positions themselves on the network path between source and destination, or compromises an upstream component

in the source zone.

2. Crafts requests with forged or replayed identity claims (cookies, tokens, IP-based trust).
3. Agent Gateway accepts the request because identity is not re-verified at the boundary, treating the upstream zone as trusted.
4. Attacker can now perform actions as the impersonated principal across the boundary.

DREAD — D:7 R:6 E:9 A:10 D:9 → **Total 41/50**

■ How to mitigate:

[preventive] **Re-authenticate at the boundary** — Require a fresh, audience-bound credential at the receiver. Do not infer identity from network position, source IP, or upstream session.

[preventive] **Use mutual TLS or signed tokens with audience claims** — For service-to-service calls into Agent Gateway, terminate mTLS at the boundary and reject any caller without a valid client cert. For human-driven flows, use OAuth2 / OIDC with audience and issuer validation.

[detective] **Log every cross-boundary auth decision** — Emit an authentication event (success and failure) tagged with both source and destination zone. Forward to SIEM with retention ≥ 90 days.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Cross-boundary input not validated: Identity Provider / Okta MCP → Agent Gateway [cross-boundary]

STRIDE → Tampering | Component: **Agent Gateway** (api) | Source: rule-based

CWE: CWE-20 — Improper Input Validation

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:N/I:H/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:N/VI:H/VA:N/SC:L/SI:H/SA:N

Boundary crossing: Third-party / Untrusted services → DMZ (public-facing)

■ **Location:** On the data flow **Identity Provider / Okta MCP → Agent Gateway** (label: *Identity Provider / Okta MCP → Agent Gateway*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `Third-party / Untrusted services` into `DMZ (public-facing)`. The receiving component is **Agent Gateway** (`api`).

■ **Description:** Data crossing the trust boundary into 'DMZ (public-facing)' must be treated as untrusted, even if the source is internal. Implicit trust is the most common cause of injection / SSRF / deserialization bugs.

■■ Attack scenario:

1. Attacker compromises a less-trusted upstream zone or directly sends malformed input to Agent Gateway.
2. Sends payloads with unexpected types, lengths, or special characters (e.g. SQL meta-chars, NUL bytes, control sequences).
3. Agent Gateway processes the input under the assumption that the upstream zone has already validated it — but the boundary changes the security context.
4. Injection, deserialization flaws, or buffer issues trigger; attacker pivots into Agent Gateway or the zone behind it.

DREAD — D:7 R:6 E:9 A:10 D:9 → **Total 41/50**

■ How to mitigate:

[preventive] **Validate against an allow-list schema at the boundary** — Define an explicit schema (JSON Schema, Protobuf, OpenAPI) for every accepted message at this boundary. Reject anything that doesn't match — type, length, charset, enum values, nested depth.

[preventive] **Canonicalize before validation** — Decode URL-encoding, Unicode normalization (NFC), trim whitespace, and resolve relative paths before validating. Avoid double-decoding bypasses.

[preventive] **Apply context-specific output encoding** — Whatever Agent Gateway does with the input — SQL: parameterized queries, HTML: contextual escaping, OS commands: avoid shell, use exec arrays, LDAP: escape filter chars.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-20 — Improper Input Validation](#)

Cross-boundary data exposure risk: Identity Provider / Okta MCP → Agent Gateway [cross-boundary]

STRIDE → Information Disclosure | Component: **Agent Gateway** (api) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Third-party / Untrusted services → DMZ (public-facing)

■ **Location:** On the data flow **Identity Provider / Okta MCP → Agent Gateway** (label: *Identity Provider / Okta MCP → Agent Gateway*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `Third-party / Untrusted services`

into `DMZ (public-facing)`. The receiving component is **Agent Gateway** (`api`).

■ **Description:** Information leaving 'Third-party / Untrusted services' into 'DMZ (public-facing)' may include data the receiving zone is not authorized to see. Cross-boundary egress is a common data-leak surface.

■ **Attack scenario:**

1. Attacker observes traffic leaving the source zone, or compromises a component in the destination zone.
2. The destination zone receives more data than it strictly needs (over-fetching, verbose error responses, full record dumps).
3. Attacker harvests sensitive fields (PII, secrets, internal IDs) that should never have left the source zone.
4. Data is exfiltrated, sold, or used to plan a deeper attack on the originating zone.

DREAD — D:7 R:6 E:9 A:10 D:10 → **Total 42/50**

■ **How to mitigate:**

[preventive] **Enforce minimum-data-needed at the egress point** — Whitelist exactly which fields leave the source zone. Strip everything else server-side before serialization.

[preventive] **Tokenize or redact sensitive fields** — Replace PII / PCI / PHI fields with reversible tokens or one-way hashes when the destination zone doesn't need the cleartext value.

[detective] **Log cross-boundary data flows for review** — Sample-log the field set crossing this boundary (not the values) so DLP and privacy reviews can audit what's egressing.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Data-in-transit modification

STRIDE → Tampering | Component: **Agent Registry** (datastore) | Source: rule-based

CWE: CWE-345 — Insufficient Verification of Data Authenticity

CVSS 3.1: 7.3 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Registry** (`datastore`).

■ **Description:** Attacker on the network path alters request/response payloads.

■ **Attack scenario:**

1. Attacker reaches an input vector exposed by Agent Registry (request body, query string, message queue payload, persisted state).
2. Submits malformed input designed to alter the program's logic or stored data (injection, parameter tampering, deserialization gadgets).
3. Agent Registry processes the input without strict validation, and the malicious change takes effect.
4. Tampered data is persisted, executed, or relayed downstream — corrupting integrity beyond the original entry point.

DREAD — D:8 R:6 E:7 A:5 D:7 → **Total 33/50**

■ **How to mitigate:**

[preventive] **Schema-validate and parameterize all inputs** — JSON Schema / OpenAPI validation at API edges; parameterized queries / prepared statements for SQL; ORM-level validators on persisted models.

[preventive] **Sign and verify integrity-critical messages** — For commands or state mutations, attach an HMAC or signature so downstream consumers can detect tampering in transit or at rest.

[detective] **Use append-only / write-once stores for audit-critical data** — Where data must not be silently changed, write to an append-only log (e.g., immutable S3 bucket, CloudTrail-like store) and reconcile against the mutable copy.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-345 — Insufficient Verification of Data Authenticity](#)

Config / secret tampering

STRIDE → Tampering | Component: **Agent Registry** (datastore) | Source: rule-based

CWE: CWE-345 — Insufficient Verification of Data Authenticity

CVSS 3.1: 7.3 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Registry** (`datastore`).

■ **Description:** Unauthorized changes to runtime config alter security behavior.

■ Attack scenario:

1. Attacker reaches an input vector exposed by Agent Registry (request body, query string, message queue payload, persisted state).
2. Submits malformed input designed to alter the program's logic or stored data (injection, parameter tampering, deserialization gadgets).
3. Agent Registry processes the input without strict validation, and the malicious change takes effect.
4. Tampered data is persisted, executed, or relayed downstream — corrupting integrity beyond the original entry point.

DREAD — D:8 R:6 E:7 A:5 D:7 → **Total 33/50**

■ How to mitigate:

[preventive] **Schema-validate and parameterize all inputs** — JSON Schema / OpenAPI validation at API edges; parameterized queries / prepared statements for SQL; ORM-level validators on persisted models.

[preventive] **Sign and verify integrity-critical messages** — For commands or state mutations, attach an HMAC or signature so downstream consumers can detect tampering in transit or at rest.

[detective] **Use append-only / write-once stores for audit-critical data** — Where data must not be silently changed, write to an append-only log (e.g., immutable S3 bucket, CloudTrail-like store) and reconcile against the mutable copy.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-345 — Insufficient Verification of Data Authenticity](#)

Sensitive data exposure in transit

STRIDE → Information Disclosure | Component: **Agent Registry** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Registry** (`datastore`).

■ **Description:** PII, secrets, or tokens transmitted over unencrypted channels.

■ Attack scenario:

1. Attacker reaches Agent Registry via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:8 R:6 E:7 A:5 D:8 → **Total 34/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Sensitive data exposure at rest

STRIDE → Information Disclosure | Component: **Agent Registry** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Registry** (`datastore`).

■ **Description:** Stored PII / secrets accessible without proper authorization.

■ Attack scenario:

1. Attacker reaches Agent Registry via a legitimate or guessable channel.

2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:8 R:6 E:7 A:5 D:8 → **Total 34/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Insecure direct object reference (IDOR)

STRIDE → Information Disclosure | Component: **Agent Registry** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Registry** (`datastore`).

■ **Description:** User can access another user's resources by guessing IDs.

■■ Attack scenario:

1. Attacker reaches Agent Registry via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:8 R:8 E:7 A:5 D:8 → **Total 36/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Trust-boundary crossing without strong authn: Agent Runtime / Orchestrator → Agent Registry [cross-boundary]

STRIDE → Spoofing | Component: **Agent Registry** (datastore) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 6.8 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Agent Runtime / Orchestrator** → **Agent Registry** (label: *Agent Runtime / Orchestrator → Agent Registry*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Agent Registry** (`datastore`).

■ **Description:** Flow 'Agent Runtime / Orchestrator → Agent Registry' crosses trust boundary 'Application tier' → 'Data tier'. Caller identity must be re-verified at the boundary; existing trust does not transit.

■ ■ **Attack scenario:**

1. Attacker positions themselves on the network path between source and destination, or compromises an upstream component in the source zone.
2. Crafts requests with forged or replayed identity claims (cookies, tokens, IP-based trust).
3. Agent Registry accepts the request because identity is not re-verified at the boundary, treating the upstream zone as trusted.
4. Attacker can now perform actions as the impersonated principal across the boundary.

DREAD — D:8 R:6 E:7 A:7 D:7 → **Total 35/50**

■ **How to mitigate:**

[preventive] **Re-authenticate at the boundary** — Require a fresh, audience-bound credential at the receiver. Do not infer identity from network position, source IP, or upstream session.

[preventive] **Use mutual TLS or signed tokens with audience claims** — For service-to-service calls into Agent Registry, terminate mTLS at the boundary and reject any caller without a valid client cert. For human-driven flows, use OAuth2 / OIDC with audience and issuer validation.

[detective] **Log every cross-boundary auth decision** — Emit an authentication event (success and failure) tagged with both source and destination zone. Forward to SIEM with retention ≥ 90 days.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Cross-boundary input not validated: Agent Runtime / Orchestrator → Agent Registry [cross-boundary]

STRIDE → Tampering | Component: **Agent Registry** (datastore) | Source: rule-based

CWE: CWE-20 — Improper Input Validation

CVSS 3.1: 8.7 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:L/SI:H/SA:N

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Agent Runtime / Orchestrator → Agent Registry** (label: *Agent Runtime / Orchestrator → Agent Registry*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from 'Application tier' into 'Data tier'. The receiving component is **Agent Registry** ('datastore').

■ **Description:** Data crossing the trust boundary into 'Data tier' must be treated as untrusted, even if the source is internal. Implicit trust is the most common cause of injection / SSRF / deserialization bugs.

■ ■ **Attack scenario:**

1. Attacker compromises a less-trusted upstream zone or directly sends malformed input to Agent Registry.
2. Sends payloads with unexpected types, lengths, or special characters (e.g. SQL meta-chars, NUL bytes, control sequences).
3. Agent Registry processes the input under the assumption that the upstream zone has already validated it — but the boundary changes the security context.
4. Injection, deserialization flaws, or buffer issues trigger; attacker pivots into Agent Registry or the zone behind it.

DREAD — D:8 R:6 E:7 A:7 D:7 → **Total 35/50**

■ **How to mitigate:**

[preventive] **Validate against an allow-list schema at the boundary** — Define an explicit schema (JSON Schema, Protobuf, OpenAPI) for every accepted message at this boundary. Reject anything that doesn't match — type, length, charset, enum values, nested depth.

[preventive] **Canonicalize before validation** — Decode URL-encoding, Unicode normalization (NFC), trim whitespace, and resolve relative paths before validating. Avoid double-decoding bypasses.

[preventive] **Apply context-specific output encoding** — Whatever Agent Registry does with the input — SQL: parameterized queries, HTML: contextual escaping, OS commands: avoid shell, use exec arrays, LDAP: escape filter chars.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-20 — Improper Input Validation](#)

Cross-boundary data exposure risk: Agent Runtime / Orchestrator → Agent Registry [cross-boundary]

STRIDE → Information Disclosure | Component: **Agent Registry** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 6.8 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Agent Runtime / Orchestrator** → **Agent Registry** (label: *Agent Runtime / Orchestrator → Agent Registry*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from 'Application tier' into 'Data tier'. The receiving component is **Agent Registry** ('datastore').

■ **Description:** Information leaving 'Application tier' into 'Data tier' may include data the receiving zone is not authorized to see. Cross-boundary egress is a common data-leak surface.

■ ■ **Attack scenario:**

1. Attacker observes traffic leaving the source zone, or compromises a component in the destination zone.
2. The destination zone receives more data than it strictly needs (over-fetching, verbose error responses, full record dumps).
3. Attacker harvests sensitive fields (PII, secrets, internal IDs) that should never have left the source zone.
4. Data is exfiltrated, sold, or used to plan a deeper attack on the originating zone.

DREAD — D:8 R:6 E:7 A:7 D:8 → **Total 36/50**

■ **How to mitigate:**

[preventive] **Enforce minimum-data-needed at the egress point** — Whitelist exactly which fields leave the source zone. Strip everything else server-side before serialization.

[preventive] **Tokenize or redact sensitive fields** — Replace PII / PCI / PHI fields with reversible tokens or one-way hashes when the destination zone doesn't need the cleartext value.

[detective] **Log cross-boundary data flows for review** — Sample-log the field set crossing this boundary (not the values) so DLP and privacy reviews can audit what's egressing.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Authentication bypass via credential stuffing

STRIDE → Spoofing | Component: **Agent Runtime / Orchestrator** (api) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Runtime / Orchestrator** ('api').

■ **Description:** Attackers reuse leaked credentials to impersonate legitimate users.

■ ■ **Attack scenario:**

1. Attacker locates Agent Runtime / Orchestrator's identity claim mechanism (token format, header name, signing key handling).
2. Forges or replays a token to assert another principal's identity.
3. Agent Runtime / Orchestrator accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
4. Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:7 R:6 E:7 A:10 D:7 → **Total 37/50**

■ **How to mitigate:**

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Session token hijacking

STRIDE → Spoofing | Component: **Agent Runtime / Orchestrator** (api) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Runtime / Orchestrator** ('api').

■ **Description:** Attacker steals or predicts a session token to impersonate a user.

■ ■ **Attack scenario:**

1. Attacker locates Agent Runtime / Orchestrator's identity claim mechanism (token format, header name, signing key handling).
2. Forges or replays a token to assert another principal's identity.
3. Agent Runtime / Orchestrator accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
4. Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:7 R:6 E:7 A:10 D:7 → **Total 37/50**

■ **How to mitigate:**

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

API key / service identity spoofing

STRIDE → Spoofing | Component: **Agent Runtime / Orchestrator** (api) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Runtime / Orchestrator** (`api`).

■ **Description:** An attacker obtains or forges an API key to act as a trusted service.

■ ■ **Attack scenario:**

1. Attacker locates Agent Runtime / Orchestrator's identity claim mechanism (token format, header name, signing key handling).
2. Forges or replays a token to assert another principal's identity.
3. Agent Runtime / Orchestrator accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
4. Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:7 R:6 E:7 A:10 D:7 → **Total 37/50**

■ **How to mitigate:**

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Log tampering

STRIDE → Repudiation | Component: **Agent Runtime / Orchestrator** (api) | Source: rule-based

CWE: CWE-778 — Insufficient Logging

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Runtime / Orchestrator** (`api`).

■ **Description:** An attacker with access modifies or deletes audit trails.

■ ■ **Attack scenario:**

1. User performs a sensitive action through Agent Runtime / Orchestrator (a transfer, a permission change, a deletion).
2. Audit logs are absent, incomplete, or modifiable by the same principal who performed the action.
3. User later denies having performed the action, or an attacker covers their tracks.

4. Without tamper-evident logs, neither responsible party can be identified — leading to fraud, dispute, or regulatory exposure.

DREAD — D:7 R:6 E:7 A:10 D:7 → **Total 37/50**

■ How to mitigate:

[*detective*] **Emit tamper-evident audit logs** — Sign log entries (HMAC chain, hash-linked) so insertion or deletion is detectable. Forward to a separate, append-only system the action's principal cannot administer.

[*detective*] **Capture sufficient detail per audit event** — Who (authenticated principal, not just session ID), what (specific action and target), when (UTC, monotonic-clock-corroborated), where (source IP, request ID), why (correlated to the upstream business event).

[*detective*] **Periodic log integrity verification** — Schedule daily integrity checks on the audit log chain. Alert on any gap.

[A09:2021 — Security Logging and Monitoring Failures](#) | [STRIDE reference](#) | [CWE-778 — Insufficient Logging](#)

Sensitive data exposure in transit

STRIDE → Information Disclosure | Component: **Agent Runtime / Orchestrator** (api) | Source: rule-based

CWE: CWE-319 — Cleartext Transmission of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Runtime / Orchestrator** (`api`).

■ **Description:** PII, secrets, or tokens transmitted over unencrypted channels.

■■ Attack scenario:

1. Attacker reaches Agent Runtime / Orchestrator via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:6 E:7 A:10 D:8 → **Total 38/50**

■ How to mitigate:

[*preventive*] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[*preventive*] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[*preventive*] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-319 — Cleartext Transmission of Sensitive Information](#)

Sensitive data exposure at rest

STRIDE → Information Disclosure | Component: **Agent Runtime / Orchestrator** (api) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Runtime / Orchestrator** (`api`).

■ **Description:** Stored PII / secrets accessible without proper authorization.

■■ Attack scenario:

1. Attacker reaches Agent Runtime / Orchestrator via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:6 E:7 A:10 D:8 → **Total 38/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Insecure direct object reference (IDOR)

STRIDE → Information Disclosure | Component: **Agent Runtime / Orchestrator** (api) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Runtime / Orchestrator** (`api`).

■ **Description:** User can access another user's resources by guessing IDs.

■■ Attack scenario:

1. Attacker reaches Agent Runtime / Orchestrator via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:8 E:7 A:10 D:8 → **Total 40/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Application-layer DDoS

STRIDE → Denial of Service | Component: **Agent Runtime / Orchestrator** (api) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Runtime / Orchestrator** (`api`).

■ **Description:** Attacker floods expensive endpoints (e.g., search, login).

■■ Attack scenario:

1. Attacker identifies a costly operation in Agent Runtime / Orchestrator (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Agent Runtime / Orchestrator's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Agent Runtime / Orchestrator; cascading failure may take down dependents.

DREAD — D:7 R:6 E:7 A:10 D:7 → **Total 37/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[*preventive*] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[*corrective*] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021](#) — [Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400](#) — [Uncontrolled Resource Consumption \(DoS\)](#)

Privilege escalation via mass assignment

STRIDE → Elevation of Privilege | Component: **Agent Runtime / Orchestrator** (api) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Runtime / Orchestrator** (`api`).

■ **Description:** User submits extra fields (e.g., role=admin) and they bind to the model.

■ ■ ■ Attack scenario:

1. Attacker authenticates to Agent Runtime / Orchestrator as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:7 R:8 E:7 A:10 D:7 → **Total 39/50**

■ How to mitigate:

[*preventive*] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[*preventive*] **Apply least-privilege to service identities** — The IAM/role attached to Agent Runtime / Orchestrator should grant only the actions it actually performs. Audit IAM grants quarterly.

[*detective*] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021](#) — [Broken Access Control](#) | [STRIDE reference](#) | [CWE-269](#) — [Improper Privilege Management](#)

Data-in-transit modification

STRIDE → Tampering | Component: **Agent Tools Catalog** (datastore) | Source: rule-based

CWE: CWE-345 — Insufficient Verification of Data Authenticity

CVSS 3.1: 7.3 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Tools Catalog** (`datastore`).

■ **Description:** Attacker on the network path alters request/response payloads.

■ ■ ■ Attack scenario:

1. Attacker reaches an input vector exposed by Agent Tools Catalog (request body, query string, message queue payload, persisted state).
2. Submits malformed input designed to alter the program's logic or stored data (injection, parameter tampering, deserialization gadgets).
3. Agent Tools Catalog processes the input without strict validation, and the malicious change takes effect.
4. Tampered data is persisted, executed, or relayed downstream — corrupting integrity beyond the original entry point.

DREAD — D:8 R:6 E:7 A:5 D:7 → **Total 33/50**

■ How to mitigate:

[*preventive*] **Schema-validate and parameterize all inputs** — JSON Schema / OpenAPI validation at API edges; parameterized queries / prepared statements for SQL; ORM-level validators on persisted models.

[*preventive*] **Sign and verify integrity-critical messages** — For commands or state mutations, attach an HMAC or signature so downstream

consumers can detect tampering in transit or at rest.

[*detective*] **Use append-only / write-once stores for audit-critical data** — Where data must not be silently changed, write to an append-only log (e.g., immutable S3 bucket, CloudTrail-like store) and reconcile against the mutable copy.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-345 — Insufficient Verification of Data Authenticity](#)

Config / secret tampering

STRIDE → Tampering | Component: **Agent Tools Catalog** (datastore) | Source: rule-based

CWE: CWE-345 — Insufficient Verification of Data Authenticity

CVSS 3.1: 7.3 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Tools Catalog** (datastore).

■ **Description:** Unauthorized changes to runtime config alter security behavior.

■ ■ ■ Attack scenario:

1. Attacker reaches an input vector exposed by Agent Tools Catalog (request body, query string, message queue payload, persisted state).
2. Submits malformed input designed to alter the program's logic or stored data (injection, parameter tampering, deserialization gadgets).
3. Agent Tools Catalog processes the input without strict validation, and the malicious change takes effect.
4. Tampered data is persisted, executed, or relayed downstream — corrupting integrity beyond the original entry point.

DREAD — D:8 R:6 E:7 A:5 D:7 → **Total 33/50**

■ How to mitigate:

[*preventive*] **Schema-validate and parameterize all inputs** — JSON Schema / OpenAPI validation at API edges; parameterized queries / prepared statements for SQL; ORM-level validators on persisted models.

[*preventive*] **Sign and verify integrity-critical messages** — For commands or state mutations, attach an HMAC or signature so downstream consumers can detect tampering in transit or at rest.

[*detective*] **Use append-only / write-once stores for audit-critical data** — Where data must not be silently changed, write to an append-only log (e.g., immutable S3 bucket, CloudTrail-like store) and reconcile against the mutable copy.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-345 — Insufficient Verification of Data Authenticity](#)

Sensitive data exposure in transit

STRIDE → Information Disclosure | Component: **Agent Tools Catalog** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Tools Catalog** (datastore).

■ **Description:** PII, secrets, or tokens transmitted over unencrypted channels.

■ ■ ■ Attack scenario:

1. Attacker reaches Agent Tools Catalog via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:8 R:6 E:7 A:5 D:8 → **Total 34/50**

■ How to mitigate:

[*preventive*] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[*preventive*] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[*preventive*] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the

encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Sensitive data exposure at rest

STRIDE → Information Disclosure | Component: **Agent Tools Catalog** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Tools Catalog** (`datastore`).

■ **Description:** Stored PII / secrets accessible without proper authorization.

■■ Attack scenario:

1. Attacker reaches Agent Tools Catalog via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:8 R:6 E:7 A:5 D:8 → **Total 34/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Insecure direct object reference (IDOR)

STRIDE → Information Disclosure | Component: **Agent Tools Catalog** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Tools Catalog** (`datastore`).

■ **Description:** User can access another user's resources by guessing IDs.

■■ Attack scenario:

1. Attacker reaches Agent Tools Catalog via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:8 R:8 E:7 A:5 D:8 → **Total 36/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Trust-boundary crossing without strong authn: Agent Runtime / Orchestrator → Agent Tools Catalog [cross-boundary]

STRIDE → Spoofing | Component: **Agent Tools Catalog** (datastore) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 6.8 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Agent Runtime / Orchestrator** → **Agent Tools Catalog** (label: *Agent Runtime / Orchestrator → Agent Tools Catalog*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Agent Tools Catalog** (`datastore`).

■ **Description:** Flow 'Agent Runtime / Orchestrator → Agent Tools Catalog' crosses trust boundary 'Application tier' → 'Data tier'. Caller identity must be re-verified at the boundary; existing trust does not transit.

■ ■ Attack scenario:

1. Attacker positions themselves on the network path between source and destination, or compromises an upstream component in the source zone.
2. Crafts requests with forged or replayed identity claims (cookies, tokens, IP-based trust).
3. Agent Tools Catalog accepts the request because identity is not re-verified at the boundary, treating the upstream zone as trusted.
4. Attacker can now perform actions as the impersonated principal across the boundary.

DREAD — D:8 R:6 E:7 A:7 D:7 → **Total 35/50**

■ How to mitigate:

[preventive] **Re-authenticate at the boundary** — Require a fresh, audience-bound credential at the receiver. Do not infer identity from network position, source IP, or upstream session.

[preventive] **Use mutual TLS or signed tokens with audience claims** — For service-to-service calls into Agent Tools Catalog, terminate mTLS at the boundary and reject any caller without a valid client cert. For human-driven flows, use OAuth2 / OIDC with audience and issuer validation.

[detective] **Log every cross-boundary auth decision** — Emit an authentication event (success and failure) tagged with both source and destination zone. Forward to SIEM with retention ≥ 90 days.

A07:2021 — Identification and Authentication Failures | STRIDE reference | CWE-287 — Improper Authentication

Cross-boundary input not validated: Agent Runtime / Orchestrator → Agent Tools Catalog [cross-boundary]

STRIDE → Tampering | Component: **Agent Tools Catalog** (datastore) | Source: rule-based

CWE: CWE-20 — Improper Input Validation

CVSS 3.1: 8.7 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:L/SI:H/SA:N

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Agent Runtime / Orchestrator** → **Agent Tools Catalog** (label: *Agent Runtime / Orchestrator → Agent Tools Catalog*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Agent Tools Catalog** (`datastore`).

■ **Description:** Data crossing the trust boundary into 'Data tier' must be treated as untrusted, even if the source is internal. Implicit trust is the most common cause of injection / SSRF / deserialization bugs.

■ ■ Attack scenario:

1. Attacker compromises a less-trusted upstream zone or directly sends malformed input to Agent Tools Catalog.
2. Sends payloads with unexpected types, lengths, or special characters (e.g. SQL meta-chars, NUL bytes, control sequences).
3. Agent Tools Catalog processes the input under the assumption that the upstream zone has already validated it — but the boundary changes the security context.
4. Injection, deserialization flaws, or buffer issues trigger; attacker pivots into Agent Tools Catalog or the zone behind it.

DREAD — D:8 R:6 E:7 A:7 D:7 → **Total 35/50**

■ How to mitigate:

[preventive] **Validate against an allow-list schema at the boundary** — Define an explicit schema (JSON Schema, Protobuf, OpenAPI) for

every accepted message at this boundary. Reject anything that doesn't match — type, length, charset, enum values, nested depth.

[preventive] **Canonicalize before validation** — Decode URL-encoding, Unicode normalization (NFC), trim whitespace, and resolve relative paths before validating. Avoid double-decoding bypasses.

[preventive] **Apply context-specific output encoding** — Whatever Agent Tools Catalog does with the input — SQL: parameterized queries, HTML: contextual escaping, OS commands: avoid shell, use exec arrays, LDAP: escape filter chars.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-20 — Improper Input Validation](#)

Cross-boundary data exposure risk: Agent Runtime / Orchestrator → Agent Tools Catalog [cross-boundary]

STRIDE → Information Disclosure | Component: **Agent Tools Catalog** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 6.8 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Agent Runtime / Orchestrator** → **Agent Tools Catalog** (label: *Agent Runtime / Orchestrator → Agent Tools Catalog*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Agent Tools Catalog** (`datastore`).

■ **Description:** Information leaving 'Application tier' into 'Data tier' may include data the receiving zone is not authorized to see. Cross-boundary egress is a common data-leak surface.

■ ■ Attack scenario:

1. Attacker observes traffic leaving the source zone, or compromises a component in the destination zone.
2. The destination zone receives more data than it strictly needs (over-fetching, verbose error responses, full record dumps).
3. Attacker harvests sensitive fields (PII, secrets, internal IDs) that should never have left the source zone.
4. Data is exfiltrated, sold, or used to plan a deeper attack on the originating zone.

DREAD — D:8 R:6 E:7 A:7 D:8 → **Total 36/50**

■ How to mitigate:

[preventive] **Enforce minimum-data-needed at the egress point** — Whitelist exactly which fields leave the source zone. Strip everything else server-side before serialization.

[preventive] **Tokenize or redact sensitive fields** — Replace PII / PCI / PHI fields with reversible tokens or one-way hashes when the destination zone doesn't need the cleartext value.

[detective] **Log cross-boundary data flows for review** — Sample-log the field set crossing this boundary (not the values) so DLP and privacy reviews can audit what's egressing.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Authentication bypass via credential stuffing

STRIDE → Spoofing | Component: **Core Agent Layer** (api) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Core Agent Layer** (`api`).

■ **Description:** Attackers reuse leaked credentials to impersonate legitimate users.

■ ■ Attack scenario:

1. Attacker locates Core Agent Layer's identity claim mechanism (token format, header name, signing key handling).
2. Forges or replays a token to assert another principal's identity.
3. Core Agent Layer accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
4. Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:7 R:6 E:9 A:10 D:9 → **Total 41/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a

vettted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Session token hijacking

STRIDE → Spoofing | Component: **Core Agent Layer** (api) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Core Agent Layer** (`api`).

■ **Description:** Attacker steals or predicts a session token to impersonate a user.

■ ■ Attack scenario:

1. Attacker locates Core Agent Layer's identity claim mechanism (token format, header name, signing key handling).
2. Forges or replays a token to assert another principal's identity.
3. Core Agent Layer accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
4. Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:7 R:6 E:9 A:10 D:9 → **Total 41/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

API key / service identity spoofing

STRIDE → Spoofing | Component: **Core Agent Layer** (api) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Core Agent Layer** (`api`).

■ **Description:** An attacker obtains or forges an API key to act as a trusted service.

■ ■ Attack scenario:

1. Attacker locates Core Agent Layer's identity claim mechanism (token format, header name, signing key handling).
2. Forges or replays a token to assert another principal's identity.
3. Core Agent Layer accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
4. Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:7 R:6 E:9 A:10 D:9 → **Total 41/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Log tampering

STRIDE → Repudiation | Component: **Core Agent Layer** (api) | Source: rule-based

CWE: CWE-778 — Insufficient Logging

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:H/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Core Agent Layer** (`api`).

■ **Description:** An attacker with access modifies or deletes audit trails.

■ ■ Attack scenario:

1. User performs a sensitive action through Core Agent Layer (a transfer, a permission change, a deletion).
2. Audit logs are absent, incomplete, or modifiable by the same principal who performed the action.
3. User later denies having performed the action, or an attacker covers their tracks.
4. Without tamper-evident logs, neither responsible party can be identified — leading to fraud, dispute, or regulatory exposure.

DREAD — D:7 R:6 E:9 A:10 D:9 → **Total 41/50**

■ How to mitigate:

[*detective*] **Emit tamper-evident audit logs** — Sign log entries (HMAC chain, hash-linked) so insertion or deletion is detectable. Forward to a separate, append-only system the action's principal cannot administer.

[*detective*] **Capture sufficient detail per audit event** — Who (authenticated principal, not just session ID), what (specific action and target), when (UTC, monotonic-clock-corroborated), where (source IP, request ID), why (correlated to the upstream business event).

[*detective*] **Periodic log integrity verification** — Schedule daily integrity checks on the audit log chain. Alert on any gap.

[A09:2021 — Security Logging and Monitoring Failures](#) | [STRIDE reference](#) | [CWE-778 — Insufficient Logging](#)

Sensitive data exposure in transit

STRIDE → Information Disclosure | Component: **Core Agent Layer** (api) | Source: rule-based

CWE: CWE-319 — Cleartext Transmission of Sensitive Information

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Core Agent Layer** (`api`).

■ **Description:** PII, secrets, or tokens transmitted over unencrypted channels.

■ ■ Attack scenario:

1. Attacker reaches Core Agent Layer via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:6 E:9 A:10 D:10 → **Total 42/50**

■ How to mitigate:

[*preventive*] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[*preventive*] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[*preventive*] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-319 — Cleartext Transmission of Sensitive Information](#)

Sensitive data exposure at rest

STRIDE → Information Disclosure | Component: **Core Agent Layer** (api) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Core Agent Layer** (`api`).

■ **Description:** Stored PII / secrets accessible without proper authorization.

■■ **Attack scenario:**

1. Attacker reaches Core Agent Layer via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:6 E:9 A:10 D:10 → **Total 42/50**

■ **How to mitigate:**

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Insecure direct object reference (IDOR)

STRIDE → Information Disclosure | Component: **Core Agent Layer** (api) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Core Agent Layer** (`api`).

■ **Description:** User can access another user's resources by guessing IDs.

■■ **Attack scenario:**

1. Attacker reaches Core Agent Layer via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:8 E:9 A:10 D:10 → **Total 44/50**

■ **How to mitigate:**

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Application-layer DDoS

STRIDE → Denial of Service | Component: **Core Agent Layer** (api) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:H/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Core Agent Layer** (`api`).

■ **Description:** Attacker floods expensive endpoints (e.g., search, login).

■ ■ **Attack scenario:**

1. Attacker identifies a costly operation in Core Agent Layer (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Core Agent Layer's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Core Agent Layer; cascading failure may take down dependents.

DREAD — D:7 R:6 E:9 A:10 D:9 → **Total 41/50**

■ **How to mitigate:**

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Privilege escalation via mass assignment

STRIDE → Elevation of Privilege | Component: **Core Agent Layer** (api) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **Core Agent Layer** (`api`).

■ **Description:** User submits extra fields (e.g., role=admin) and they bind to the model.

■ ■ **Attack scenario:**

1. Attacker authenticates to Core Agent Layer as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:7 R:8 E:9 A:10 D:9 → **Total 43/50**

■ **How to mitigate:**

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to Core Agent Layer should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Trust-boundary crossing without strong authn: Agent Gateway → Core Agent Layer [cross-boundary]

STRIDE → Spoofing | Component: **Core Agent Layer** (api) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: DMZ (public-facing) → Application tier

■ **Location:** On the data flow **Agent Gateway** → **Core Agent Layer** (label: *Agent Gateway → Core Agent Layer*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `DMZ (public-facing)` into `Application tier`. The receiving component is **Core Agent Layer** (`api`).

■ **Description:** Flow 'Agent Gateway → Core Agent Layer' crosses trust boundary 'DMZ (public-facing)' → 'Application tier'. Caller identity must be re-verified at the boundary; existing trust does not transit.

■ ■ **Attack scenario:**

1. Attacker positions themselves on the network path between source and destination, or compromises an upstream component in the source zone.
2. Crafts requests with forged or replayed identity claims (cookies, tokens, IP-based trust).
3. Core Agent Layer accepts the request because identity is not re-verified at the boundary, treating the upstream zone as trusted.
4. Attacker can now perform actions as the impersonated principal across the boundary.

DREAD — D:7 R:6 E:9 A:10 D:9 → **Total 41/50**

■ **How to mitigate:**

[preventive] **Re-authenticate at the boundary** — Require a fresh, audience-bound credential at the receiver. Do not infer identity from network position, source IP, or upstream session.

[preventive] **Use mutual TLS or signed tokens with audience claims** — For service-to-service calls into Core Agent Layer, terminate mTLS at the boundary and reject any caller without a valid client cert. For human-driven flows, use OAuth2 / OIDC with audience and issuer validation.

[detective] **Log every cross-boundary auth decision** — Emit an authentication event (success and failure) tagged with both source and destination zone. Forward to SIEM with retention ≥ 90 days.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Cross-boundary input not validated: Agent Gateway → Core Agent Layer [cross-boundary]

STRIDE → Tampering | Component: **Core Agent Layer** (api) | Source: rule-based

CWE: CWE-20 — Improper Input Validation

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:N/I:H/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:N/VI:H/VA:N/SC:L/SI:H/SA:N

Boundary crossing: DMZ (public-facing) → Application tier

■ **Location:** On the data flow **Agent Gateway → Core Agent Layer** (label: *Agent Gateway → Core Agent Layer*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `DMZ (public-facing)` into `Application tier`. The receiving component is **Core Agent Layer** (`api`).

■ **Description:** Data crossing the trust boundary into 'Application tier' must be treated as untrusted, even if the source is internal. Implicit trust is the most common cause of injection / SSRF / deserialization bugs.

■ ■ **Attack scenario:**

1. Attacker compromises a less-trusted upstream zone or directly sends malformed input to Core Agent Layer.
2. Sends payloads with unexpected types, lengths, or special characters (e.g. SQL meta-chars, NUL bytes, control sequences).
3. Core Agent Layer processes the input under the assumption that the upstream zone has already validated it — but the boundary changes the security context.
4. Injection, deserialization flaws, or buffer issues trigger; attacker pivots into Core Agent Layer or the zone behind it.

DREAD — D:7 R:6 E:9 A:10 D:9 → **Total 41/50**

■ **How to mitigate:**

[preventive] **Validate against an allow-list schema at the boundary** — Define an explicit schema (JSON Schema, Protobuf, OpenAPI) for every accepted message at this boundary. Reject anything that doesn't match — type, length, charset, enum values, nested depth.

[preventive] **Canonicalize before validation** — Decode URL-encoding, Unicode normalization (NFC), trim whitespace, and resolve relative paths before validating. Avoid double-decoding bypasses.

[preventive] **Apply context-specific output encoding** — Whatever Core Agent Layer does with the input — SQL: parameterized queries, HTML: contextual escaping, OS commands: avoid shell, use exec arrays, LDAP: escape filter chars.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-20 — Improper Input Validation](#)

Cross-boundary data exposure risk: Agent Gateway → Core Agent Layer [cross-boundary]

STRIDE → Information Disclosure | Component: **Core Agent Layer** (api) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) [CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N](#)

Boundary crossing: DMZ (public-facing) → Application tier

■ **Location:** On the data flow **Agent Gateway** → **Core Agent Layer** (label: *Agent Gateway → Core Agent Layer*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `DMZ (public-facing)` into `Application tier`. The receiving component is **Core Agent Layer** (`api`).

■ **Description:** Information leaving 'DMZ (public-facing)' into 'Application tier' may include data the receiving zone is not authorized to see. Cross-boundary egress is a common data-leak surface.

■ ■ **Attack scenario:**

1. Attacker observes traffic leaving the source zone, or compromises a component in the destination zone.
2. The destination zone receives more data than it strictly needs (over-fetching, verbose error responses, full record dumps).
3. Attacker harvests sensitive fields (PII, secrets, internal IDs) that should never have left the source zone.
4. Data is exfiltrated, sold, or used to plan a deeper attack on the originating zone.

DREAD — D:7 R:6 E:9 A:10 D:10 → **Total 42/50**

■ **How to mitigate:**

[preventive] **Enforce minimum-data-needed at the egress point** — Whitelist exactly which fields leave the source zone. Strip everything else server-side before serialization.

[preventive] **Tokenize or redact sensitive fields** — Replace PII / PCI / PHI fields with reversible tokens or one-way hashes when the destination zone doesn't need the cleartext value.

[detective] **Log cross-boundary data flows for review** — Sample-log the field set crossing this boundary (not the values) so DLP and privacy reviews can audit what's egressing.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Authentication bypass via credential stuffing

STRIDE → Spoofing | Component: **CrowdStrike AIDR** (external_entity) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **CrowdStrike AIDR** (`external\_entity`).

■ **Description:** Attackers reuse leaked credentials to impersonate legitimate users.

■ ■ **Attack scenario:**

1. Attacker obtains valid credentials via phishing, credential stuffing, or a third-party breach.
2. Logs in as the legitimate user/entity at CrowdStrike AIDR.
3. Performs actions indistinguishable from the real user (auth events look normal).
4. Damage scales with the impersonated user's permissions; admin accounts cause the most harm.

DREAD — D:7 R:6 E:9 A:5 D:9 → **Total 36/50**

■ **How to mitigate:**

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Session token hijacking

STRIDE → Spoofing | Component: **CrowdStrike AIDR** (external_entity) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **CrowdStrike AIDR** (`external\_entity`).

■ **Description:** Attacker steals or predicts a session token to impersonate a user.

■ ■ **Attack scenario:**

1. Attacker obtains valid credentials via phishing, credential stuffing, or a third-party breach.
2. Logs in as the legitimate user/entity at CrowdStrike AIDR.
3. Performs actions indistinguishable from the real user (auth events look normal).
4. Damage scales with the impersonated user's permissions; admin accounts cause the most harm.

DREAD — D:7 R:6 E:9 A:5 D:9 → **Total 36/50**

■ **How to mitigate:**

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

API key / service identity spoofing

STRIDE → Spoofing | Component: **CrowdStrike AIDR** (external_entity) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **CrowdStrike AIDR** (`external\_entity`).

■ **Description:** An attacker obtains or forges an API key to act as a trusted service.

■ ■ **Attack scenario:**

1. Attacker obtains valid credentials via phishing, credential stuffing, or a third-party breach.
2. Logs in as the legitimate user/entity at CrowdStrike AIDR.
3. Performs actions indistinguishable from the real user (auth events look normal).
4. Damage scales with the impersonated user's permissions; admin accounts cause the most harm.

DREAD — D:7 R:6 E:9 A:5 D:9 → **Total 36/50**

■ **How to mitigate:**

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Sensitive data exposure in transit

STRIDE → Information Disclosure | Component: **CrowdStrike AIDR** (external_entity) | Source: rule-based

CWE: CWE-319 — Cleartext Transmission of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **CrowdStrike AIDR** (`external\_entity`).

■ **Description:** PII, secrets, or tokens transmitted over unencrypted channels.

■ ■ **Attack scenario:**

1. Attacker reaches CrowdStrike AIDR via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).

3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:6 E:9 A:5 D:10 → **Total 37/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For external_entity, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-319 — Cleartext Transmission of Sensitive Information](#)

Sensitive data exposure at rest

STRIDE → Information Disclosure | Component: **CrowdStrike AIDR** (external_entity) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **CrowdStrike AIDR** (external_entity).

■ **Description:** Stored PII / secrets accessible without proper authorization.

■■ Attack scenario:

1. Attacker reaches CrowdStrike AIDR via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:6 E:9 A:5 D:10 → **Total 37/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For external_entity, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Insecure direct object reference (IDOR)

STRIDE → Information Disclosure | Component: **CrowdStrike AIDR** (external_entity) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **CrowdStrike AIDR** (external_entity).

■ **Description:** User can access another user's resources by guessing IDs.

■■ Attack scenario:

1. Attacker reaches CrowdStrike AIDR via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:8 E:9 A:5 D:10 → **Total 39/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For external_entity, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Application-layer DDoS

STRIDE → Denial of Service | Component: **CrowdStrike AIDR** (external_entity) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **CrowdStrike AIDR** (external_entity).

■ **Description:** Attacker floods expensive endpoints (e.g., search, login).

■■ Attack scenario:

1. Attacker identifies a costly operation in CrowdStrike AIDR (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. CrowdStrike AIDR's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach CrowdStrike AIDR; cascading failure may take down dependents.

DREAD — D:7 R:6 E:9 A:5 D:9 → **Total 36/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Trust-boundary crossing without strong authn: AI Detection & Response → CrowdStrike AIDR [cross-boundary]

STRIDE → Spoofing | Component: **CrowdStrike AIDR** (external_entity) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Application tier → Internet

■ **Location:** On the data flow **AI Detection & Response → CrowdStrike AIDR** (label: *AI Detection & Response → CrowdStrike AIDR*, protocol: HTTPS, auth: api_key, encrypted: yes), crossing the trust boundary from 'Application tier' into 'Internet'. The receiving component is **CrowdStrike AIDR** (external_entity).

■ **Description:** Flow 'AI Detection & Response → CrowdStrike AIDR' crosses trust boundary 'Application tier' → 'Internet'. Caller identity must be re-verified at the boundary; existing trust does not transit.

■■ Attack scenario:

1. Attacker positions themselves on the network path between source and destination, or compromises an upstream component in the source zone.
2. Crafts requests with forged or replayed identity claims (cookies, tokens, IP-based trust).
3. CrowdStrike AIDR accepts the request because identity is not re-verified at the boundary, treating the upstream zone as

trusted.

4. Attacker can now perform actions as the impersonated principal across the boundary.

DREAD — D:7 R:6 E:9 A:7 D:9 → **Total 38/50**

■ How to mitigate:

[preventive] **Re-authenticate at the boundary** — Require a fresh, audience-bound credential at the receiver. Do not infer identity from network position, source IP, or upstream session.

[preventive] **Use mutual TLS or signed tokens with audience claims** — For service-to-service calls into CrowdStrike ADR, terminate mTLS at the boundary and reject any caller without a valid client cert. For human-driven flows, use OAuth2 / OIDC with audience and issuer validation.

[detective] **Log every cross-boundary auth decision** — Emit an authentication event (success and failure) tagged with both source and destination zone. Forward to SIEM with retention ≥ 90 days.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Cross-boundary input not validated: AI Detection & Response → CrowdStrike ADR [cross-boundary]

STRIDE → Tampering | Component: **CrowdStrike ADR** (external_entity) | Source: rule-based

CWE: CWE-20 — Improper Input Validation

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:N/I:H/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:N/VI:H/VA:N/SC:L/SI:H/SA:N

Boundary crossing: Application tier → Internet

■ **Location:** On the data flow **AI Detection & Response** → **CrowdStrike ADR** (label: *AI Detection & Response → CrowdStrike ADR*, protocol: HTTPS, auth: api_key, encrypted: yes), crossing the trust boundary from `Application tier` into `Internet`. The receiving component is **CrowdStrike ADR** (external_entity).

■ **Description:** Data crossing the trust boundary into 'Internet' must be treated as untrusted, even if the source is internal. Implicit trust is the most common cause of injection / SSRF / deserialization bugs.

■■ Attack scenario:

1. Attacker compromises a less-trusted upstream zone or directly sends malformed input to CrowdStrike ADR.
2. Sends payloads with unexpected types, lengths, or special characters (e.g. SQL meta-chars, NUL bytes, control sequences).
3. CrowdStrike ADR processes the input under the assumption that the upstream zone has already validated it — but the boundary changes the security context.
4. Injection, deserialization flaws, or buffer issues trigger; attacker pivots into CrowdStrike ADR or the zone behind it.

DREAD — D:7 R:6 E:9 A:7 D:9 → **Total 38/50**

■ How to mitigate:

[preventive] **Validate against an allow-list schema at the boundary** — Define an explicit schema (JSON Schema, Protobuf, OpenAPI) for every accepted message at this boundary. Reject anything that doesn't match — type, length, charset, enum values, nested depth.

[preventive] **Canonicalize before validation** — Decode URL-encoding, Unicode normalization (NFC), trim whitespace, and resolve relative paths before validating. Avoid double-decoding bypasses.

[preventive] **Apply context-specific output encoding** — Whatever CrowdStrike ADR does with the input — SQL: parameterized queries, HTML: contextual escaping, OS commands: avoid shell, use exec arrays, LDAP: escape filter chars.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-20 — Improper Input Validation](#)

Cross-boundary data exposure risk: AI Detection & Response → CrowdStrike ADR [cross-boundary]

STRIDE → Information Disclosure | Component: **CrowdStrike ADR** (external_entity) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Application tier → Internet

■ **Location:** On the data flow **AI Detection & Response** → **CrowdStrike ADR** (label: *AI Detection & Response → CrowdStrike ADR*, protocol: HTTPS, auth: api_key, encrypted: yes), crossing the trust boundary from `Application tier` into `Internet`. The receiving component is **CrowdStrike ADR** (external_entity).

■ **Description:** Information leaving 'Application tier' into 'Internet' may include data the receiving zone is not authorized to see. Cross-boundary egress is a common data-leak surface.

■ ■ **Attack scenario:**

1. Attacker observes traffic leaving the source zone, or compromises a component in the destination zone.
2. The destination zone receives more data than it strictly needs (over-fetching, verbose error responses, full record dumps).
3. Attacker harvests sensitive fields (PII, secrets, internal IDs) that should never have left the source zone.
4. Data is exfiltrated, sold, or used to plan a deeper attack on the originating zone.

DREAD — D:7 R:6 E:9 A:7 D:10 → **Total 39/50**

■ **How to mitigate:**

[preventive] **Enforce minimum-data-needed at the egress point** — Whitelist exactly which fields leave the source zone. Strip everything else server-side before serialization.

[preventive] **Tokenize or redact sensitive fields** — Replace PII / PCI / PHI fields with reversible tokens or one-way hashes when the destination zone doesn't need the cleartext value.

[detective] **Log cross-boundary data flows for review** — Sample-log the field set crossing this boundary (not the values) so DLP and privacy reviews can audit what's egressing.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Authentication bypass via credential stuffing

STRIDE → Spoofing | Component: **Identity Provider / Okta MCP** (auth_service) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Identity Provider / Okta MCP** (`auth\_service`).

■ **Description:** Attackers reuse leaked credentials to impersonate legitimate users.

■ ■ **Attack scenario:**

1. Attacker locates Identity Provider / Okta MCP's identity claim mechanism (token format, header name, signing key handling).
2. Forges or replays a token to assert another principal's identity.
3. Identity Provider / Okta MCP accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
4. Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:8 R:6 E:9 A:8 D:9 → **Total 40/50**

■ **How to mitigate:**

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Session token hijacking

STRIDE → Spoofing | Component: **Identity Provider / Okta MCP** (auth_service) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Identity Provider / Okta MCP** (`auth\_service`).

■ **Description:** Attacker steals or predicts a session token to impersonate a user.

■ ■ **Attack scenario:**

1. Attacker locates Identity Provider / Okta MCP's identity claim mechanism (token format, header name, signing key handling).
2. Forges or replays a token to assert another principal's identity.

3. Identity Provider / Okta MCP accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
4. Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:8 R:6 E:9 A:8 D:9 → **Total 40/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

API key / service identity spoofing

STRIDE → Spoofing | Component: **Identity Provider / Okta MCP** (auth_service) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Identity Provider / Okta MCP** (`auth\_service`).

■ **Description:** An attacker obtains or forges an API key to act as a trusted service.

■■ Attack scenario:

1. Attacker locates Identity Provider / Okta MCP's identity claim mechanism (token format, header name, signing key handling).
2. Forges or replays a token to assert another principal's identity.
3. Identity Provider / Okta MCP accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
4. Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:8 R:6 E:9 A:8 D:9 → **Total 40/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Log tampering

STRIDE → Repudiation | Component: **Identity Provider / Okta MCP** (auth_service) | Source: rule-based

CWE: CWE-778 — Insufficient Logging

CVSS 3.1: 9.1 (Critical) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Identity Provider / Okta MCP** (`auth\_service`).

■ **Description:** An attacker with access modifies or deletes audit trails.

■■ Attack scenario:

1. User performs a sensitive action through Identity Provider / Okta MCP (a transfer, a permission change, a deletion).
2. Audit logs are absent, incomplete, or modifiable by the same principal who performed the action.
3. User later denies having performed the action, or an attacker covers their tracks.
4. Without tamper-evident logs, neither responsible party can be identified — leading to fraud, dispute, or regulatory exposure.

DREAD — D:8 R:6 E:9 A:8 D:9 → **Total 40/50**

■ How to mitigate:

[*detective*] **Emit tamper-evident audit logs** — Sign log entries (HMAC chain, hash-linked) so insertion or deletion is detectable. Forward to a separate, append-only system the action's principal cannot administer.

[*detective*] **Capture sufficient detail per audit event** — Who (authenticated principal, not just session ID), what (specific action and target), when (UTC, monotonic-clock-corroborated), where (source IP, request ID), why (correlated to the upstream business event).

[*detective*] **Periodic log integrity verification** — Schedule daily integrity checks on the audit log chain. Alert on any gap.

[A09:2021 — Security Logging and Monitoring Failures](#) | [STRIDE reference](#) | [CWE-778 — Insufficient Logging](#)

Application-layer DDoS

[STRIDE](#) → Denial of Service | Component: **Identity Provider / Okta MCP** (auth_service) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 9.1 (Critical) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **Identity Provider / Okta MCP** (`auth\_service`).

■ **Description:** Attacker floods expensive endpoints (e.g., search, login).

■■ Attack scenario:

1. Attacker identifies a costly operation in Identity Provider / Okta MCP (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Identity Provider / Okta MCP's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Identity Provider / Okta MCP; cascading failure may take down dependents.

DREAD — D:8 R:6 E:9 A:8 D:9 → **Total 40/50**

■ How to mitigate:

[*preventive*] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[*preventive*] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[*corrective*] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Privilege escalation via mass assignment

[STRIDE](#) → Elevation of Privilege | Component: **Identity Provider / Okta MCP** (auth_service) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **Identity Provider / Okta MCP** (`auth\_service`).

■ **Description:** User submits extra fields (e.g., role=admin) and they bind to the model.

■■ Attack scenario:

1. Attacker authenticates to Identity Provider / Okta MCP as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:8 R:8 E:9 A:8 D:9 → **Total 42/50**

■ How to mitigate:

[*preventive*] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[*preventive*] **Apply least-privilege to service identities** — The IAM/role attached to Identity Provider / Okta MCP should grant only the actions it

actually performs. Audit IAM grants quarterly.

[*detective*] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Trust-boundary crossing without strong authn: Agent Gateway → Identity Provider / Okta MCP [cross-boundary]

STRIDE → Spoofing | Component: **Identity Provider / Okta MCP** (auth_service) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: DMZ (public-facing) → Third-party / Untrusted services

■ **Location:** On the data flow **Agent Gateway → Identity Provider / Okta MCP** (label: *Agent Gateway → Identity Provider / Okta MCP*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `DMZ (public-facing)` into `Third-party / Untrusted services`. The receiving component is **Identity Provider / Okta MCP** (`auth\_service`).

■ **Description:** Flow 'Agent Gateway → Identity Provider / Okta MCP' crosses trust boundary 'DMZ (public-facing)' → 'Third-party / Untrusted services'. Caller identity must be re-verified at the boundary; existing trust does not transit.

■ ■ Attack scenario:

1. Attacker positions themselves on the network path between source and destination, or compromises an upstream component in the source zone.
2. Crafts requests with forged or replayed identity claims (cookies, tokens, IP-based trust).
3. Identity Provider / Okta MCP accepts the request because identity is not re-verified at the boundary, treating the upstream zone as trusted.
4. Attacker can now perform actions as the impersonated principal across the boundary.

DREAD — D:8 R:6 E:9 A:10 D:9 → **Total 42/50**

■ How to mitigate:

[*preventive*] **Re-authenticate at the boundary** — Require a fresh, audience-bound credential at the receiver. Do not infer identity from network position, source IP, or upstream session.

[*preventive*] **Use mutual TLS or signed tokens with audience claims** — For service-to-service calls into Identity Provider / Okta MCP, terminate mTLS at the boundary and reject any caller without a valid client cert. For human-driven flows, use OAuth2 / OIDC with audience and issuer validation.

[*detective*] **Log every cross-boundary auth decision** — Emit an authentication event (success and failure) tagged with both source and destination zone. Forward to SIEM with retention ≥ 90 days.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Cross-boundary input not validated: Agent Gateway → Identity Provider / Okta MCP [cross-boundary]

STRIDE → Tampering | Component: **Identity Provider / Okta MCP** (auth_service) | Source: rule-based

CWE: CWE-20 — Improper Input Validation

CVSS 3.1: 9.6 (Critical) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:L/SI:H/SA:N

Boundary crossing: DMZ (public-facing) → Third-party / Untrusted services

■ **Location:** On the data flow **Agent Gateway → Identity Provider / Okta MCP** (label: *Agent Gateway → Identity Provider / Okta MCP*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `DMZ (public-facing)` into `Third-party / Untrusted services`. The receiving component is **Identity Provider / Okta MCP** (`auth\_service`).

■ **Description:** Data crossing the trust boundary into 'Third-party / Untrusted services' must be treated as untrusted, even if the source is internal. Implicit trust is the most common cause of injection / SSRF / deserialization bugs.

■ ■ Attack scenario:

1. Attacker compromises a less-trusted upstream zone or directly sends malformed input to Identity Provider / Okta MCP.
2. Sends payloads with unexpected types, lengths, or special characters (e.g. SQL meta-chars, NUL bytes, control sequences).

3. Identity Provider / Okta MCP processes the input under the assumption that the upstream zone has already validated it — but the boundary changes the security context.
4. Injection, deserialization flaws, or buffer issues trigger; attacker pivots into Identity Provider / Okta MCP or the zone behind it.

DREAD — D:8 R:6 E:9 A:10 D:9 → **Total 42/50**

■ How to mitigate:

[preventive] **Validate against an allow-list schema at the boundary** — Define an explicit schema (JSON Schema, Protobuf, OpenAPI) for every accepted message at this boundary. Reject anything that doesn't match — type, length, charset, enum values, nested depth.

[preventive] **Canonicalize before validation** — Decode URL-encoding, Unicode normalization (NFC), trim whitespace, and resolve relative paths before validating. Avoid double-decoding bypasses.

[preventive] **Apply context-specific output encoding** — Whatever Identity Provider / Okta MCP does with the input — SQL: parameterized queries, HTML: contextual escaping, OS commands: avoid shell, use exec arrays, LDAP: escape filter chars.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-20 — Improper Input Validation](#)

Cross-boundary data exposure risk: Agent Gateway → Identity Provider / Okta MCP [cross-boundary]

STRIDE → Information Disclosure | Component: **Identity Provider / Okta MCP** (auth_service) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: DMZ (public-facing) → Third-party / Untrusted services

■ **Location:** On the data flow **Agent Gateway → Identity Provider / Okta MCP** (label: *Agent Gateway → Identity Provider / Okta MCP*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `DMZ (public-facing)` into `Third-party / Untrusted services`. The receiving component is **Identity Provider / Okta MCP** (`auth\_service`).

■ **Description:** Information leaving 'DMZ (public-facing)' into 'Third-party / Untrusted services' may include data the receiving zone is not authorized to see. Cross-boundary egress is a common data-leak surface.

■■ Attack scenario:

1. Attacker observes traffic leaving the source zone, or compromises a component in the destination zone.
2. The destination zone receives more data than it strictly needs (over-fetching, verbose error responses, full record dumps).
3. Attacker harvests sensitive fields (PII, secrets, internal IDs) that should never have left the source zone.
4. Data is exfiltrated, sold, or used to plan a deeper attack on the originating zone.

DREAD — D:8 R:6 E:9 A:10 D:10 → **Total 43/50**

■ How to mitigate:

[preventive] **Enforce minimum-data-needed at the egress point** — Whitelist exactly which fields leave the source zone. Strip everything else server-side before serialization.

[preventive] **Tokenize or redact sensitive fields** — Replace PII / PCI / PHI fields with reversible tokens or one-way hashes when the destination zone doesn't need the cleartext value.

[detective] **Log cross-boundary data flows for review** — Sample-log the field set crossing this boundary (not the values) so DLP and privacy reviews can audit what's egressing.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Authentication bypass via credential stuffing

STRIDE → Spoofing | Component: **LLM Models** (external_entity) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **LLM Models** (`external\_entity`).

■ **Description:** Attackers reuse leaked credentials to impersonate legitimate users.

■■ Attack scenario:

1. Attacker obtains valid credentials via phishing, credential stuffing, or a third-party breach.

2. Logs in as the legitimate user/entity at LLM Models.
3. Performs actions indistinguishable from the real user (auth events look normal).
4. Damage scales with the impersonated user's permissions; admin accounts cause the most harm.

DREAD — D:7 R:6 E:9 A:5 D:9 → **Total 36/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Session token hijacking

STRIDE → Spoofing | Component: **LLM Models** (external_entity) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **LLM Models** (`external\_entity`).

■ **Description:** Attacker steals or predicts a session token to impersonate a user.

■■ Attack scenario:

1. Attacker obtains valid credentials via phishing, credential stuffing, or a third-party breach.
2. Logs in as the legitimate user/entity at LLM Models.
3. Performs actions indistinguishable from the real user (auth events look normal).
4. Damage scales with the impersonated user's permissions; admin accounts cause the most harm.

DREAD — D:7 R:6 E:9 A:5 D:9 → **Total 36/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

API key / service identity spoofing

STRIDE → Spoofing | Component: **LLM Models** (external_entity) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **LLM Models** (`external\_entity`).

■ **Description:** An attacker obtains or forges an API key to act as a trusted service.

■■ Attack scenario:

1. Attacker obtains valid credentials via phishing, credential stuffing, or a third-party breach.
2. Logs in as the legitimate user/entity at LLM Models.
3. Performs actions indistinguishable from the real user (auth events look normal).
4. Damage scales with the impersonated user's permissions; admin accounts cause the most harm.

DREAD — D:7 R:6 E:9 A:5 D:9 → **Total 36/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021](#) — [Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287](#) — [Improper Authentication](#)

Sensitive data exposure in transit

STRIDE → Information Disclosure | Component: **LLM Models** (external_entity) | Source: rule-based

CWE: CWE-319 — Cleartext Transmission of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **LLM Models** (`external\_entity`).

■ **Description:** PII, secrets, or tokens transmitted over unencrypted channels.

■■ Attack scenario:

1. Attacker reaches LLM Models via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:6 E:9 A:5 D:10 → **Total 37/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For external_entity, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021](#) — [Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-319](#) — [Cleartext Transmission of Sensitive Information](#)

Sensitive data exposure at rest

STRIDE → Information Disclosure | Component: **LLM Models** (external_entity) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **LLM Models** (`external\_entity`).

■ **Description:** Stored PII / secrets accessible without proper authorization.

■■ Attack scenario:

1. Attacker reaches LLM Models via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:6 E:9 A:5 D:10 → **Total 37/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces,

SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For external_entity, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Insecure direct object reference (IDOR)

STRIDE → Information Disclosure | Component: **LLM Models** (external_entity) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **LLM Models** (external_entity).

■ **Description:** User can access another user's resources by guessing IDs.

■ ■ Attack scenario:

1. Attacker reaches LLM Models via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:8 E:9 A:5 D:10 → **Total 39/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For external_entity, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Application-layer DDoS

STRIDE → Denial of Service | Component: **LLM Models** (external_entity) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **LLM Models** (external_entity).

■ **Description:** Attacker floods expensive endpoints (e.g., search, login).

■ ■ Attack scenario:

1. Attacker identifies a costly operation in LLM Models (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. LLM Models's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach LLM Models; cascading failure may take down dependents.

DREAD — D:7 R:6 E:9 A:5 D:9 → **Total 36/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Trust-boundary crossing without strong authn: Model Gateway → LLM Models [cross-boundary]

STRIDE → Spoofing | Component: **LLM Models** (external_entity) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: DMZ (public-facing) → Internet

■ **Location:** On the data flow **Model Gateway → LLM Models** (label: *Model Gateway → LLM Models*, protocol: HTTPS, auth: api_key, encrypted: yes), crossing the trust boundary from `DMZ (public-facing)` into `Internet`. The receiving component is **LLM Models** (`external\_entity`).

■ **Description:** Flow 'Model Gateway → LLM Models' crosses trust boundary 'DMZ (public-facing)' → 'Internet'. Caller identity must be re-verified at the boundary; existing trust does not transit.

■ ■ Attack scenario:

1. Attacker positions themselves on the network path between source and destination, or compromises an upstream component in the source zone.
2. Crafts requests with forged or replayed identity claims (cookies, tokens, IP-based trust).
3. LLM Models accepts the request because identity is not re-verified at the boundary, treating the upstream zone as trusted.
4. Attacker can now perform actions as the impersonated principal across the boundary.

DREAD — D:7 R:6 E:9 A:7 D:9 → **Total 38/50**

■ How to mitigate:

[preventive] **Re-authenticate at the boundary** — Require a fresh, audience-bound credential at the receiver. Do not infer identity from network position, source IP, or upstream session.

[preventive] **Use mutual TLS or signed tokens with audience claims** — For service-to-service calls into LLM Models, terminate mTLS at the boundary and reject any caller without a valid client cert. For human-driven flows, use OAuth2 / OIDC with audience and issuer validation.

[detective] **Log every cross-boundary auth decision** — Emit an authentication event (success and failure) tagged with both source and destination zone. Forward to SIEM with retention ≥ 90 days.

A07:2021 — Identification and Authentication Failures | STRIDE reference | CWE-287 — Improper Authentication

Cross-boundary input not validated: Model Gateway → LLM Models [cross-boundary]

STRIDE → Tampering | Component: **LLM Models** (external_entity) | Source: rule-based

CWE: CWE-20 — Improper Input Validation

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:N/I:H/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:N/VI:H/VA:N/SC:L/SI:H/SA:N

Boundary crossing: DMZ (public-facing) → Internet

■ **Location:** On the data flow **Model Gateway → LLM Models** (label: *Model Gateway → LLM Models*, protocol: HTTPS, auth: api_key, encrypted: yes), crossing the trust boundary from `DMZ (public-facing)` into `Internet`. The receiving component is **LLM Models** (`external\_entity`).

■ **Description:** Data crossing the trust boundary into 'Internet' must be treated as untrusted, even if the source is internal. Implicit trust is the most common cause of injection / SSRF / deserialization bugs.

■ ■ Attack scenario:

1. Attacker compromises a less-trusted upstream zone or directly sends malformed input to LLM Models.
2. Sends payloads with unexpected types, lengths, or special characters (e.g. SQL meta-chars, NUL bytes, control sequences).
3. LLM Models processes the input under the assumption that the upstream zone has already validated it — but the boundary changes the security context.
4. Injection, deserialization flaws, or buffer issues trigger; attacker pivots into LLM Models or the zone behind it.

DREAD — D:7 R:6 E:9 A:7 D:9 → **Total 38/50**

■ How to mitigate:

[preventive] **Validate against an allow-list schema at the boundary** — Define an explicit schema (JSON Schema, Protobuf, OpenAPI) for every accepted message at this boundary. Reject anything that doesn't match — type, length, charset, enum values, nested depth.

[preventive] **Canonicalize before validation** — Decode URL-encoding, Unicode normalization (NFC), trim whitespace, and resolve relative paths before validating. Avoid double-decoding bypasses.

[preventive] **Apply context-specific output encoding** — Whatever LLM Models does with the input — SQL: parameterized queries, HTML: contextual escaping, OS commands: avoid shell, use exec arrays, LDAP: escape filter chars.

[A03:2021](#) — [Injection](#) | [STRIDE reference](#) | [CWE-20](#) — [Improper Input Validation](#)

Cross-boundary data exposure risk: Model Gateway → LLM Models [cross-boundary]

STRIDE → Information Disclosure | Component: **LLM Models** (external_entity) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: DMZ (public-facing) → Internet

■ **Location:** On the data flow **Model Gateway → LLM Models** (label: *Model Gateway → LLM Models*, protocol: HTTPS, auth: api_key, encrypted: yes), crossing the trust boundary from `DMZ (public-facing)` into `Internet`. The receiving component is **LLM Models** (`external\_entity`).

■ **Description:** Information leaving 'DMZ (public-facing)' into 'Internet' may include data the receiving zone is not authorized to see. Cross-boundary egress is a common data-leak surface.

■ ■ Attack scenario:

1. Attacker observes traffic leaving the source zone, or compromises a component in the destination zone.
2. The destination zone receives more data than it strictly needs (over-fetching, verbose error responses, full record dumps).
3. Attacker harvests sensitive fields (PII, secrets, internal IDs) that should never have left the source zone.
4. Data is exfiltrated, sold, or used to plan a deeper attack on the originating zone.

DREAD — D:7 R:6 E:9 A:7 D:10 → **Total 39/50**

■ How to mitigate:

[preventive] **Enforce minimum-data-needed at the egress point** — Whitelist exactly which fields leave the source zone. Strip everything else server-side before serialization.

[preventive] **Tokenize or redact sensitive fields** — Replace PII / PCI / PHI fields with reversible tokens or one-way hashes when the destination zone doesn't need the cleartext value.

[detective] **Log cross-boundary data flows for review** — Sample-log the field set crossing this boundary (not the values) so DLP and privacy reviews can audit what's egressing.

[A02:2021](#) — [Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200](#) — [Exposure of Sensitive Information](#)

Authentication bypass via credential stuffing

STRIDE → Spoofing | Component: **Lambda Functions** (api) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Lambda Functions** (`api`).

■ **Description:** Attackers reuse leaked credentials to impersonate legitimate users.

■ ■ Attack scenario:

1. Attacker locates Lambda Functions's identity claim mechanism (token format, header name, signing key handling).
2. Forges or replays a token to assert another principal's identity.
3. Lambda Functions accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
4. Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:7 R:6 E:7 A:7 D:7 → **Total 34/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a

vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Session token hijacking

STRIDE → Spoofing | Component: **Lambda Functions** (api) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Lambda Functions** (`api`).

■ **Description:** Attacker steals or predicts a session token to impersonate a user.

■ ■ Attack scenario:

1. Attacker locates Lambda Functions's identity claim mechanism (token format, header name, signing key handling).
2. Forges or replays a token to assert another principal's identity.
3. Lambda Functions accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
4. Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:7 R:6 E:7 A:7 D:7 → **Total 34/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

API key / service identity spoofing

STRIDE → Spoofing | Component: **Lambda Functions** (api) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Lambda Functions** (`api`).

■ **Description:** An attacker obtains or forges an API key to act as a trusted service.

■ ■ Attack scenario:

1. Attacker locates Lambda Functions's identity claim mechanism (token format, header name, signing key handling).
2. Forges or replays a token to assert another principal's identity.
3. Lambda Functions accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
4. Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:7 R:6 E:7 A:7 D:7 → **Total 34/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Log tampering

STRIDE → Repudiation | Component: **Lambda Functions** (api) | Source: rule-based

CWE: CWE-778 — Insufficient Logging

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:H/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Lambda Functions** (`api`).

■ **Description:** An attacker with access modifies or deletes audit trails.

■ ■ Attack scenario:

1. User performs a sensitive action through Lambda Functions (a transfer, a permission change, a deletion).
2. Audit logs are absent, incomplete, or modifiable by the same principal who performed the action.
3. User later denies having performed the action, or an attacker covers their tracks.
4. Without tamper-evident logs, neither responsible party can be identified — leading to fraud, dispute, or regulatory exposure.

DREAD — D:7 R:6 E:7 A:7 D:7 → **Total 34/50**

■ How to mitigate:

[*detective*] **Emit tamper-evident audit logs** — Sign log entries (HMAC chain, hash-linked) so insertion or deletion is detectable. Forward to a separate, append-only system the action's principal cannot administer.

[*detective*] **Capture sufficient detail per audit event** — Who (authenticated principal, not just session ID), what (specific action and target), when (UTC, monotonic-clock-corroborated), where (source IP, request ID), why (correlated to the upstream business event).

[*detective*] **Periodic log integrity verification** — Schedule daily integrity checks on the audit log chain. Alert on any gap.

[A09:2021 — Security Logging and Monitoring Failures](#) | [STRIDE reference](#) | [CWE-778 — Insufficient Logging](#)

Sensitive data exposure in transit

STRIDE → Information Disclosure | Component: **Lambda Functions** (api) | Source: rule-based

CWE: CWE-319 — Cleartext Transmission of Sensitive Information

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Lambda Functions** (`api`).

■ **Description:** PII, secrets, or tokens transmitted over unencrypted channels.

■ ■ Attack scenario:

1. Attacker reaches Lambda Functions via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:6 E:7 A:7 D:8 → **Total 35/50**

■ How to mitigate:

[*preventive*] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[*preventive*] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[*preventive*] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-319 — Cleartext Transmission of Sensitive Information](#)

Sensitive data exposure at rest

STRIDE → Information Disclosure | Component: **Lambda Functions** (api) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Lambda Functions** (``api``).

■ **Description:** Stored PII / secrets accessible without proper authorization.

■ ■ **Attack scenario:**

1. Attacker reaches Lambda Functions via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:6 E:7 A:7 D:8 → **Total 35/50**

■ **How to mitigate:**

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Insecure direct object reference (IDOR)

STRIDE → Information Disclosure | Component: **Lambda Functions** (api) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Lambda Functions** (``api``).

■ **Description:** User can access another user's resources by guessing IDs.

■ ■ **Attack scenario:**

1. Attacker reaches Lambda Functions via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:8 E:7 A:7 D:8 → **Total 37/50**

■ **How to mitigate:**

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Application-layer DDoS

STRIDE → Denial of Service | Component: **Lambda Functions** (api) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Lambda Functions** (`api`).

■ **Description:** Attacker floods expensive endpoints (e.g., search, login).

■ ■ **Attack scenario:**

1. Attacker identifies a costly operation in Lambda Functions (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Lambda Functions's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Lambda Functions; cascading failure may take down dependents.

DREAD — D:7 R:6 E:7 A:7 D:7 → **Total 34/50**

■ **How to mitigate:**

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Privilege escalation via mass assignment

STRIDE → Elevation of Privilege | Component: **Lambda Functions** (api) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **Lambda Functions** (`api`).

■ **Description:** User submits extra fields (e.g., role=admin) and they bind to the model.

■ ■ **Attack scenario:**

1. Attacker authenticates to Lambda Functions as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:7 R:8 E:7 A:7 D:7 → **Total 36/50**

■ **How to mitigate:**

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to Lambda Functions should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Data-in-transit modification

STRIDE → Tampering | Component: **Long Term Memory** (datastore) | Source: rule-based

CWE: CWE-345 — Insufficient Verification of Data Authenticity

CVSS 3.1: 7.3 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Long Term Memory** (`datastore`).

■ **Description:** Attacker on the network path alters request/response payloads.

■ Attack scenario:

1. Attacker reaches an input vector exposed by Long Term Memory (request body, query string, message queue payload, persisted state).
2. Submits malformed input designed to alter the program's logic or stored data (injection, parameter tampering, deserialization gadgets).
3. Long Term Memory processes the input without strict validation, and the malicious change takes effect.
4. Tampered data is persisted, executed, or relayed downstream — corrupting integrity beyond the original entry point.

DREAD — D:8 R:6 E:7 A:5 D:7 → **Total 33/50**

■ How to mitigate:

[preventive] **Schema-validate and parameterize all inputs** — JSON Schema / OpenAPI validation at API edges; parameterized queries / prepared statements for SQL; ORM-level validators on persisted models.

[preventive] **Sign and verify integrity-critical messages** — For commands or state mutations, attach an HMAC or signature so downstream consumers can detect tampering in transit or at rest.

[detective] **Use append-only / write-once stores for audit-critical data** — Where data must not be silently changed, write to an append-only log (e.g., immutable S3 bucket, CloudTrail-like store) and reconcile against the mutable copy.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-345 — Insufficient Verification of Data Authenticity](#)

Config / secret tampering

STRIDE → Tampering | Component: **Long Term Memory** (datastore) | Source: rule-based

CWE: CWE-345 — Insufficient Verification of Data Authenticity

CVSS 3.1: 7.3 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Long Term Memory** (`datastore`).

■ **Description:** Unauthorized changes to runtime config alter security behavior.

■ Attack scenario:

1. Attacker reaches an input vector exposed by Long Term Memory (request body, query string, message queue payload, persisted state).
2. Submits malformed input designed to alter the program's logic or stored data (injection, parameter tampering, deserialization gadgets).
3. Long Term Memory processes the input without strict validation, and the malicious change takes effect.
4. Tampered data is persisted, executed, or relayed downstream — corrupting integrity beyond the original entry point.

DREAD — D:8 R:6 E:7 A:5 D:7 → **Total 33/50**

■ How to mitigate:

[preventive] **Schema-validate and parameterize all inputs** — JSON Schema / OpenAPI validation at API edges; parameterized queries / prepared statements for SQL; ORM-level validators on persisted models.

[preventive] **Sign and verify integrity-critical messages** — For commands or state mutations, attach an HMAC or signature so downstream consumers can detect tampering in transit or at rest.

[detective] **Use append-only / write-once stores for audit-critical data** — Where data must not be silently changed, write to an append-only log (e.g., immutable S3 bucket, CloudTrail-like store) and reconcile against the mutable copy.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-345 — Insufficient Verification of Data Authenticity](#)

Sensitive data exposure in transit

STRIDE → Information Disclosure | Component: **Long Term Memory** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Long Term Memory** (`datastore`).

■ **Description:** PII, secrets, or tokens transmitted over unencrypted channels.

■ ■ ■ Attack scenario:

1. Attacker reaches Long Term Memory via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:8 R:6 E:7 A:5 D:8 → **Total 34/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Sensitive data exposure at rest

STRIDE → Information Disclosure | Component: **Long Term Memory** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Long Term Memory** (`datastore`).

■ **Description:** Stored PII / secrets accessible without proper authorization.

■ ■ ■ Attack scenario:

1. Attacker reaches Long Term Memory via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:8 R:6 E:7 A:5 D:8 → **Total 34/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Insecure direct object reference (IDOR)

STRIDE → Information Disclosure | Component: **Long Term Memory** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Long Term Memory** (`datastore`).

■ **Description:** User can access another user's resources by guessing IDs.

■ ■ ■ Attack scenario:

1. Attacker reaches Long Term Memory via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs,

debug data, full PII fields).

3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:8 R:8 E:7 A:5 D:8 → **Total 36/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Trust-boundary crossing without strong authn: Agent Runtime / Orchestrator → Long Term Memory [cross-boundary]

STRIDE → Spoofing | Component: **Long Term Memory** (datastore) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 6.8 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Agent Runtime / Orchestrator → Long Term Memory** (label: *Agent Runtime / Orchestrator → Long Term Memory*, protocol: TCP, auth: credentials, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Long Term Memory** (`datastore`).

■ **Description:** Flow 'Agent Runtime / Orchestrator → Long Term Memory' crosses trust boundary 'Application tier' → 'Data tier'. Caller identity must be re-verified at the boundary; existing trust does not transit.

■■ Attack scenario:

1. Attacker positions themselves on the network path between source and destination, or compromises an upstream component in the source zone.
2. Crafts requests with forged or replayed identity claims (cookies, tokens, IP-based trust).
3. Long Term Memory accepts the request because identity is not re-verified at the boundary, treating the upstream zone as trusted.
4. Attacker can now perform actions as the impersonated principal across the boundary.

DREAD — D:8 R:6 E:7 A:7 D:7 → **Total 35/50**

■ How to mitigate:

[preventive] **Re-authenticate at the boundary** — Require a fresh, audience-bound credential at the receiver. Do not infer identity from network position, source IP, or upstream session.

[preventive] **Use mutual TLS or signed tokens with audience claims** — For service-to-service calls into Long Term Memory, terminate mTLS at the boundary and reject any caller without a valid client cert. For human-driven flows, use OAuth2 / OIDC with audience and issuer validation.

[detective] **Log every cross-boundary auth decision** — Emit an authentication event (success and failure) tagged with both source and destination zone. Forward to SIEM with retention ≥ 90 days.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Cross-boundary input not validated: Agent Runtime / Orchestrator → Long Term Memory [cross-boundary]

STRIDE → Tampering | Component: **Long Term Memory** (datastore) | Source: rule-based

CWE: CWE-20 — Improper Input Validation

CVSS 3.1: 8.7 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:L/SI:H/SA:N

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Agent Runtime / Orchestrator** → **Long Term Memory** (label: *Agent Runtime / Orchestrator → Long Term Memory*, protocol: TCP, auth: credentials, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Long Term Memory** (`datastore`).

■ **Description:** Data crossing the trust boundary into 'Data tier' must be treated as untrusted, even if the source is internal. Implicit trust is the most common cause of injection / SSRF / deserialization bugs.

■ ■ **Attack scenario:**

1. Attacker compromises a less-trusted upstream zone or directly sends malformed input to Long Term Memory.
2. Sends payloads with unexpected types, lengths, or special characters (e.g. SQL meta-chars, NUL bytes, control sequences).
3. Long Term Memory processes the input under the assumption that the upstream zone has already validated it — but the boundary changes the security context.
4. Injection, deserialization flaws, or buffer issues trigger; attacker pivots into Long Term Memory or the zone behind it.

DREAD — D:8 R:6 E:7 A:7 D:7 → **Total 35/50**

■ **How to mitigate:**

[preventive] **Validate against an allow-list schema at the boundary** — Define an explicit schema (JSON Schema, Protobuf, OpenAPI) for every accepted message at this boundary. Reject anything that doesn't match — type, length, charset, enum values, nested depth.

[preventive] **Canonicalize before validation** — Decode URL-encoding, Unicode normalization (NFC), trim whitespace, and resolve relative paths before validating. Avoid double-decoding bypasses.

[preventive] **Apply context-specific output encoding** — Whatever Long Term Memory does with the input — SQL: parameterized queries, HTML: contextual escaping, OS commands: avoid shell, use exec arrays, LDAP: escape filter chars.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-20 — Improper Input Validation](#)

Cross-boundary data exposure risk: Agent Runtime / Orchestrator → Long Term Memory [cross-boundary]

STRIDE → Information Disclosure | Component: **Long Term Memory** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 6.8 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Agent Runtime / Orchestrator** → **Long Term Memory** (label: *Agent Runtime / Orchestrator → Long Term Memory*, protocol: TCP, auth: credentials, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Long Term Memory** (`datastore`).

■ **Description:** Information leaving 'Application tier' into 'Data tier' may include data the receiving zone is not authorized to see. Cross-boundary egress is a common data-leak surface.

■ ■ **Attack scenario:**

1. Attacker observes traffic leaving the source zone, or compromises a component in the destination zone.
2. The destination zone receives more data than it strictly needs (over-fetching, verbose error responses, full record dumps).
3. Attacker harvests sensitive fields (PII, secrets, internal IDs) that should never have left the source zone.
4. Data is exfiltrated, sold, or used to plan a deeper attack on the originating zone.

DREAD — D:8 R:6 E:7 A:7 D:8 → **Total 36/50**

■ **How to mitigate:**

[preventive] **Enforce minimum-data-needed at the egress point** — Whitelist exactly which fields leave the source zone. Strip everything else server-side before serialization.

[preventive] **Tokenize or redact sensitive fields** — Replace PII / PCI / PHI fields with reversible tokens or one-way hashes when the destination zone doesn't need the cleartext value.

[detective] **Log cross-boundary data flows for review** — Sample-log the field set crossing this boundary (not the values) so DLP and privacy reviews can audit what's egressing.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Authentication bypass via credential stuffing

STRIDE → Spoofing | Component: **MCP Server Hosting** (api) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **MCP Server Hosting** (`api`).

■ **Description:** Attackers reuse leaked credentials to impersonate legitimate users.

■ ■ **Attack scenario:**

1. Attacker locates MCP Server Hosting's identity claim mechanism (token format, header name, signing key handling).
2. Forges or replays a token to assert another principal's identity.
3. MCP Server Hosting accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
4. Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:7 R:6 E:7 A:8 D:7 → **Total 35/50**

■ **How to mitigate:**

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Session token hijacking

STRIDE → Spoofing | Component: **MCP Server Hosting** (api) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **MCP Server Hosting** (`api`).

■ **Description:** Attacker steals or predicts a session token to impersonate a user.

■ ■ **Attack scenario:**

1. Attacker locates MCP Server Hosting's identity claim mechanism (token format, header name, signing key handling).
2. Forges or replays a token to assert another principal's identity.
3. MCP Server Hosting accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
4. Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:7 R:6 E:7 A:8 D:7 → **Total 35/50**

■ **How to mitigate:**

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

API key / service identity spoofing

STRIDE → Spoofing | Component: **MCP Server Hosting** (api) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **MCP Server Hosting** (`api`).

■ **Description:** An attacker obtains or forges an API key to act as a trusted service.

■ ■ **Attack scenario:**

1. Attacker locates MCP Server Hosting's identity claim mechanism (token format, header name, signing key handling).
2. Forges or replays a token to assert another principal's identity.
3. MCP Server Hosting accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
4. Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:7 R:6 E:7 A:8 D:7 → **Total 35/50**

■ **How to mitigate:**

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Log tampering

STRIDE → Repudiation | Component: **MCP Server Hosting** (api) | Source: rule-based

CWE: CWE-778 — Insufficient Logging

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **MCP Server Hosting** (`api`).

■ **Description:** An attacker with access modifies or deletes audit trails.

■ ■ **Attack scenario:**

1. User performs a sensitive action through MCP Server Hosting (a transfer, a permission change, a deletion).
2. Audit logs are absent, incomplete, or modifiable by the same principal who performed the action.
3. User later denies having performed the action, or an attacker covers their tracks.
4. Without tamper-evident logs, neither responsible party can be identified — leading to fraud, dispute, or regulatory exposure.

DREAD — D:7 R:6 E:7 A:8 D:7 → **Total 35/50**

■ **How to mitigate:**

[detective] **Emit tamper-evident audit logs** — Sign log entries (HMAC chain, hash-linked) so insertion or deletion is detectable. Forward to a separate, append-only system the action's principal cannot administer.

[detective] **Capture sufficient detail per audit event** — Who (authenticated principal, not just session ID), what (specific action and target), when (UTC, monotonic-clock-corroborated), where (source IP, request ID), why (correlated to the upstream business event).

[detective] **Periodic log integrity verification** — Schedule daily integrity checks on the audit log chain. Alert on any gap.

[A09:2021 — Security Logging and Monitoring Failures](#) | [STRIDE reference](#) | [CWE-778 — Insufficient Logging](#)

Sensitive data exposure in transit

STRIDE → Information Disclosure | Component: **MCP Server Hosting** (api) | Source: rule-based

CWE: CWE-319 — Cleartext Transmission of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **MCP Server Hosting** (`api`).

■ **Description:** PII, secrets, or tokens transmitted over unencrypted channels.

■ ■ **Attack scenario:**

1. Attacker reaches MCP Server Hosting via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.

4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:6 E:7 A:8 D:8 → **Total 36/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-319 — Cleartext Transmission of Sensitive Information](#)

Sensitive data exposure at rest

STRIDE → Information Disclosure | Component: **MCP Server Hosting** (api) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **MCP Server Hosting** (`api`).

■ **Description:** Stored PII / secrets accessible without proper authorization.

■■ Attack scenario:

1. Attacker reaches MCP Server Hosting via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:6 E:7 A:8 D:8 → **Total 36/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Insecure direct object reference (IDOR)

STRIDE → Information Disclosure | Component: **MCP Server Hosting** (api) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **MCP Server Hosting** (`api`).

■ **Description:** User can access another user's resources by guessing IDs.

■■ Attack scenario:

1. Attacker reaches MCP Server Hosting via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:8 E:7 A:8 D:8 → **Total 38/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Application-layer DDoS

STRIDE → Denial of Service | Component: **MCP Server Hosting** (api) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **MCP Server Hosting** (`api`).

■ **Description:** Attacker floods expensive endpoints (e.g., search, login).

■■ Attack scenario:

1. Attacker identifies a costly operation in MCP Server Hosting (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. MCP Server Hosting's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach MCP Server Hosting; cascading failure may take down dependents.

DREAD — D:7 R:6 E:7 A:8 D:7 → **Total 35/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Privilege escalation via mass assignment

STRIDE → Elevation of Privilege | Component: **MCP Server Hosting** (api) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **MCP Server Hosting** (`api`).

■ **Description:** User submits extra fields (e.g., role=admin) and they bind to the model.

■■ Attack scenario:

1. Attacker authenticates to MCP Server Hosting as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:7 R:8 E:7 A:8 D:7 → **Total 37/50**

■ How to mitigate:

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to MCP Server Hosting should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Authentication bypass via credential stuffing

STRIDE → Spoofing | Component: **MCP Server Targets** (external_entity) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **MCP Server Targets** (external_entity).

■ **Description:** Attackers reuse leaked credentials to impersonate legitimate users.

■ ■ Attack scenario:

1. Attacker obtains valid credentials via phishing, credential stuffing, or a third-party breach.
2. Logs in as the legitimate user/entity at MCP Server Targets.
3. Performs actions indistinguishable from the real user (auth events look normal).
4. Damage scales with the impersonated user's permissions; admin accounts cause the most harm.

DREAD — D:7 R:6 E:9 A:5 D:9 → **Total 36/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Session token hijacking

STRIDE → Spoofing | Component: **MCP Server Targets** (external_entity) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **MCP Server Targets** (external_entity).

■ **Description:** Attacker steals or predicts a session token to impersonate a user.

■ ■ Attack scenario:

1. Attacker obtains valid credentials via phishing, credential stuffing, or a third-party breach.
2. Logs in as the legitimate user/entity at MCP Server Targets.
3. Performs actions indistinguishable from the real user (auth events look normal).
4. Damage scales with the impersonated user's permissions; admin accounts cause the most harm.

DREAD — D:7 R:6 E:9 A:5 D:9 → **Total 36/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

API key / service identity spoofing

STRIDE → Spoofing | Component: **MCP Server Targets** (external_entity) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **MCP Server Targets** (external_entity).

■ **Description:** An attacker obtains or forges an API key to act as a trusted service.

■ ■ Attack scenario:

1. Attacker obtains valid credentials via phishing, credential stuffing, or a third-party breach.
2. Logs in as the legitimate user/entity at MCP Server Targets.
3. Performs actions indistinguishable from the real user (auth events look normal).
4. Damage scales with the impersonated user's permissions; admin accounts cause the most harm.

DREAD — D:7 R:6 E:9 A:5 D:9 → **Total 36/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Sensitive data exposure in transit

STRIDE → Information Disclosure | Component: **MCP Server Targets** (external_entity) | Source: rule-based

CWE: CWE-319 — Cleartext Transmission of Sensitive Information

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **MCP Server Targets** (external_entity).

■ **Description:** PII, secrets, or tokens transmitted over unencrypted channels.

■ ■ Attack scenario:

1. Attacker reaches MCP Server Targets via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:6 E:9 A:5 D:10 → **Total 37/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For external_entity, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-319 — Cleartext Transmission of Sensitive Information](#)

Sensitive data exposure at rest

STRIDE → Information Disclosure | Component: **MCP Server Targets** (external_entity) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **MCP Server Targets** (`external\_entity`).

■ **Description:** Stored PII / secrets accessible without proper authorization.

■ ■ **Attack scenario:**

1. Attacker reaches MCP Server Targets via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:6 E:9 A:5 D:10 → **Total 37/50**

■ **How to mitigate:**

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For external_entity, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Insecure direct object reference (IDOR)

[STRIDE](#) → Information Disclosure | Component: **MCP Server Targets** (external_entity) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **MCP Server Targets** (`external\_entity`).

■ **Description:** User can access another user's resources by guessing IDs.

■ ■ **Attack scenario:**

1. Attacker reaches MCP Server Targets via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:8 E:9 A:5 D:10 → **Total 39/50**

■ **How to mitigate:**

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For external_entity, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Application-layer DDoS

[STRIDE](#) → Denial of Service | Component: **MCP Server Targets** (external_entity) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **MCP Server Targets** (`external\_entity`).

■ **Description:** Attacker floods expensive endpoints (e.g., search, login).

■ ■ **Attack scenario:**

1. Attacker identifies a costly operation in MCP Server Targets (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. MCP Server Targets's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach MCP Server Targets; cascading failure may take down dependents.

DREAD — D:7 R:6 E:9 A:5 D:9 → **Total 36/50**

■ **How to mitigate:**

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Trust-boundary crossing without strong authn: MCP Server Hosting → MCP Server Targets [cross-boundary]

STRIDE → Spoofing | Component: **MCP Server Targets** (external_entity) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Application tier → Internet

■ **Location:** On the data flow **MCP Server Hosting** → **MCP Server Targets** (label: *MCP Server Hosting → MCP Server Targets*, protocol: HTTPS, auth: mTLS, encrypted: yes), crossing the trust boundary from `Application tier` into `Internet`. The receiving component is **MCP Server Targets** (`external\_entity`).

■ **Description:** Flow 'MCP Server Hosting → MCP Server Targets' crosses trust boundary 'Application tier' → 'Internet'. Caller identity must be re-verified at the boundary; existing trust does not transit.

■ ■ **Attack scenario:**

1. Attacker positions themselves on the network path between source and destination, or compromises an upstream component in the source zone.
2. Crafts requests with forged or replayed identity claims (cookies, tokens, IP-based trust).
3. MCP Server Targets accepts the request because identity is not re-verified at the boundary, treating the upstream zone as trusted.
4. Attacker can now perform actions as the impersonated principal across the boundary.

DREAD — D:7 R:6 E:7 A:7 D:9 → **Total 36/50**

■ **How to mitigate:**

[preventive] **Re-authenticate at the boundary** — Require a fresh, audience-bound credential at the receiver. Do not infer identity from network position, source IP, or upstream session.

[preventive] **Use mutual TLS or signed tokens with audience claims** — For service-to-service calls into MCP Server Targets, terminate mTLS at the boundary and reject any caller without a valid client cert. For human-driven flows, use OAuth2 / OIDC with audience and issuer validation.

[detective] **Log every cross-boundary auth decision** — Emit an authentication event (success and failure) tagged with both source and destination zone. Forward to SIEM with retention ≥ 90 days.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Cross-boundary input not validated: MCP Server Hosting → MCP Server Targets [cross-boundary]

STRIDE → Tampering | Component: **MCP Server Targets** (external_entity) | Source: rule-based

CWE: CWE-20 — Improper Input Validation

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:N/I:H/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:N/VI:H/VA:N/SC:L/SI:H/SA:N

Boundary crossing: Application tier → Internet

■ **Location:** On the data flow **MCP Server Hosting** → **MCP Server Targets** (label: *MCP Server Hosting → MCP Server Targets*, protocol: HTTPS, auth: mtls, encrypted: yes), crossing the trust boundary from `Application tier` into `Internet`. The receiving component is **MCP Server Targets** (`external\_entity`).

■ **Description:** Data crossing the trust boundary into 'Internet' must be treated as untrusted, even if the source is internal. Implicit trust is the most common cause of injection / SSRF / deserialization bugs.

■ ■ **Attack scenario:**

1. Attacker compromises a less-trusted upstream zone or directly sends malformed input to MCP Server Targets.
2. Sends payloads with unexpected types, lengths, or special characters (e.g. SQL meta-chars, NUL bytes, control sequences).
3. MCP Server Targets processes the input under the assumption that the upstream zone has already validated it — but the boundary changes the security context.
4. Injection, deserialization flaws, or buffer issues trigger; attacker pivots into MCP Server Targets or the zone behind it.

DREAD — D:7 R:6 E:7 A:7 D:9 → **Total 36/50**

■ **How to mitigate:**

[preventive] **Validate against an allow-list schema at the boundary** — Define an explicit schema (JSON Schema, Protobuf, OpenAPI) for every accepted message at this boundary. Reject anything that doesn't match — type, length, charset, enum values, nested depth.

[preventive] **Canonicalize before validation** — Decode URL-encoding, Unicode normalization (NFC), trim whitespace, and resolve relative paths before validating. Avoid double-decoding bypasses.

[preventive] **Apply context-specific output encoding** — Whatever MCP Server Targets does with the input — SQL: parameterized queries, HTML: contextual escaping, OS commands: avoid shell, use exec arrays, LDAP: escape filter chars.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-20 — Improper Input Validation](#)

Cross-boundary data exposure risk: MCP Server Hosting → MCP Server Targets [cross-boundary]

STRIDE → Information Disclosure | Component: **MCP Server Targets** (external_entity) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Application tier → Internet

■ **Location:** On the data flow **MCP Server Hosting** → **MCP Server Targets** (label: *MCP Server Hosting → MCP Server Targets*, protocol: HTTPS, auth: mtls, encrypted: yes), crossing the trust boundary from `Application tier` into `Internet`. The receiving component is **MCP Server Targets** (`external\_entity`).

■ **Description:** Information leaving 'Application tier' into 'Internet' may include data the receiving zone is not authorized to see. Cross-boundary egress is a common data-leak surface.

■ ■ **Attack scenario:**

1. Attacker observes traffic leaving the source zone, or compromises a component in the destination zone.
2. The destination zone receives more data than it strictly needs (over-fetching, verbose error responses, full record dumps).
3. Attacker harvests sensitive fields (PII, secrets, internal IDs) that should never have left the source zone.
4. Data is exfiltrated, sold, or used to plan a deeper attack on the originating zone.

DREAD — D:7 R:6 E:7 A:7 D:10 → **Total 37/50**

■ **How to mitigate:**

[preventive] **Enforce minimum-data-needed at the egress point** — Whitelist exactly which fields leave the source zone. Strip everything else server-side before serialization.

[preventive] **Tokenize or redact sensitive fields** — Replace PII / PCI / PHI fields with reversible tokens or one-way hashes when the destination zone doesn't need the cleartext value.

[detective] **Log cross-boundary data flows for review** — Sample-log the field set crossing this boundary (not the values) so DLP and privacy reviews can audit what's egressing.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Data-in-transit modification

STRIDE → Tampering | Component: **Metrics & Tracing** (datastore) | Source: rule-based

CWE: CWE-345 — Insufficient Verification of Data Authenticity

CVSS 3.1: 7.3 (High) [CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N](#)

CVSS 4.0: 5.2 (Medium) [CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/NVC:L/VI:H/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Metrics & Tracing** (`datastore`).

■ **Description:** Attacker on the network path alters request/response payloads.

■ ■ **Attack scenario:**

1. Attacker reaches an input vector exposed by Metrics & Tracing (request body, query string, message queue payload, persisted state).
2. Submits malformed input designed to alter the program's logic or stored data (injection, parameter tampering, deserialization gadgets).
3. Metrics & Tracing processes the input without strict validation, and the malicious change takes effect.
4. Tampered data is persisted, executed, or relayed downstream — corrupting integrity beyond the original entry point.

DREAD — D:8 R:6 E:7 A:5 D:7 → **Total 33/50**

■ **How to mitigate:**

[preventive] **Schema-validate and parameterize all inputs** — JSON Schema / OpenAPI validation at API edges; parameterized queries / prepared statements for SQL; ORM-level validators on persisted models.

[preventive] **Sign and verify integrity-critical messages** — For commands or state mutations, attach an HMAC or signature so downstream consumers can detect tampering in transit or at rest.

[detective] **Use append-only / write-once stores for audit-critical data** — Where data must not be silently changed, write to an append-only log (e.g., immutable S3 bucket, CloudTrail-like store) and reconcile against the mutable copy.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-345 — Insufficient Verification of Data Authenticity](#)

Config / secret tampering

STRIDE → Tampering | Component: **Metrics & Tracing** (datastore) | Source: rule-based

CWE: CWE-345 — Insufficient Verification of Data Authenticity

CVSS 3.1: 7.3 (High) [CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N](#)

CVSS 4.0: 5.2 (Medium) [CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/NVC:L/VI:H/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Metrics & Tracing** (`datastore`).

■ **Description:** Unauthorized changes to runtime config alter security behavior.

■ ■ **Attack scenario:**

1. Attacker reaches an input vector exposed by Metrics & Tracing (request body, query string, message queue payload, persisted state).
2. Submits malformed input designed to alter the program's logic or stored data (injection, parameter tampering, deserialization gadgets).
3. Metrics & Tracing processes the input without strict validation, and the malicious change takes effect.
4. Tampered data is persisted, executed, or relayed downstream — corrupting integrity beyond the original entry point.

DREAD — D:8 R:6 E:7 A:5 D:7 → **Total 33/50**

■ **How to mitigate:**

[preventive] **Schema-validate and parameterize all inputs** — JSON Schema / OpenAPI validation at API edges; parameterized queries / prepared statements for SQL; ORM-level validators on persisted models.

[preventive] **Sign and verify integrity-critical messages** — For commands or state mutations, attach an HMAC or signature so downstream consumers can detect tampering in transit or at rest.

[detective] **Use append-only / write-once stores for audit-critical data** — Where data must not be silently changed, write to an append-only log (e.g., immutable S3 bucket, CloudTrail-like store) and reconcile against the mutable copy.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-345 — Insufficient Verification of Data Authenticity](#)

Sensitive data exposure in transit

STRIDE → Information Disclosure | Component: **Metrics & Tracing** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) [CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 5.2 (Medium) [CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Metrics & Tracing** (`datastore`).

■ **Description:** PII, secrets, or tokens transmitted over unencrypted channels.

■■ **Attack scenario:**

1. Attacker reaches Metrics & Tracing via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:8 R:6 E:7 A:5 D:8 → **Total 34/50**

■ **How to mitigate:**

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Sensitive data exposure at rest

STRIDE → Information Disclosure | Component: **Metrics & Tracing** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) [CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 5.2 (Medium) [CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Metrics & Tracing** (`datastore`).

■ **Description:** Stored PII / secrets accessible without proper authorization.

■■ **Attack scenario:**

1. Attacker reaches Metrics & Tracing via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:8 R:6 E:7 A:5 D:8 → **Total 34/50**

■ **How to mitigate:**

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Insecure direct object reference (IDOR)

STRIDE → Information Disclosure | Component: **Metrics & Tracing** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) [CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 5.2 (Medium) [CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Metrics & Tracing** (`datastore`).

■ **Description:** User can access another user's resources by guessing IDs.

■ ■ **Attack scenario:**

1. Attacker reaches Metrics & Tracing via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:8 R:8 E:7 A:5 D:8 → **Total 36/50**

■ **How to mitigate:**

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Trust-boundary crossing without strong authn: Core Agent Layer → Metrics & Tracing [cross-boundary]

STRIDE → Spoofing | Component: **Metrics & Tracing** (datastore) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 6.8 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Core Agent Layer → Metrics & Tracing** (label: *Core Agent Layer → Metrics & Tracing*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Metrics & Tracing** (`datastore`).

■ **Description:** Flow 'Core Agent Layer → Metrics & Tracing' crosses trust boundary 'Application tier' → 'Data tier'. Caller identity must be re-verified at the boundary; existing trust does not transit.

■ ■ **Attack scenario:**

1. Attacker positions themselves on the network path between source and destination, or compromises an upstream component in the source zone.
2. Crafts requests with forged or replayed identity claims (cookies, tokens, IP-based trust).
3. Metrics & Tracing accepts the request because identity is not re-verified at the boundary, treating the upstream zone as trusted.
4. Attacker can now perform actions as the impersonated principal across the boundary.

DREAD — D:8 R:6 E:7 A:7 D:7 → **Total 35/50**

■ **How to mitigate:**

[preventive] **Re-authenticate at the boundary** — Require a fresh, audience-bound credential at the receiver. Do not infer identity from network position, source IP, or upstream session.

[preventive] **Use mutual TLS or signed tokens with audience claims** — For service-to-service calls into Metrics & Tracing, terminate mTLS at the boundary and reject any caller without a valid client cert. For human-driven flows, use OAuth2 / OIDC with audience and issuer validation.

[detective] **Log every cross-boundary auth decision** — Emit an authentication event (success and failure) tagged with both source and destination zone. Forward to SIEM with retention ≥ 90 days.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Cross-boundary input not validated: Core Agent Layer → Metrics & Tracing [cross-boundary]

STRIDE → Tampering | Component: **Metrics & Tracing** (datastore) | Source: rule-based

CWE: CWE-20 — Improper Input Validation

CVSS 3.1: 8.7 (High) [CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:N](#)

CVSS 4.0: 5.8 (Medium) [CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:L/SI:H/SA:N](#)

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Core Agent Layer** → **Metrics & Tracing** (label: *Core Agent Layer → Metrics & Tracing*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Metrics & Tracing** (`datastore`).

■ **Description:** Data crossing the trust boundary into 'Data tier' must be treated as untrusted, even if the source is internal. Implicit trust is the most common cause of injection / SSRF / deserialization bugs.

■ ■ **Attack scenario:**

1. Attacker compromises a less-trusted upstream zone or directly sends malformed input to Metrics & Tracing.
2. Sends payloads with unexpected types, lengths, or special characters (e.g. SQL meta-chars, NUL bytes, control sequences).
3. Metrics & Tracing processes the input under the assumption that the upstream zone has already validated it — but the boundary changes the security context.
4. Injection, deserialization flaws, or buffer issues trigger; attacker pivots into Metrics & Tracing or the zone behind it.

DREAD — D:8 R:6 E:7 A:7 D:7 → **Total 35/50**

■ **How to mitigate:**

[preventive] **Validate against an allow-list schema at the boundary** — Define an explicit schema (JSON Schema, Protobuf, OpenAPI) for every accepted message at this boundary. Reject anything that doesn't match — type, length, charset, enum values, nested depth.

[preventive] **Canonicalize before validation** — Decode URL-encoding, Unicode normalization (NFC), trim whitespace, and resolve relative paths before validating. Avoid double-decoding bypasses.

[preventive] **Apply context-specific output encoding** — Whatever Metrics & Tracing does with the input — SQL: parameterized queries, HTML: contextual escaping, OS commands: avoid shell, use exec arrays, LDAP: escape filter chars.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-20 — Improper Input Validation](#)

Cross-boundary data exposure risk: Core Agent Layer → Metrics & Tracing [cross-boundary]

STRIDE → Information Disclosure | Component: **Metrics & Tracing** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 6.8 (Medium) [CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N](#)

CVSS 4.0: 5.8 (Medium) [CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N](#)

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Core Agent Layer** → **Metrics & Tracing** (label: *Core Agent Layer → Metrics & Tracing*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Metrics & Tracing** (`datastore`).

■ **Description:** Information leaving 'Application tier' into 'Data tier' may include data the receiving zone is not authorized to see. Cross-boundary egress is a common data-leak surface.

■ ■ **Attack scenario:**

1. Attacker observes traffic leaving the source zone, or compromises a component in the destination zone.
2. The destination zone receives more data than it strictly needs (over-fetching, verbose error responses, full record dumps).
3. Attacker harvests sensitive fields (PII, secrets, internal IDs) that should never have left the source zone.
4. Data is exfiltrated, sold, or used to plan a deeper attack on the originating zone.

DREAD — D:8 R:6 E:7 A:7 D:8 → **Total 36/50**

■ **How to mitigate:**

[preventive] **Enforce minimum-data-needed at the egress point** — Whitelist exactly which fields leave the source zone. Strip everything else server-side before serialization.

[preventive] **Tokenize or redact sensitive fields** — Replace PII / PCI / PHI fields with reversible tokens or one-way hashes when the destination zone doesn't need the cleartext value.

[detective] **Log cross-boundary data flows for review** — Sample-log the field set crossing this boundary (not the values) so DLP and privacy reviews can audit what's egressing.

Authentication bypass via credential stuffing

STRIDE → Spoofing | Component: **Model Gateway** (api) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Model Gateway** (`api`).

■ **Description:** Attackers reuse leaked credentials to impersonate legitimate users.

■ ■ ■ Attack scenario:

1. Attacker locates Model Gateway's identity claim mechanism (token format, header name, signing key handling).
2. Forges or replays a token to assert another principal's identity.
3. Model Gateway accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
4. Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:7 R:6 E:9 A:8 D:9 → **Total 39/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Session token hijacking

STRIDE → Spoofing | Component: **Model Gateway** (api) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Model Gateway** (`api`).

■ **Description:** Attacker steals or predicts a session token to impersonate a user.

■ ■ ■ Attack scenario:

1. Attacker locates Model Gateway's identity claim mechanism (token format, header name, signing key handling).
2. Forges or replays a token to assert another principal's identity.
3. Model Gateway accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
4. Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:7 R:6 E:9 A:8 D:9 → **Total 39/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

API key / service identity spoofing

STRIDE → Spoofing | Component: **Model Gateway** (api) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Model Gateway** (`api`).

■ **Description:** An attacker obtains or forges an API key to act as a trusted service.

■■ **Attack scenario:**

1. Attacker locates Model Gateway's identity claim mechanism (token format, header name, signing key handling).
2. Forges or replays a token to assert another principal's identity.
3. Model Gateway accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
4. Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:7 R:6 E:9 A:8 D:9 → **Total 39/50**

■ **How to mitigate:**

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Log tampering

STRIDE → Repudiation | Component: **Model Gateway** (api) | Source: rule-based

CWE: CWE-778 — Insufficient Logging

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:H/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Model Gateway** (`api`).

■ **Description:** An attacker with access modifies or deletes audit trails.

■■ **Attack scenario:**

1. User performs a sensitive action through Model Gateway (a transfer, a permission change, a deletion).
2. Audit logs are absent, incomplete, or modifiable by the same principal who performed the action.
3. User later denies having performed the action, or an attacker covers their tracks.
4. Without tamper-evident logs, neither responsible party can be identified — leading to fraud, dispute, or regulatory exposure.

DREAD — D:7 R:6 E:9 A:8 D:9 → **Total 39/50**

■ **How to mitigate:**

[detective] **Emit tamper-evident audit logs** — Sign log entries (HMAC chain, hash-linked) so insertion or deletion is detectable. Forward to a separate, append-only system the action's principal cannot administer.

[detective] **Capture sufficient detail per audit event** — Who (authenticated principal, not just session ID), what (specific action and target), when (UTC, monotonic-clock-corroborated), where (source IP, request ID), why (correlated to the upstream business event).

[detective] **Periodic log integrity verification** — Schedule daily integrity checks on the audit log chain. Alert on any gap.

[A09:2021 — Security Logging and Monitoring Failures](#) | [STRIDE reference](#) | [CWE-778 — Insufficient Logging](#)

Sensitive data exposure in transit

STRIDE → Information Disclosure | Component: **Model Gateway** (api) | Source: rule-based

CWE: CWE-319 — Cleartext Transmission of Sensitive Information

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Model Gateway** (`api`).

■ **Description:** PII, secrets, or tokens transmitted over unencrypted channels.

■ ■ Attack scenario:

1. Attacker reaches Model Gateway via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:6 E:9 A:8 D:10 → **Total 40/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-319 — Cleartext Transmission of Sensitive Information](#)

Sensitive data exposure at rest

STRIDE → Information Disclosure | Component: **Model Gateway** (api) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Model Gateway** (`api`).

■ **Description:** Stored PII / secrets accessible without proper authorization.

■ ■ Attack scenario:

1. Attacker reaches Model Gateway via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:6 E:9 A:8 D:10 → **Total 40/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Insecure direct object reference (IDOR)

STRIDE → Information Disclosure | Component: **Model Gateway** (api) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Model Gateway** (`api`).

■ **Description:** User can access another user's resources by guessing IDs.

■ ■ Attack scenario:

1. Attacker reaches Model Gateway via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs,

debug data, full PII fields).

3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:8 E:9 A:8 D:10 → **Total 42/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Application-layer DDoS

STRIDE → Denial of Service | Component: **Model Gateway** (api) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **Model Gateway** (`api`).

■ **Description:** Attacker floods expensive endpoints (e.g., search, login).

■ Attack scenario:

1. Attacker identifies a costly operation in Model Gateway (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Model Gateway's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Model Gateway; cascading failure may take down dependents.

DREAD — D:7 R:6 E:9 A:8 D:9 → **Total 39/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Privilege escalation via mass assignment

STRIDE → Elevation of Privilege | Component: **Model Gateway** (api) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **Model Gateway** (`api`).

■ **Description:** User submits extra fields (e.g., role=admin) and they bind to the model.

■ Attack scenario:

1. Attacker authenticates to Model Gateway as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:7 R:8 E:9 A:8 D:9 → **Total 41/50**

■ How to mitigate:

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to Model Gateway should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Trust-boundary crossing without strong authn: Agent Runtime / Orchestrator → Model Gateway [cross-boundary]

STRIDE → Spoofing | Component: **Model Gateway** (api) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Application tier → DMZ (public-facing)

■ **Location:** On the data flow **Agent Runtime / Orchestrator → Model Gateway** (label: *Agent Runtime / Orchestrator → Model Gateway*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `Application tier` into `DMZ (public-facing)`. The receiving component is **Model Gateway** (`api`).

■ **Description:** Flow 'Agent Runtime / Orchestrator → Model Gateway' crosses trust boundary 'Application tier' → 'DMZ (public-facing)'. Caller identity must be re-verified at the boundary; existing trust does not transit.

■■ Attack scenario:

1. Attacker positions themselves on the network path between source and destination, or compromises an upstream component in the source zone.
2. Crafts requests with forged or replayed identity claims (cookies, tokens, IP-based trust).
3. Model Gateway accepts the request because identity is not re-verified at the boundary, treating the upstream zone as trusted.
4. Attacker can now perform actions as the impersonated principal across the boundary.

DREAD — D:7 R:6 E:9 A:10 D:9 → **Total 41/50**

■ How to mitigate:

[preventive] **Re-authenticate at the boundary** — Require a fresh, audience-bound credential at the receiver. Do not infer identity from network position, source IP, or upstream session.

[preventive] **Use mutual TLS or signed tokens with audience claims** — For service-to-service calls into Model Gateway, terminate mTLS at the boundary and reject any caller without a valid client cert. For human-driven flows, use OAuth2 / OIDC with audience and issuer validation.

[detective] **Log every cross-boundary auth decision** — Emit an authentication event (success and failure) tagged with both source and destination zone. Forward to SIEM with retention ≥ 90 days.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Cross-boundary input not validated: Agent Runtime / Orchestrator → Model Gateway [cross-boundary]

STRIDE → Tampering | Component: **Model Gateway** (api) | Source: rule-based

CWE: CWE-20 — Improper Input Validation

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:N/I:H/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:N/VI:H/VA:N/SC:L/SI:H/SA:N

Boundary crossing: Application tier → DMZ (public-facing)

■ **Location:** On the data flow **Agent Runtime / Orchestrator → Model Gateway** (label: *Agent Runtime / Orchestrator → Model Gateway*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `Application tier` into `DMZ (public-facing)`. The receiving component is **Model Gateway** (`api`).

■ **Description:** Data crossing the trust boundary into 'DMZ (public-facing)' must be treated as untrusted, even if the source is internal. Implicit trust is the most common cause of injection / SSRF / deserialization bugs.

■ Attack scenario:

1. Attacker compromises a less-trusted upstream zone or directly sends malformed input to Model Gateway.
2. Sends payloads with unexpected types, lengths, or special characters (e.g. SQL meta-chars, NUL bytes, control sequences).
3. Model Gateway processes the input under the assumption that the upstream zone has already validated it — but the boundary changes the security context.
4. Injection, deserialization flaws, or buffer issues trigger; attacker pivots into Model Gateway or the zone behind it.

DREAD — D:7 R:6 E:9 A:10 D:9 → **Total 41/50**

■ How to mitigate:

[preventive] **Validate against an allow-list schema at the boundary** — Define an explicit schema (JSON Schema, Protobuf, OpenAPI) for every accepted message at this boundary. Reject anything that doesn't match — type, length, charset, enum values, nested depth.

[preventive] **Canonicalize before validation** — Decode URL-encoding, Unicode normalization (NFC), trim whitespace, and resolve relative paths before validating. Avoid double-decoding bypasses.

[preventive] **Apply context-specific output encoding** — Whatever Model Gateway does with the input — SQL: parameterized queries, HTML: contextual escaping, OS commands: avoid shell, use exec arrays, LDAP: escape filter chars.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-20 — Improper Input Validation](#)

Cross-boundary data exposure risk: Agent Runtime / Orchestrator → Model Gateway [cross-boundary]

STRIDE → Information Disclosure | Component: **Model Gateway** (api) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Application tier → DMZ (public-facing)

■ **Location:** On the data flow **Agent Runtime / Orchestrator** → **Model Gateway** (label: *Agent Runtime / Orchestrator → Model Gateway*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from 'Application tier' into 'DMZ (public-facing)'. The receiving component is **Model Gateway** ('api').

■ **Description:** Information leaving 'Application tier' into 'DMZ (public-facing)' may include data the receiving zone is not authorized to see. Cross-boundary egress is a common data-leak surface.

■ Attack scenario:

1. Attacker observes traffic leaving the source zone, or compromises a component in the destination zone.
2. The destination zone receives more data than it strictly needs (over-fetching, verbose error responses, full record dumps).
3. Attacker harvests sensitive fields (PII, secrets, internal IDs) that should never have left the source zone.
4. Data is exfiltrated, sold, or used to plan a deeper attack on the originating zone.

DREAD — D:7 R:6 E:9 A:10 D:10 → **Total 42/50**

■ How to mitigate:

[preventive] **Enforce minimum-data-needed at the egress point** — Whitelist exactly which fields leave the source zone. Strip everything else server-side before serialization.

[preventive] **Tokenize or redact sensitive fields** — Replace PII / PCI / PHI fields with reversible tokens or one-way hashes when the destination zone doesn't need the cleartext value.

[detective] **Log cross-boundary data flows for review** — Sample-log the field set crossing this boundary (not the values) so DLP and privacy reviews can audit what's egressing.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Authentication bypass via credential stuffing

STRIDE → Spoofing | Component: **Planning & Workflow** (api) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Planning & Workflow** ('api').

■ **Description:** Attackers reuse leaked credentials to impersonate legitimate users.

■ Attack scenario:

1. Attacker locates Planning & Workflow's identity claim mechanism (token format, header name, signing key handling).
2. Forges or replays a token to assert another principal's identity.
3. Planning & Workflow accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
4. Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:7 R:6 E:7 A:7 D:7 → **Total 34/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Session token hijacking

STRIDE → Spoofing | Component: **Planning & Workflow (api)** | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Planning & Workflow** (`api`).

■ **Description:** Attacker steals or predicts a session token to impersonate a user.

■ Attack scenario:

1. Attacker locates Planning & Workflow's identity claim mechanism (token format, header name, signing key handling).
2. Forges or replays a token to assert another principal's identity.
3. Planning & Workflow accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
4. Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:7 R:6 E:7 A:7 D:7 → **Total 34/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

API key / service identity spoofing

STRIDE → Spoofing | Component: **Planning & Workflow (api)** | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Planning & Workflow** (`api`).

■ **Description:** An attacker obtains or forges an API key to act as a trusted service.

■ Attack scenario:

1. Attacker locates Planning & Workflow's identity claim mechanism (token format, header name, signing key handling).
2. Forges or replays a token to assert another principal's identity.
3. Planning & Workflow accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).

4. Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:7 R:6 E:7 A:7 D:7 → **Total 34/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Log tampering

STRIDE → Repudiation | Component: **Planning & Workflow** (api) | Source: rule-based

CWE: CWE-778 — Insufficient Logging

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Planning & Workflow** (`api`).

■ **Description:** An attacker with access modifies or deletes audit trails.

■ ■ Attack scenario:

1. User performs a sensitive action through Planning & Workflow (a transfer, a permission change, a deletion).
2. Audit logs are absent, incomplete, or modifiable by the same principal who performed the action.
3. User later denies having performed the action, or an attacker covers their tracks.
4. Without tamper-evident logs, neither responsible party can be identified — leading to fraud, dispute, or regulatory exposure.

DREAD — D:7 R:6 E:7 A:7 D:7 → **Total 34/50**

■ How to mitigate:

[detective] **Emit tamper-evident audit logs** — Sign log entries (HMAC chain, hash-linked) so insertion or deletion is detectable. Forward to a separate, append-only system the action's principal cannot administer.

[detective] **Capture sufficient detail per audit event** — Who (authenticated principal, not just session ID), what (specific action and target), when (UTC, monotonic-clock-corroborated), where (source IP, request ID), why (correlated to the upstream business event).

[detective] **Periodic log integrity verification** — Schedule daily integrity checks on the audit log chain. Alert on any gap.

[A09:2021 — Security Logging and Monitoring Failures](#) | [STRIDE reference](#) | [CWE-778 — Insufficient Logging](#)

Sensitive data exposure in transit

STRIDE → Information Disclosure | Component: **Planning & Workflow** (api) | Source: rule-based

CWE: CWE-319 — Cleartext Transmission of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Planning & Workflow** (`api`).

■ **Description:** PII, secrets, or tokens transmitted over unencrypted channels.

■ ■ Attack scenario:

1. Attacker reaches Planning & Workflow via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:6 E:7 A:7 D:8 → **Total 35/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on

the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-319 — Cleartext Transmission of Sensitive Information](#)

Sensitive data exposure at rest

STRIDE → Information Disclosure | Component: **Planning & Workflow** (api) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Planning & Workflow** (`api`).

■ **Description:** Stored PII / secrets accessible without proper authorization.

■■ Attack scenario:

1. Attacker reaches Planning & Workflow via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:6 E:7 A:7 D:8 → **Total 35/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Insecure direct object reference (IDOR)

STRIDE → Information Disclosure | Component: **Planning & Workflow** (api) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Planning & Workflow** (`api`).

■ **Description:** User can access another user's resources by guessing IDs.

■■ Attack scenario:

1. Attacker reaches Planning & Workflow via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:8 E:7 A:7 D:8 → **Total 37/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Application-layer DDoS

STRIDE → Denial of Service | Component: **Planning & Workflow** (api) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **Planning & Workflow** (`api`).

■ **Description:** Attacker floods expensive endpoints (e.g., search, login).

■ ■ Attack scenario:

1. Attacker identifies a costly operation in Planning & Workflow (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Planning & Workflow's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Planning & Workflow; cascading failure may take down dependents.

DREAD — D:7 R:6 E:7 A:7 D:7 → **Total 34/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Privilege escalation via mass assignment

STRIDE → Elevation of Privilege | Component: **Planning & Workflow** (api) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **Planning & Workflow** (`api`).

■ **Description:** User submits extra fields (e.g., role=admin) and they bind to the model.

■ ■ Attack scenario:

1. Attacker authenticates to Planning & Workflow as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:7 R:8 E:7 A:7 D:7 → **Total 36/50**

■ How to mitigate:

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to Planning & Workflow should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

Data-in-transit modification

STRIDE → Tampering | Component: **Prompt & Cyber Logging** (datastore) | Source: rule-based

CWE: CWE-345 — Insufficient Verification of Data Authenticity

CVSS 3.1: 7.3 (High) [CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N](#)

CVSS 4.0: 5.2 (Medium) [CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Prompt & Cyber Logging** (`datastore`).

■ **Description:** Attacker on the network path alters request/response payloads.

■■ Attack scenario:

1. Attacker reaches an input vector exposed by Prompt & Cyber Logging (request body, query string, message queue payload, persisted state).
2. Submits malformed input designed to alter the program's logic or stored data (injection, parameter tampering, deserialization gadgets).
3. Prompt & Cyber Logging processes the input without strict validation, and the malicious change takes effect.
4. Tampered data is persisted, executed, or relayed downstream — corrupting integrity beyond the original entry point.

DREAD — D:8 R:6 E:7 A:5 D:7 → **Total 33/50**

■ How to mitigate:

[preventive] **Schema-validate and parameterize all inputs** — JSON Schema / OpenAPI validation at API edges; parameterized queries / prepared statements for SQL; ORM-level validators on persisted models.

[preventive] **Sign and verify integrity-critical messages** — For commands or state mutations, attach an HMAC or signature so downstream consumers can detect tampering in transit or at rest.

[detective] **Use append-only / write-once stores for audit-critical data** — Where data must not be silently changed, write to an append-only log (e.g., immutable S3 bucket, CloudTrail-like store) and reconcile against the mutable copy.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-345 — Insufficient Verification of Data Authenticity](#)

Config / secret tampering

STRIDE → Tampering | Component: **Prompt & Cyber Logging** (datastore) | Source: rule-based

CWE: CWE-345 — Insufficient Verification of Data Authenticity

CVSS 3.1: 7.3 (High) [CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N](#)

CVSS 4.0: 5.2 (Medium) [CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Prompt & Cyber Logging** (`datastore`).

■ **Description:** Unauthorized changes to runtime config alter security behavior.

■■ Attack scenario:

1. Attacker reaches an input vector exposed by Prompt & Cyber Logging (request body, query string, message queue payload, persisted state).
2. Submits malformed input designed to alter the program's logic or stored data (injection, parameter tampering, deserialization gadgets).
3. Prompt & Cyber Logging processes the input without strict validation, and the malicious change takes effect.
4. Tampered data is persisted, executed, or relayed downstream — corrupting integrity beyond the original entry point.

DREAD — D:8 R:6 E:7 A:5 D:7 → **Total 33/50**

■ How to mitigate:

[preventive] **Schema-validate and parameterize all inputs** — JSON Schema / OpenAPI validation at API edges; parameterized queries / prepared statements for SQL; ORM-level validators on persisted models.

[preventive] **Sign and verify integrity-critical messages** — For commands or state mutations, attach an HMAC or signature so downstream consumers can detect tampering in transit or at rest.

[detective] **Use append-only / write-once stores for audit-critical data** — Where data must not be silently changed, write to an append-only log (e.g., immutable S3 bucket, CloudTrail-like store) and reconcile against the mutable copy.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-345 — Insufficient Verification of Data Authenticity](#)

Sensitive data exposure in transit

STRIDE → Information Disclosure | Component: **Prompt & Cyber Logging** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) [CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 5.2 (Medium) [CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Prompt & Cyber Logging** (`datastore`).

■ **Description:** PII, secrets, or tokens transmitted over unencrypted channels.

■ ■ **Attack scenario:**

1. Attacker reaches Prompt & Cyber Logging via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:8 R:6 E:7 A:5 D:8 → **Total 34/50**

■ **How to mitigate:**

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Sensitive data exposure at rest

STRIDE → Information Disclosure | Component: **Prompt & Cyber Logging** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) [CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 5.2 (Medium) [CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Prompt & Cyber Logging** (`datastore`).

■ **Description:** Stored PII / secrets accessible without proper authorization.

■ ■ **Attack scenario:**

1. Attacker reaches Prompt & Cyber Logging via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:8 R:6 E:7 A:5 D:8 → **Total 34/50**

■ **How to mitigate:**

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Insecure direct object reference (IDOR)

STRIDE → Information Disclosure | Component: **Prompt & Cyber Logging** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) [CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 5.2 (Medium) [CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Prompt & Cyber Logging** (`datastore`).

■ **Description:** User can access another user's resources by guessing IDs.

■■ **Attack scenario:**

1. Attacker reaches Prompt & Cyber Logging via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:8 R:8 E:7 A:5 D:8 → **Total 36/50**

■ **How to mitigate:**

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Trust-boundary crossing without strong authn: Core Agent Layer → Prompt & Cyber Logging

[cross-boundary]

STRIDE → Spoofing | Component: **Prompt & Cyber Logging** (datastore) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 6.8 (Medium) [CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N](#)

CVSS 4.0: 5.8 (Medium) [CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N](#)

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Core Agent Layer** → **Prompt & Cyber Logging** (label: *Core Agent Layer → Prompt & Cyber Logging*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Prompt & Cyber Logging** (`datastore`).

■ **Description:** Flow 'Core Agent Layer → Prompt & Cyber Logging' crosses trust boundary 'Application tier' → 'Data tier'. Caller identity must be re-verified at the boundary; existing trust does not transit.

■■ **Attack scenario:**

1. Attacker positions themselves on the network path between source and destination, or compromises an upstream component in the source zone.
2. Crafts requests with forged or replayed identity claims (cookies, tokens, IP-based trust).
3. Prompt & Cyber Logging accepts the request because identity is not re-verified at the boundary, treating the upstream zone as trusted.
4. Attacker can now perform actions as the impersonated principal across the boundary.

DREAD — D:8 R:6 E:7 A:7 D:7 → **Total 35/50**

■ **How to mitigate:**

[preventive] **Re-authenticate at the boundary** — Require a fresh, audience-bound credential at the receiver. Do not infer identity from network position, source IP, or upstream session.

[preventive] **Use mutual TLS or signed tokens with audience claims** — For service-to-service calls into Prompt & Cyber Logging, terminate mTLS at the boundary and reject any caller without a valid client cert. For human-driven flows, use OAuth2 / OIDC with audience and issuer validation.

[detective] **Log every cross-boundary auth decision** — Emit an authentication event (success and failure) tagged with both source and destination zone. Forward to SIEM with retention ≥ 90 days.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Cross-boundary input not validated: Core Agent Layer → Prompt & Cyber Logging [cross-boundary]

STRIDE → Tampering | Component: **Prompt & Cyber Logging** (datastore) | Source: rule-based

CWE: CWE-20 — Improper Input Validation

CVSS 3.1: 8.7 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:L/SI:H/SA:N

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Core Agent Layer** → **Prompt & Cyber Logging** (label: *Core Agent Layer → Prompt & Cyber Logging*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Prompt & Cyber Logging** (`datastore`).

■ **Description:** Data crossing the trust boundary into 'Data tier' must be treated as untrusted, even if the source is internal. Implicit trust is the most common cause of injection / SSRF / deserialization bugs.

■ Attack scenario:

1. Attacker compromises a less-trusted upstream zone or directly sends malformed input to Prompt & Cyber Logging.
2. Sends payloads with unexpected types, lengths, or special characters (e.g. SQL meta-chars, NUL bytes, control sequences).
3. Prompt & Cyber Logging processes the input under the assumption that the upstream zone has already validated it — but the boundary changes the security context.
4. Injection, deserialization flaws, or buffer issues trigger; attacker pivots into Prompt & Cyber Logging or the zone behind it.

DREAD — D:8 R:6 E:7 A:7 D:7 → **Total 35/50**

■ How to mitigate:

[preventive] **Validate against an allow-list schema at the boundary** — Define an explicit schema (JSON Schema, Protobuf, OpenAPI) for every accepted message at this boundary. Reject anything that doesn't match — type, length, charset, enum values, nested depth.

[preventive] **Canonicalize before validation** — Decode URL-encoding, Unicode normalization (NFC), trim whitespace, and resolve relative paths before validating. Avoid double-decoding bypasses.

[preventive] **Apply context-specific output encoding** — Whatever Prompt & Cyber Logging does with the input — SQL: parameterized queries, HTML: contextual escaping, OS commands: avoid shell, use exec arrays, LDAP: escape filter chars.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-20 — Improper Input Validation](#)

Cross-boundary data exposure risk: Core Agent Layer → Prompt & Cyber Logging [cross-boundary]

STRIDE → Information Disclosure | Component: **Prompt & Cyber Logging** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 6.8 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Core Agent Layer** → **Prompt & Cyber Logging** (label: *Core Agent Layer → Prompt & Cyber Logging*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Prompt & Cyber Logging** (`datastore`).

■ **Description:** Information leaving 'Application tier' into 'Data tier' may include data the receiving zone is not authorized to see. Cross-boundary egress is a common data-leak surface.

■ Attack scenario:

1. Attacker observes traffic leaving the source zone, or compromises a component in the destination zone.
2. The destination zone receives more data than it strictly needs (over-fetching, verbose error responses, full record dumps).
3. Attacker harvests sensitive fields (PII, secrets, internal IDs) that should never have left the source zone.
4. Data is exfiltrated, sold, or used to plan a deeper attack on the originating zone.

DREAD — D:8 R:6 E:7 A:7 D:8 → **Total 36/50**

■ How to mitigate:

[preventive] **Enforce minimum-data-needed at the egress point** — Whitelist exactly which fields leave the source zone. Strip everything else server-side before serialization.

[preventive] **Tokenize or redact sensitive fields** — Replace PII / PCI / PHI fields with reversible tokens or one-way hashes when the destination

zone doesn't need the cleartext value.

[detective] **Log cross-boundary data flows for review** — Sample-log the field set crossing this boundary (not the values) so DLP and privacy reviews can audit what's egressing.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Authentication bypass via credential stuffing

STRIDE → Spoofing | Component: **Retrieval (RAG)** (api) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Retrieval (RAG)** (`api`).

■ **Description:** Attackers reuse leaked credentials to impersonate legitimate users.

■ ■ Attack scenario:

1. Attacker locates Retrieval (RAG)'s identity claim mechanism (token format, header name, signing key handling).
2. Forges or replays a token to assert another principal's identity.
3. Retrieval (RAG) accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
4. Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:7 R:6 E:7 A:8 D:7 → **Total 35/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Session token hijacking

STRIDE → Spoofing | Component: **Retrieval (RAG)** (api) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Retrieval (RAG)** (`api`).

■ **Description:** Attacker steals or predicts a session token to impersonate a user.

■ ■ Attack scenario:

1. Attacker locates Retrieval (RAG)'s identity claim mechanism (token format, header name, signing key handling).
2. Forges or replays a token to assert another principal's identity.
3. Retrieval (RAG) accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
4. Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:7 R:6 E:7 A:8 D:7 → **Total 35/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

API key / service identity spoofing

STRIDE → Spoofing | Component: **Retrieval (RAG)** (api) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Retrieval (RAG)** (`api`).

■ **Description:** An attacker obtains or forges an API key to act as a trusted service.

■ ■ Attack scenario:

1. Attacker locates Retrieval (RAG)'s identity claim mechanism (token format, header name, signing key handling).
2. Forges or replays a token to assert another principal's identity.
3. Retrieval (RAG) accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
4. Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:7 R:6 E:7 A:8 D:7 → **Total 35/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Log tampering

STRIDE → Repudiation | Component: **Retrieval (RAG)** (api) | Source: rule-based

CWE: CWE-778 — Insufficient Logging

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:H/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Retrieval (RAG)** (`api`).

■ **Description:** An attacker with access modifies or deletes audit trails.

■ ■ Attack scenario:

1. User performs a sensitive action through Retrieval (RAG) (a transfer, a permission change, a deletion).
2. Audit logs are absent, incomplete, or modifiable by the same principal who performed the action.
3. User later denies having performed the action, or an attacker covers their tracks.
4. Without tamper-evident logs, neither responsible party can be identified — leading to fraud, dispute, or regulatory exposure.

DREAD — D:7 R:6 E:7 A:8 D:7 → **Total 35/50**

■ How to mitigate:

[detective] **Emit tamper-evident audit logs** — Sign log entries (HMAC chain, hash-linked) so insertion or deletion is detectable. Forward to a separate, append-only system the action's principal cannot administer.

[detective] **Capture sufficient detail per audit event** — Who (authenticated principal, not just session ID), what (specific action and target), when (UTC, monotonic-clock-corroborated), where (source IP, request ID), why (correlated to the upstream business event).

[detective] **Periodic log integrity verification** — Schedule daily integrity checks on the audit log chain. Alert on any gap.

[A09:2021 — Security Logging and Monitoring Failures](#) | [STRIDE reference](#) | [CWE-778 — Insufficient Logging](#)

Sensitive data exposure in transit

STRIDE → Information Disclosure | Component: **Retrieval (RAG)** (api) | Source: rule-based

CWE: CWE-319 — Cleartext Transmission of Sensitive Information

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Retrieval (RAG)** (`api`).

■ **Description:** PII, secrets, or tokens transmitted over unencrypted channels.

■ ■ **Attack scenario:**

1. Attacker reaches Retrieval (RAG) via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:6 E:7 A:8 D:8 → **Total 36/50**

■ **How to mitigate:**

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-319 — Cleartext Transmission of Sensitive Information](#)

Sensitive data exposure at rest

STRIDE → Information Disclosure | Component: **Retrieval (RAG)** (api) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Retrieval (RAG)** (`api`).

■ **Description:** Stored PII / secrets accessible without proper authorization.

■ ■ **Attack scenario:**

1. Attacker reaches Retrieval (RAG) via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:6 E:7 A:8 D:8 → **Total 36/50**

■ **How to mitigate:**

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Insecure direct object reference (IDOR)

STRIDE → Information Disclosure | Component: **Retrieval (RAG)** (api) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Retrieval (RAG)** (`api`).

■ **Description:** User can access another user's resources by guessing IDs.

■■ Attack scenario:

1. Attacker reaches Retrieval (RAG) via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:8 E:7 A:8 D:8 → **Total 38/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Application-layer DDoS

STRIDE → Denial of Service | Component: **Retrieval (RAG)** (api) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **Retrieval (RAG)** (`api`).

■ **Description:** Attacker floods expensive endpoints (e.g., search, login).

■■ Attack scenario:

1. Attacker identifies a costly operation in Retrieval (RAG) (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Retrieval (RAG)'s resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Retrieval (RAG); cascading failure may take down dependents.

DREAD — D:7 R:6 E:7 A:8 D:7 → **Total 35/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Privilege escalation via mass assignment

STRIDE → Elevation of Privilege | Component: **Retrieval (RAG)** (api) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **Retrieval (RAG)** (`api`).

■ **Description:** User submits extra fields (e.g., role=admin) and they bind to the model.

■■ Attack scenario:

1. Attacker authenticates to Retrieval (RAG) as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.

- Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
- Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:7 R:8 E:7 A:8 D:7 → **Total 37/50**

■ How to mitigate:

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to Retrieval (RAG) should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Authentication bypass via credential stuffing

STRIDE → Spoofing | Component: **SailPoint Agent Identity** (auth_service) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **SailPoint Agent Identity** (auth_service).

■ **Description:** Attackers reuse leaked credentials to impersonate legitimate users.

■ ■ Attack scenario:

- Attacker locates SailPoint Agent Identity's identity claim mechanism (token format, header name, signing key handling).
- Forges or replays a token to assert another principal's identity.
- SailPoint Agent Identity accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
- Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:8 R:6 E:9 A:7 D:9 → **Total 39/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Session token hijacking

STRIDE → Spoofing | Component: **SailPoint Agent Identity** (auth_service) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **SailPoint Agent Identity** (auth_service).

■ **Description:** Attacker steals or predicts a session token to impersonate a user.

■ ■ Attack scenario:

- Attacker locates SailPoint Agent Identity's identity claim mechanism (token format, header name, signing key handling).
- Forges or replays a token to assert another principal's identity.
- SailPoint Agent Identity accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
- Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:8 R:6 E:9 A:7 D:9 → **Total 39/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

API key / service identity spoofing

STRIDE → Spoofing | Component: **SailPoint Agent Identity** (auth_service) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **SailPoint Agent Identity** (^auth_service^).

■ **Description:** An attacker obtains or forges an API key to act as a trusted service.

■■ Attack scenario:

1. Attacker locates SailPoint Agent Identity's identity claim mechanism (token format, header name, signing key handling).
2. Forges or replays a token to assert another principal's identity.
3. SailPoint Agent Identity accepts the claim because verification is missing or weak (e.g., signature not checked, expired tokens accepted, no audience validation).
4. Attacker acts as the spoofed principal — reading data, triggering workflows, or pivoting to internal services.

DREAD — D:8 R:6 E:9 A:7 D:9 → **Total 39/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Log tampering

STRIDE → Repudiation | Component: **SailPoint Agent Identity** (auth_service) | Source: rule-based

CWE: CWE-778 — Insufficient Logging

CVSS 3.1: 9.1 (Critical) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **SailPoint Agent Identity** (^auth_service^).

■ **Description:** An attacker with access modifies or deletes audit trails.

■■ Attack scenario:

1. User performs a sensitive action through SailPoint Agent Identity (a transfer, a permission change, a deletion).
2. Audit logs are absent, incomplete, or modifiable by the same principal who performed the action.
3. User later denies having performed the action, or an attacker covers their tracks.
4. Without tamper-evident logs, neither responsible party can be identified — leading to fraud, dispute, or regulatory exposure.

DREAD — D:8 R:6 E:9 A:7 D:9 → **Total 39/50**

■ How to mitigate:

[detective] **Emit tamper-evident audit logs** — Sign log entries (HMAC chain, hash-linked) so insertion or deletion is detectable. Forward to a separate, append-only system the action's principal cannot administer.

[detective] **Capture sufficient detail per audit event** — Who (authenticated principal, not just session ID), what (specific action and target), when (UTC, monotonic-clock-corroborated), where (source IP, request ID), why (correlated to the upstream business event).

[detective] **Periodic log integrity verification** — Schedule daily integrity checks on the audit log chain. Alert on any gap.

Application-layer DDoS

STRIDE → Denial of Service | Component: **SailPoint Agent Identity** (auth_service) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 9.1 (Critical) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **SailPoint Agent Identity** (auth_service).

■ **Description:** Attacker floods expensive endpoints (e.g., search, login).

■ ■ Attack scenario:

1. Attacker identifies a costly operation in SailPoint Agent Identity (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. SailPoint Agent Identity's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach SailPoint Agent Identity; cascading failure may take down dependents.

DREAD — D:8 R:6 E:9 A:7 D:9 → **Total 39/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Privilege escalation via mass assignment

STRIDE → Elevation of Privilege | Component: **SailPoint Agent Identity** (auth_service) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 8.3 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **SailPoint Agent Identity** (auth_service).

■ **Description:** User submits extra fields (e.g., role=admin) and they bind to the model.

■ ■ Attack scenario:

1. Attacker authenticates to SailPoint Agent Identity as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:8 R:8 E:9 A:7 D:9 → **Total 41/50**

■ How to mitigate:

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to SailPoint Agent Identity should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Trust-boundary crossing without strong authn: Agent Gateway → SailPoint Agent Identity [cross-boundary]

STRIDE → Spoofing | Component: **SailPoint Agent Identity** (auth_service) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: DMZ (public-facing) → Application tier

■ **Location:** On the data flow **Agent Gateway → SailPoint Agent Identity** (label: *Agent Gateway → SailPoint Agent Identity*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `DMZ (public-facing)` into `Application tier`. The receiving component is **SailPoint Agent Identity** (`auth\_service`).

■ **Description:** Flow 'Agent Gateway → SailPoint Agent Identity' crosses trust boundary 'DMZ (public-facing)' → 'Application tier'. Caller identity must be re-verified at the boundary; existing trust does not transit.

■ ■ **Attack scenario:**

1. Attacker positions themselves on the network path between source and destination, or compromises an upstream component in the source zone.
2. Crafts requests with forged or replayed identity claims (cookies, tokens, IP-based trust).
3. SailPoint Agent Identity accepts the request because identity is not re-verified at the boundary, treating the upstream zone as trusted.
4. Attacker can now perform actions as the impersonated principal across the boundary.

DREAD — D:8 R:6 E:9 A:9 D:9 → **Total 41/50**

■ **How to mitigate:**

[preventive] **Re-authenticate at the boundary** — Require a fresh, audience-bound credential at the receiver. Do not infer identity from network position, source IP, or upstream session.

[preventive] **Use mutual TLS or signed tokens with audience claims** — For service-to-service calls into SailPoint Agent Identity, terminate mTLS at the boundary and reject any caller without a valid client cert. For human-driven flows, use OAuth2 / OIDC with audience and issuer validation.

[detective] **Log every cross-boundary auth decision** — Emit an authentication event (success and failure) tagged with both source and destination zone. Forward to SIEM with retention ≥ 90 days.

A07:2021 — Identification and Authentication Failures | STRIDE reference | CWE-287 — Improper Authentication

Cross-boundary input not validated: Agent Gateway → SailPoint Agent Identity [cross-boundary]

STRIDE → Tampering | Component: **SailPoint Agent Identity** (auth_service) | Source: rule-based

CWE: CWE-20 — Improper Input Validation

CVSS 3.1: 9.6 (Critical) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:L/SI:H/SA:N

Boundary crossing: DMZ (public-facing) → Application tier

■ **Location:** On the data flow **Agent Gateway → SailPoint Agent Identity** (label: *Agent Gateway → SailPoint Agent Identity*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `DMZ (public-facing)` into `Application tier`. The receiving component is **SailPoint Agent Identity** (`auth\_service`).

■ **Description:** Data crossing the trust boundary into 'Application tier' must be treated as untrusted, even if the source is internal. Implicit trust is the most common cause of injection / SSRF / deserialization bugs.

■ ■ **Attack scenario:**

1. Attacker compromises a less-trusted upstream zone or directly sends malformed input to SailPoint Agent Identity.
2. Sends payloads with unexpected types, lengths, or special characters (e.g. SQL meta-chars, NUL bytes, control sequences).
3. SailPoint Agent Identity processes the input under the assumption that the upstream zone has already validated it — but the boundary changes the security context.
4. Injection, deserialization flaws, or buffer issues trigger; attacker pivots into SailPoint Agent Identity or the zone behind it.

DREAD — D:8 R:6 E:9 A:9 D:9 → **Total 41/50**

■ **How to mitigate:**

[preventive] **Validate against an allow-list schema at the boundary** — Define an explicit schema (JSON Schema, Protobuf, OpenAPI) for every accepted message at this boundary. Reject anything that doesn't match — type, length, charset, enum values, nested depth.

[preventive] **Canonicalize before validation** — Decode URL-encoding, Unicode normalization (NFC), trim whitespace, and resolve relative paths before validating. Avoid double-decoding bypasses.

[preventive] **Apply context-specific output encoding** — Whatever SailPoint Agent Identity does with the input — SQL: parameterized queries, HTML: contextual escaping, OS commands: avoid shell, use exec arrays, LDAP: escape filter chars.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-20 — Improper Input Validation](#)

Cross-boundary data exposure risk: Agent Gateway → SailPoint Agent Identity [cross-boundary]

STRIDE → Information Disclosure | Component: **SailPoint Agent Identity** (auth_service) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: DMZ (public-facing) → Application tier

■ **Location:** On the data flow **Agent Gateway → SailPoint Agent Identity** (label: *Agent Gateway → SailPoint Agent Identity*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `DMZ (public-facing)` into `Application tier`. The receiving component is **SailPoint Agent Identity** (`auth\_service`).

■ **Description:** Information leaving 'DMZ (public-facing)' into 'Application tier' may include data the receiving zone is not authorized to see. Cross-boundary egress is a common data-leak surface.

■ Attack scenario:

1. Attacker observes traffic leaving the source zone, or compromises a component in the destination zone.
2. The destination zone receives more data than it strictly needs (over-fetching, verbose error responses, full record dumps).
3. Attacker harvests sensitive fields (PII, secrets, internal IDs) that should never have left the source zone.
4. Data is exfiltrated, sold, or used to plan a deeper attack on the originating zone.

DREAD — D:8 R:6 E:9 A:9 D:10 → **Total 42/50**

■ How to mitigate:

[preventive] **Enforce minimum-data-needed at the egress point** — Whitelist exactly which fields leave the source zone. Strip everything else server-side before serialization.

[preventive] **Tokenize or redact sensitive fields** — Replace PII / PCI / PHI fields with reversible tokens or one-way hashes when the destination zone doesn't need the cleartext value.

[detective] **Log cross-boundary data flows for review** — Sample-log the field set crossing this boundary (not the values) so DLP and privacy reviews can audit what's egressing.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Data-in-transit modification

STRIDE → Tampering | Component: **Session Memory** (datastore) | Source: rule-based

CWE: CWE-345 — Insufficient Verification of Data Authenticity

CVSS 3.1: 7.3 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Session Memory** (`datastore`).

■ **Description:** Attacker on the network path alters request/response payloads.

■ Attack scenario:

1. Attacker reaches an input vector exposed by Session Memory (request body, query string, message queue payload, persisted state).
2. Submits malformed input designed to alter the program's logic or stored data (injection, parameter tampering, deserialization gadgets).
3. Session Memory processes the input without strict validation, and the malicious change takes effect.
4. Tampered data is persisted, executed, or relayed downstream — corrupting integrity beyond the original entry point.

DREAD — D:8 R:6 E:7 A:5 D:7 → **Total 33/50**

■ How to mitigate:

[preventive] **Schema-validate and parameterize all inputs** — JSON Schema / OpenAPI validation at API edges; parameterized queries / prepared statements for SQL; ORM-level validators on persisted models.

[preventive] **Sign and verify integrity-critical messages** — For commands or state mutations, attach an HMAC or signature so downstream consumers can detect tampering in transit or at rest.

[detective] **Use append-only / write-once stores for audit-critical data** — Where data must not be silently changed, write to an append-only log (e.g., immutable S3 bucket, CloudTrail-like store) and reconcile against the mutable copy.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-345 — Insufficient Verification of Data Authenticity](#)

Config / secret tampering

STRIDE → Tampering | Component: **Session Memory** (datastore) | Source: rule-based

CWE: CWE-345 — Insufficient Verification of Data Authenticity

CVSS 3.1: 7.3 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Session Memory** (datastore).

■ **Description:** Unauthorized changes to runtime config alter security behavior.

■ ■ Attack scenario:

1. Attacker reaches an input vector exposed by Session Memory (request body, query string, message queue payload, persisted state).
2. Submits malformed input designed to alter the program's logic or stored data (injection, parameter tampering, deserialization gadgets).
3. Session Memory processes the input without strict validation, and the malicious change takes effect.
4. Tampered data is persisted, executed, or relayed downstream — corrupting integrity beyond the original entry point.

DREAD — D:8 R:6 E:7 A:5 D:7 → **Total 33/50**

■ How to mitigate:

[preventive] **Schema-validate and parameterize all inputs** — JSON Schema / OpenAPI validation at API edges; parameterized queries / prepared statements for SQL; ORM-level validators on persisted models.

[preventive] **Sign and verify integrity-critical messages** — For commands or state mutations, attach an HMAC or signature so downstream consumers can detect tampering in transit or at rest.

[detective] **Use append-only / write-once stores for audit-critical data** — Where data must not be silently changed, write to an append-only log (e.g., immutable S3 bucket, CloudTrail-like store) and reconcile against the mutable copy.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-345 — Insufficient Verification of Data Authenticity](#)

Sensitive data exposure in transit

STRIDE → Information Disclosure | Component: **Session Memory** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Session Memory** (datastore).

■ **Description:** PII, secrets, or tokens transmitted over unencrypted channels.

■ ■ Attack scenario:

1. Attacker reaches Session Memory via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:8 R:6 E:7 A:5 D:8 → **Total 34/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on

the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Sensitive data exposure at rest

STRIDE → Information Disclosure | Component: **Session Memory** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Session Memory** (`datastore`).

■ **Description:** Stored PII / secrets accessible without proper authorization.

■■ Attack scenario:

1. Attacker reaches Session Memory via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:8 R:6 E:7 A:5 D:8 → **Total 34/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Insecure direct object reference (IDOR)

STRIDE → Information Disclosure | Component: **Session Memory** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Session Memory** (`datastore`).

■ **Description:** User can access another user's resources by guessing IDs.

■■ Attack scenario:

1. Attacker reaches Session Memory via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:8 R:8 E:7 A:5 D:8 → **Total 36/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Trust-boundary crossing without strong authn: Agent Runtime / Orchestrator → Session Memory [cross-boundary]

STRIDE → Spoofing | Component: **Session Memory** (datastore) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 6.8 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Agent Runtime / Orchestrator** → **Session Memory** (label: *Agent Runtime / Orchestrator → Session Memory*, protocol: TCP, auth: credentials, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Session Memory** (`datastore`).

■ **Description:** Flow 'Agent Runtime / Orchestrator → Session Memory' crosses trust boundary 'Application tier' → 'Data tier'. Caller identity must be re-verified at the boundary; existing trust does not transit.

■ ■ Attack scenario:

1. Attacker positions themselves on the network path between source and destination, or compromises an upstream component in the source zone.
2. Crafts requests with forged or replayed identity claims (cookies, tokens, IP-based trust).
3. Session Memory accepts the request because identity is not re-verified at the boundary, treating the upstream zone as trusted.
4. Attacker can now perform actions as the impersonated principal across the boundary.

DREAD — D:8 R:6 E:7 A:7 D:7 → **Total 35/50**

■ How to mitigate:

[preventive] **Re-authenticate at the boundary** — Require a fresh, audience-bound credential at the receiver. Do not infer identity from network position, source IP, or upstream session.

[preventive] **Use mutual TLS or signed tokens with audience claims** — For service-to-service calls into Session Memory, terminate mTLS at the boundary and reject any caller without a valid client cert. For human-driven flows, use OAuth2 / OIDC with audience and issuer validation.

[detective] **Log every cross-boundary auth decision** — Emit an authentication event (success and failure) tagged with both source and destination zone. Forward to SIEM with retention ≥ 90 days.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Cross-boundary input not validated: Agent Runtime / Orchestrator → Session Memory [cross-boundary]

STRIDE → Tampering | Component: **Session Memory** (datastore) | Source: rule-based

CWE: CWE-20 — Improper Input Validation

CVSS 3.1: 8.7 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:L/SI:H/SA:N

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Agent Runtime / Orchestrator** → **Session Memory** (label: *Agent Runtime / Orchestrator → Session Memory*, protocol: TCP, auth: credentials, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Session Memory** (`datastore`).

■ **Description:** Data crossing the trust boundary into 'Data tier' must be treated as untrusted, even if the source is internal. Implicit trust is the most common cause of injection / SSRF / deserialization bugs.

■ ■ Attack scenario:

1. Attacker compromises a less-trusted upstream zone or directly sends malformed input to Session Memory.
2. Sends payloads with unexpected types, lengths, or special characters (e.g. SQL meta-chars, NUL bytes, control sequences).
3. Session Memory processes the input under the assumption that the upstream zone has already validated it — but the boundary changes the security context.

4. Injection, deserialization flaws, or buffer issues trigger; attacker pivots into Session Memory or the zone behind it.

DREAD — D:8 R:6 E:7 A:7 D:7 → **Total 35/50**

■ How to mitigate:

[preventive] **Validate against an allow-list schema at the boundary** — Define an explicit schema (JSON Schema, Protobuf, OpenAPI) for every accepted message at this boundary. Reject anything that doesn't match — type, length, charset, enum values, nested depth.

[preventive] **Canonicalize before validation** — Decode URL-encoding, Unicode normalization (NFC), trim whitespace, and resolve relative paths before validating. Avoid double-decoding bypasses.

[preventive] **Apply context-specific output encoding** — Whatever Session Memory does with the input — SQL: parameterized queries, HTML: contextual escaping, OS commands: avoid shell, use exec arrays, LDAP: escape filter chars.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-20 — Improper Input Validation](#)

Cross-boundary data exposure risk: Agent Runtime / Orchestrator → Session Memory [cross-boundary]

STRIDE → Information Disclosure | Component: **Session Memory** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 6.8 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Agent Runtime / Orchestrator** → **Session Memory** (label: *Agent Runtime / Orchestrator → Session Memory*, protocol: TCP, auth: credentials, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Session Memory** (`datastore`).

■ **Description:** Information leaving 'Application tier' into 'Data tier' may include data the receiving zone is not authorized to see. Cross-boundary egress is a common data-leak surface.

■■ Attack scenario:

1. Attacker observes traffic leaving the source zone, or compromises a component in the destination zone.
2. The destination zone receives more data than it strictly needs (over-fetching, verbose error responses, full record dumps).
3. Attacker harvests sensitive fields (PII, secrets, internal IDs) that should never have left the source zone.
4. Data is exfiltrated, sold, or used to plan a deeper attack on the originating zone.

DREAD — D:8 R:6 E:7 A:7 D:8 → **Total 36/50**

■ How to mitigate:

[preventive] **Enforce minimum-data-needed at the egress point** — Whitelist exactly which fields leave the source zone. Strip everything else server-side before serialization.

[preventive] **Tokenize or redact sensitive fields** — Replace PII / PCI / PHI fields with reversible tokens or one-way hashes when the destination zone doesn't need the cleartext value.

[detective] **Log cross-boundary data flows for review** — Sample-log the field set crossing this boundary (not the values) so DLP and privacy reviews can audit what's egressing.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Data-in-transit modification

STRIDE → Tampering | Component: **Short Term Memory** (datastore) | Source: rule-based

CWE: CWE-345 — Insufficient Verification of Data Authenticity

CVSS 3.1: 7.3 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Short Term Memory** (`datastore`).

■ **Description:** Attacker on the network path alters request/response payloads.

■■ Attack scenario:

1. Attacker reaches an input vector exposed by Short Term Memory (request body, query string, message queue payload, persisted state).
2. Submits malformed input designed to alter the program's logic or stored data (injection, parameter tampering, deserialization)

gadgets).

3. Short Term Memory processes the input without strict validation, and the malicious change takes effect.
4. Tampered data is persisted, executed, or relayed downstream — corrupting integrity beyond the original entry point.

DREAD — D:8 R:6 E:7 A:5 D:7 → **Total 33/50**

■ How to mitigate:

[preventive] **Schema-validate and parameterize all inputs** — JSON Schema / OpenAPI validation at API edges; parameterized queries / prepared statements for SQL; ORM-level validators on persisted models.

[preventive] **Sign and verify integrity-critical messages** — For commands or state mutations, attach an HMAC or signature so downstream consumers can detect tampering in transit or at rest.

[detective] **Use append-only / write-once stores for audit-critical data** — Where data must not be silently changed, write to an append-only log (e.g., immutable S3 bucket, CloudTrail-like store) and reconcile against the mutable copy.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-345 — Insufficient Verification of Data Authenticity](#)

Config / secret tampering

STRIDE → Tampering | Component: **Short Term Memory** (datastore) | Source: rule-based

CWE: CWE-345 — Insufficient Verification of Data Authenticity

CVSS 3.1: 7.3 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Short Term Memory** (`datastore`).

■ **Description:** Unauthorized changes to runtime config alter security behavior.

■■ Attack scenario:

1. Attacker reaches an input vector exposed by Short Term Memory (request body, query string, message queue payload, persisted state).
2. Submits malformed input designed to alter the program's logic or stored data (injection, parameter tampering, deserialization gadgets).
3. Short Term Memory processes the input without strict validation, and the malicious change takes effect.
4. Tampered data is persisted, executed, or relayed downstream — corrupting integrity beyond the original entry point.

DREAD — D:8 R:6 E:7 A:5 D:7 → **Total 33/50**

■ How to mitigate:

[preventive] **Schema-validate and parameterize all inputs** — JSON Schema / OpenAPI validation at API edges; parameterized queries / prepared statements for SQL; ORM-level validators on persisted models.

[preventive] **Sign and verify integrity-critical messages** — For commands or state mutations, attach an HMAC or signature so downstream consumers can detect tampering in transit or at rest.

[detective] **Use append-only / write-once stores for audit-critical data** — Where data must not be silently changed, write to an append-only log (e.g., immutable S3 bucket, CloudTrail-like store) and reconcile against the mutable copy.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-345 — Insufficient Verification of Data Authenticity](#)

Sensitive data exposure in transit

STRIDE → Information Disclosure | Component: **Short Term Memory** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Short Term Memory** (`datastore`).

■ **Description:** PII, secrets, or tokens transmitted over unencrypted channels.

■■ Attack scenario:

1. Attacker reaches Short Term Memory via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.

4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:8 R:6 E:7 A:5 D:8 → **Total 34/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Sensitive data exposure at rest

STRIDE → Information Disclosure | Component: **Short Term Memory** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Short Term Memory** (`datastore`).

■ **Description:** Stored PII / secrets accessible without proper authorization.

■■ Attack scenario:

1. Attacker reaches Short Term Memory via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:8 R:6 E:7 A:5 D:8 → **Total 34/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Insecure direct object reference (IDOR)

STRIDE → Information Disclosure | Component: **Short Term Memory** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Short Term Memory** (`datastore`).

■ **Description:** User can access another user's resources by guessing IDs.

■■ Attack scenario:

1. Attacker reaches Short Term Memory via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:8 R:8 E:7 A:5 D:8 → **Total 36/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Trust-boundary crossing without strong authn: Agent Runtime / Orchestrator → Short Term Memory [cross-boundary]

STRIDE → Spoofing | Component: **Short Term Memory** (datastore) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 6.8 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Agent Runtime / Orchestrator → Short Term Memory** (label: *Agent Runtime / Orchestrator → Short Term Memory*, protocol: TCP, auth: credentials, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Short Term Memory** (`datastore`).

■ **Description:** Flow 'Agent Runtime / Orchestrator → Short Term Memory' crosses trust boundary 'Application tier' → 'Data tier'. Caller identity must be re-verified at the boundary; existing trust does not transit.

■■ Attack scenario:

1. Attacker positions themselves on the network path between source and destination, or compromises an upstream component in the source zone.
2. Crafts requests with forged or replayed identity claims (cookies, tokens, IP-based trust).
3. Short Term Memory accepts the request because identity is not re-verified at the boundary, treating the upstream zone as trusted.
4. Attacker can now perform actions as the impersonated principal across the boundary.

DREAD — D:8 R:6 E:7 A:7 D:7 → **Total 35/50**

■ How to mitigate:

[preventive] **Re-authenticate at the boundary** — Require a fresh, audience-bound credential at the receiver. Do not infer identity from network position, source IP, or upstream session.

[preventive] **Use mutual TLS or signed tokens with audience claims** — For service-to-service calls into Short Term Memory, terminate mTLS at the boundary and reject any caller without a valid client cert. For human-driven flows, use OAuth2 / OIDC with audience and issuer validation.

[detective] **Log every cross-boundary auth decision** — Emit an authentication event (success and failure) tagged with both source and destination zone. Forward to SIEM with retention ≥ 90 days.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Cross-boundary input not validated: Agent Runtime / Orchestrator → Short Term Memory [cross-boundary]

STRIDE → Tampering | Component: **Short Term Memory** (datastore) | Source: rule-based

CWE: CWE-20 — Improper Input Validation

CVSS 3.1: 8.7 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:L/SI:H/SA:N

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Agent Runtime / Orchestrator → Short Term Memory** (label: *Agent Runtime / Orchestrator → Short Term Memory*, protocol: TCP, auth: credentials, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Short Term Memory** (`datastore`).

■ **Description:** Data crossing the trust boundary into 'Data tier' must be treated as untrusted, even if the source is internal. Implicit trust is the most common cause of injection / SSRF / deserialization bugs.

■ Attack scenario:

1. Attacker compromises a less-trusted upstream zone or directly sends malformed input to Short Term Memory.
2. Sends payloads with unexpected types, lengths, or special characters (e.g. SQL meta-chars, NUL bytes, control sequences).
3. Short Term Memory processes the input under the assumption that the upstream zone has already validated it — but the boundary changes the security context.
4. Injection, deserialization flaws, or buffer issues trigger; attacker pivots into Short Term Memory or the zone behind it.

DREAD — D:8 R:6 E:7 A:7 D:7 → **Total 35/50**

■ How to mitigate:

[preventive] **Validate against an allow-list schema at the boundary** — Define an explicit schema (JSON Schema, Protobuf, OpenAPI) for every accepted message at this boundary. Reject anything that doesn't match — type, length, charset, enum values, nested depth.

[preventive] **Canonicalize before validation** — Decode URL-encoding, Unicode normalization (NFC), trim whitespace, and resolve relative paths before validating. Avoid double-decoding bypasses.

[preventive] **Apply context-specific output encoding** — Whatever Short Term Memory does with the input — SQL: parameterized queries, HTML: contextual escaping, OS commands: avoid shell, use exec arrays, LDAP: escape filter chars.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-20 — Improper Input Validation](#)

Cross-boundary data exposure risk: Agent Runtime / Orchestrator → Short Term Memory [cross-boundary]

STRIDE → Information Disclosure | Component: **Short Term Memory** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 6.8 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Agent Runtime / Orchestrator** → **Short Term Memory** (label: *Agent Runtime / Orchestrator → Short Term Memory*, protocol: TCP, auth: credentials, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Short Term Memory** (`datastore`).

■ **Description:** Information leaving 'Application tier' into 'Data tier' may include data the receiving zone is not authorized to see. Cross-boundary egress is a common data-leak surface.

■ Attack scenario:

1. Attacker observes traffic leaving the source zone, or compromises a component in the destination zone.
2. The destination zone receives more data than it strictly needs (over-fetching, verbose error responses, full record dumps).
3. Attacker harvests sensitive fields (PII, secrets, internal IDs) that should never have left the source zone.
4. Data is exfiltrated, sold, or used to plan a deeper attack on the originating zone.

DREAD — D:8 R:6 E:7 A:7 D:8 → **Total 36/50**

■ How to mitigate:

[preventive] **Enforce minimum-data-needed at the egress point** — Whitelist exactly which fields leave the source zone. Strip everything else server-side before serialization.

[preventive] **Tokenize or redact sensitive fields** — Replace PII / PCI / PHI fields with reversible tokens or one-way hashes when the destination zone doesn't need the cleartext value.

[detective] **Log cross-boundary data flows for review** — Sample-log the field set crossing this boundary (not the values) so DLP and privacy reviews can audit what's egressing.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Data-in-transit modification

STRIDE → Tampering | Component: **State Tracking Services** (datastore) | Source: rule-based

CWE: CWE-345 — Insufficient Verification of Data Authenticity

CVSS 3.1: 7.3 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **State Tracking Services** (`datastore`).

■ **Description:** Attacker on the network path alters request/response payloads.

■ Attack scenario:

1. Attacker reaches an input vector exposed by State Tracking Services (request body, query string, message queue payload, persisted state).
2. Submits malformed input designed to alter the program's logic or stored data (injection, parameter tampering, deserialization gadgets).
3. State Tracking Services processes the input without strict validation, and the malicious change takes effect.
4. Tampered data is persisted, executed, or relayed downstream — corrupting integrity beyond the original entry point.

DREAD — D:8 R:6 E:7 A:5 D:7 → **Total 33/50**

■ How to mitigate:

[preventive] **Schema-validate and parameterize all inputs** — JSON Schema / OpenAPI validation at API edges; parameterized queries / prepared statements for SQL; ORM-level validators on persisted models.

[preventive] **Sign and verify integrity-critical messages** — For commands or state mutations, attach an HMAC or signature so downstream consumers can detect tampering in transit or at rest.

[detective] **Use append-only / write-once stores for audit-critical data** — Where data must not be silently changed, write to an append-only log (e.g., immutable S3 bucket, CloudTrail-like store) and reconcile against the mutable copy.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-345 — Insufficient Verification of Data Authenticity](#)

Config / secret tampering

STRIDE → Tampering | Component: **State Tracking Services** (datastore) | Source: rule-based

CWE: CWE-345 — Insufficient Verification of Data Authenticity

CVSS 3.1: 7.3 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **State Tracking Services** (datastore).

■ **Description:** Unauthorized changes to runtime config alter security behavior.

■ Attack scenario:

1. Attacker reaches an input vector exposed by State Tracking Services (request body, query string, message queue payload, persisted state).
2. Submits malformed input designed to alter the program's logic or stored data (injection, parameter tampering, deserialization gadgets).
3. State Tracking Services processes the input without strict validation, and the malicious change takes effect.
4. Tampered data is persisted, executed, or relayed downstream — corrupting integrity beyond the original entry point.

DREAD — D:8 R:6 E:7 A:5 D:7 → **Total 33/50**

■ How to mitigate:

[preventive] **Schema-validate and parameterize all inputs** — JSON Schema / OpenAPI validation at API edges; parameterized queries / prepared statements for SQL; ORM-level validators on persisted models.

[preventive] **Sign and verify integrity-critical messages** — For commands or state mutations, attach an HMAC or signature so downstream consumers can detect tampering in transit or at rest.

[detective] **Use append-only / write-once stores for audit-critical data** — Where data must not be silently changed, write to an append-only log (e.g., immutable S3 bucket, CloudTrail-like store) and reconcile against the mutable copy.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-345 — Insufficient Verification of Data Authenticity](#)

Sensitive data exposure in transit

STRIDE → Information Disclosure | Component: **State Tracking Services** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **State Tracking Services** (datastore).

■ **Description:** PII, secrets, or tokens transmitted over unencrypted channels.

■ Attack scenario:

1. Attacker reaches State Tracking Services via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:8 R:6 E:7 A:5 D:8 → **Total 34/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Sensitive data exposure at rest

STRIDE → Information Disclosure | Component: **State Tracking Services** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **State Tracking Services** (datastore).

■ **Description:** Stored PII / secrets accessible without proper authorization.

■ Attack scenario:

1. Attacker reaches State Tracking Services via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:8 R:6 E:7 A:5 D:8 → **Total 34/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Insecure direct object reference (IDOR)

STRIDE → Information Disclosure | Component: **State Tracking Services** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **State Tracking Services** (datastore).

■ **Description:** User can access another user's resources by guessing IDs.

■ Attack scenario:

1. Attacker reaches State Tracking Services via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs,

debug data, full PII fields).

3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:8 R:8 E:7 A:5 D:8 → **Total 36/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Trust-boundary crossing without strong authn: Agent Runtime / Orchestrator → State Tracking Services [cross-boundary]

STRIDE → Spoofing | Component: **State Tracking Services** (datastore) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 6.8 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Agent Runtime / Orchestrator** → **State Tracking Services** (label: *Agent Runtime / Orchestrator → State Tracking Services*, protocol: TCP, auth: credentials, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **State Tracking Services** (`datastore`).

■ **Description:** Flow 'Agent Runtime / Orchestrator → State Tracking Services' crosses trust boundary 'Application tier' → 'Data tier'. Caller identity must be re-verified at the boundary; existing trust does not transit.

■■ Attack scenario:

1. Attacker positions themselves on the network path between source and destination, or compromises an upstream component in the source zone.
2. Crafts requests with forged or replayed identity claims (cookies, tokens, IP-based trust).
3. State Tracking Services accepts the request because identity is not re-verified at the boundary, treating the upstream zone as trusted.
4. Attacker can now perform actions as the impersonated principal across the boundary.

DREAD — D:8 R:6 E:7 A:7 D:7 → **Total 35/50**

■ How to mitigate:

[preventive] **Re-authenticate at the boundary** — Require a fresh, audience-bound credential at the receiver. Do not infer identity from network position, source IP, or upstream session.

[preventive] **Use mutual TLS or signed tokens with audience claims** — For service-to-service calls into State Tracking Services, terminate mTLS at the boundary and reject any caller without a valid client cert. For human-driven flows, use OAuth2 / OIDC with audience and issuer validation.

[detective] **Log every cross-boundary auth decision** — Emit an authentication event (success and failure) tagged with both source and destination zone. Forward to SIEM with retention ≥ 90 days.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Cross-boundary input not validated: Agent Runtime / Orchestrator → State Tracking Services [cross-boundary]

STRIDE → Tampering | Component: **State Tracking Services** (datastore) | Source: rule-based

CWE: CWE-20 — Improper Input Validation

CVSS 3.1: 8.7 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:L/SI:H/SA:N

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Agent Runtime / Orchestrator** → **State Tracking Services** (label: *Agent Runtime / Orchestrator → State Tracking Services*, protocol: TCP, auth: credentials, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **State Tracking Services** (`datastore`).

■ **Description:** Data crossing the trust boundary into 'Data tier' must be treated as untrusted, even if the source is internal. Implicit trust is the most common cause of injection / SSRF / deserialization bugs.

■ ■ **Attack scenario:**

1. Attacker compromises a less-trusted upstream zone or directly sends malformed input to State Tracking Services.
2. Sends payloads with unexpected types, lengths, or special characters (e.g. SQL meta-chars, NUL bytes, control sequences).
3. State Tracking Services processes the input under the assumption that the upstream zone has already validated it — but the boundary changes the security context.
4. Injection, deserialization flaws, or buffer issues trigger; attacker pivots into State Tracking Services or the zone behind it.

DREAD — D:8 R:6 E:7 A:7 D:7 → **Total 35/50**

■ **How to mitigate:**

[preventive] **Validate against an allow-list schema at the boundary** — Define an explicit schema (JSON Schema, Protobuf, OpenAPI) for every accepted message at this boundary. Reject anything that doesn't match — type, length, charset, enum values, nested depth.

[preventive] **Canonicalize before validation** — Decode URL-encoding, Unicode normalization (NFC), trim whitespace, and resolve relative paths before validating. Avoid double-decoding bypasses.

[preventive] **Apply context-specific output encoding** — Whatever State Tracking Services does with the input — SQL: parameterized queries, HTML: contextual escaping, OS commands: avoid shell, use exec arrays, LDAP: escape filter chars.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-20 — Improper Input Validation](#)

Cross-boundary data exposure risk: Agent Runtime / Orchestrator → State Tracking Services

[cross-boundary]

STRIDE → Information Disclosure | Component: **State Tracking Services** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 6.8 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Agent Runtime / Orchestrator** → **State Tracking Services** (label: *Agent Runtime / Orchestrator → State Tracking Services*, protocol: TCP, auth: credentials, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **State Tracking Services** (`datastore`).

■ **Description:** Information leaving 'Application tier' into 'Data tier' may include data the receiving zone is not authorized to see. Cross-boundary egress is a common data-leak surface.

■ ■ **Attack scenario:**

1. Attacker observes traffic leaving the source zone, or compromises a component in the destination zone.
2. The destination zone receives more data than it strictly needs (over-fetching, verbose error responses, full record dumps).
3. Attacker harvests sensitive fields (PII, secrets, internal IDs) that should never have left the source zone.
4. Data is exfiltrated, sold, or used to plan a deeper attack on the originating zone.

DREAD — D:8 R:6 E:7 A:7 D:8 → **Total 36/50**

■ **How to mitigate:**

[preventive] **Enforce minimum-data-needed at the egress point** — Whitelist exactly which fields leave the source zone. Strip everything else server-side before serialization.

[preventive] **Tokenize or redact sensitive fields** — Replace PII / PCI / PHI fields with reversible tokens or one-way hashes when the destination zone doesn't need the cleartext value.

[detective] **Log cross-boundary data flows for review** — Sample-log the field set crossing this boundary (not the values) so DLP and privacy reviews can audit what's egressing.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Authentication bypass via credential stuffing

STRIDE → Spoofing | Component: **Tools (Internal & External)** (external_entity) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Tools (Internal & External)** (`external\_entity`).

■ **Description:** Attackers reuse leaked credentials to impersonate legitimate users.

■ ■ **Attack scenario:**

1. Attacker obtains valid credentials via phishing, credential stuffing, or a third-party breach.
2. Logs in as the legitimate user/entity at Tools (Internal & External).
3. Performs actions indistinguishable from the real user (auth events look normal).
4. Damage scales with the impersonated user's permissions; admin accounts cause the most harm.

DREAD — D:7 R:6 E:9 A:5 D:9 → **Total 36/50**

■ **How to mitigate:**

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Session token hijacking

STRIDE → Spoofing | Component: **Tools (Internal & External)** (external_entity) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Tools (Internal & External)** (`external\_entity`).

■ **Description:** Attacker steals or predicts a session token to impersonate a user.

■ ■ **Attack scenario:**

1. Attacker obtains valid credentials via phishing, credential stuffing, or a third-party breach.
2. Logs in as the legitimate user/entity at Tools (Internal & External).
3. Performs actions indistinguishable from the real user (auth events look normal).
4. Damage scales with the impersonated user's permissions; admin accounts cause the most harm.

DREAD — D:7 R:6 E:9 A:5 D:9 → **Total 36/50**

■ **How to mitigate:**

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

API key / service identity spoofing

STRIDE → Spoofing | Component: **Tools (Internal & External)** (external_entity) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Tools (Internal & External)** (`external\_entity`).

■ **Description:** An attacker obtains or forges an API key to act as a trusted service.

■ Attack scenario:

1. Attacker obtains valid credentials via phishing, credential stuffing, or a third-party breach.
2. Logs in as the legitimate user/entity at Tools (Internal & External).
3. Performs actions indistinguishable from the real user (auth events look normal).
4. Damage scales with the impersonated user's permissions; admin accounts cause the most harm.

DREAD — D:7 R:6 E:9 A:5 D:9 → **Total 36/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Sensitive data exposure in transit

STRIDE → Information Disclosure | Component: **Tools (Internal & External)** (external_entity) | Source: rule-based

CWE: CWE-319 — Cleartext Transmission of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Tools (Internal & External)** (`external\_entity`).

■ **Description:** PII, secrets, or tokens transmitted over unencrypted channels.

■ Attack scenario:

1. Attacker reaches Tools (Internal & External) via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:6 E:9 A:5 D:10 → **Total 37/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For external_entity, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-319 — Cleartext Transmission of Sensitive Information](#)

Sensitive data exposure at rest

STRIDE → Information Disclosure | Component: **Tools (Internal & External)** (external_entity) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Tools (Internal & External)** (`external\_entity`).

■ **Description:** Stored PII / secrets accessible without proper authorization.

■ Attack scenario:

1. Attacker reaches Tools (Internal & External) via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.

4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:6 E:9 A:5 D:10 → **Total 37/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For external_entity, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Insecure direct object reference (IDOR)

STRIDE → Information Disclosure | Component: **Tools (Internal & External)** (external_entity) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Tools (Internal & External)** (external_entity).

■ **Description:** User can access another user's resources by guessing IDs.

■■ Attack scenario:

1. Attacker reaches Tools (Internal & External) via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:8 E:9 A:5 D:10 → **Total 39/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For external_entity, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Application-layer DDoS

STRIDE → Denial of Service | Component: **Tools (Internal & External)** (external_entity) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **Tools (Internal & External)** (external_entity).

■ **Description:** Attacker floods expensive endpoints (e.g., search, login).

■■ Attack scenario:

1. Attacker identifies a costly operation in Tools (Internal & External) (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Tools (Internal & External)'s resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Tools (Internal & External); cascading failure may take down dependents.

DREAD — D:7 R:6 E:9 A:5 D:9 → **Total 36/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Trust-boundary crossing without strong authn: Agent Runtime / Orchestrator → Tools (Internal & External) [cross-boundary]

STRIDE → Spoofing | Component: **Tools (Internal & External)** (external_entity) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Application tier → Internet

■ **Location:** On the data flow **Agent Runtime / Orchestrator → Tools (Internal & External)** (label: *Agent Runtime / Orchestrator → Tools (Internal & External)*, protocol: HTTPS, auth: api_key, encrypted: yes), crossing the trust boundary from `Application tier` into `Internet`. The receiving component is **Tools (Internal & External)** (`external\_entity`).

■ **Description:** Flow 'Agent Runtime / Orchestrator → Tools (Internal & External)' crosses trust boundary 'Application tier' → 'Internet'. Caller identity must be re-verified at the boundary; existing trust does not transit.

■■ Attack scenario:

1. Attacker positions themselves on the network path between source and destination, or compromises an upstream component in the source zone.
2. Crafts requests with forged or replayed identity claims (cookies, tokens, IP-based trust).
3. Tools (Internal & External) accepts the request because identity is not re-verified at the boundary, treating the upstream zone as trusted.
4. Attacker can now perform actions as the impersonated principal across the boundary.

DREAD — D:7 R:6 E:9 A:7 D:9 → **Total 38/50**

■ How to mitigate:

[preventive] **Re-authenticate at the boundary** — Require a fresh, audience-bound credential at the receiver. Do not infer identity from network position, source IP, or upstream session.

[preventive] **Use mutual TLS or signed tokens with audience claims** — For service-to-service calls into Tools (Internal & External), terminate mTLS at the boundary and reject any caller without a valid client cert. For human-driven flows, use OAuth2 / OIDC with audience and issuer validation.

[detective] **Log every cross-boundary auth decision** — Emit an authentication event (success and failure) tagged with both source and destination zone. Forward to SIEM with retention ≥ 90 days.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Cross-boundary input not validated: Agent Runtime / Orchestrator → Tools (Internal & External) [cross-boundary]

STRIDE → Tampering | Component: **Tools (Internal & External)** (external_entity) | Source: rule-based

CWE: CWE-20 — Improper Input Validation

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:N/I:H/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:N/VI:H/VA:N/SC:L/SI:H/SA:N

Boundary crossing: Application tier → Internet

■ **Location:** On the data flow **Agent Runtime / Orchestrator → Tools (Internal & External)** (label: *Agent Runtime / Orchestrator → Tools (Internal & External)*, protocol: HTTPS, auth: api_key, encrypted: yes), crossing the trust boundary from `Application tier` into `Internet`. The receiving component is **Tools (Internal & External)** (`external\_entity`).

■ **Description:** Data crossing the trust boundary into 'Internet' must be treated as untrusted, even if the source is internal. Implicit trust is the most common cause of injection / SSRF / deserialization bugs.

■ ■ **Attack scenario:**

1. Attacker compromises a less-trusted upstream zone or directly sends malformed input to Tools (Internal & External).
2. Sends payloads with unexpected types, lengths, or special characters (e.g. SQL meta-chars, NUL bytes, control sequences).
3. Tools (Internal & External) processes the input under the assumption that the upstream zone has already validated it — but the boundary changes the security context.
4. Injection, deserialization flaws, or buffer issues trigger; attacker pivots into Tools (Internal & External) or the zone behind it.

DREAD — D:7 R:6 E:9 A:7 D:9 → **Total 38/50**

■ **How to mitigate:**

[preventive] **Validate against an allow-list schema at the boundary** — Define an explicit schema (JSON Schema, Protobuf, OpenAPI) for every accepted message at this boundary. Reject anything that doesn't match — type, length, charset, enum values, nested depth.

[preventive] **Canonicalize before validation** — Decode URL-encoding, Unicode normalization (NFC), trim whitespace, and resolve relative paths before validating. Avoid double-decoding bypasses.

[preventive] **Apply context-specific output encoding** — Whatever Tools (Internal & External) does with the input — SQL: parameterized queries, HTML: contextual escaping, OS commands: avoid shell, use exec arrays, LDAP: escape filter chars.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-20 — Improper Input Validation](#)

Cross-boundary data exposure risk: Agent Runtime / Orchestrator → Tools (Internal & External) [cross-boundary]

STRIDE → Information Disclosure | Component: **Tools (Internal & External)** (external_entity) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Application tier → Internet

■ **Location:** On the data flow **Agent Runtime / Orchestrator** → **Tools (Internal & External)** (label: *Agent Runtime / Orchestrator → Tools (Internal & External)*, protocol: HTTPS, auth: api_key, encrypted: yes), crossing the trust boundary from 'Application tier' into 'Internet'. The receiving component is **Tools (Internal & External)** ('external_entity').

■ **Description:** Information leaving 'Application tier' into 'Internet' may include data the receiving zone is not authorized to see. Cross-boundary egress is a common data-leak surface.

■ ■ **Attack scenario:**

1. Attacker observes traffic leaving the source zone, or compromises a component in the destination zone.
2. The destination zone receives more data than it strictly needs (over-fetching, verbose error responses, full record dumps).
3. Attacker harvests sensitive fields (PII, secrets, internal IDs) that should never have left the source zone.
4. Data is exfiltrated, sold, or used to plan a deeper attack on the originating zone.

DREAD — D:7 R:6 E:9 A:7 D:10 → **Total 39/50**

■ **How to mitigate:**

[preventive] **Enforce minimum-data-needed at the egress point** — Whitelist exactly which fields leave the source zone. Strip everything else server-side before serialization.

[preventive] **Tokenize or redact sensitive fields** — Replace PII / PCI / PHI fields with reversible tokens or one-way hashes when the destination zone doesn't need the cleartext value.

[detective] **Log cross-boundary data flows for review** — Sample-log the field set crossing this boundary (not the values) so DLP and privacy reviews can audit what's egressing.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Authentication bypass via credential stuffing

STRIDE → Spoofing | Component: **Users** (user) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Users** (`user`).

■ **Description:** Attackers reuse leaked credentials to impersonate legitimate users.

■ ■ **Attack scenario:**

1. Attacker obtains valid credentials via phishing, credential stuffing, or a third-party breach.
2. Logs in as the legitimate user/entity at Users.
3. Performs actions indistinguishable from the real user (auth events look normal).
4. Damage scales with the impersonated user's permissions; admin accounts cause the most harm.

DREAD — D:7 R:6 E:9 A:5 D:9 → **Total 36/50**

■ **How to mitigate:**

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Session token hijacking

STRIDE → Spoofing | Component: **Users** (user) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Users** (`user`).

■ **Description:** Attacker steals or predicts a session token to impersonate a user.

■ ■ **Attack scenario:**

1. Attacker obtains valid credentials via phishing, credential stuffing, or a third-party breach.
2. Logs in as the legitimate user/entity at Users.
3. Performs actions indistinguishable from the real user (auth events look normal).
4. Damage scales with the impersonated user's permissions; admin accounts cause the most harm.

DREAD — D:7 R:6 E:9 A:5 D:9 → **Total 36/50**

■ **How to mitigate:**

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

API key / service identity spoofing

STRIDE → Spoofing | Component: **Users** (user) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Users** (`user`).

■ **Description:** An attacker obtains or forges an API key to act as a trusted service.

■ ■ **Attack scenario:**

1. Attacker obtains valid credentials via phishing, credential stuffing, or a third-party breach.
2. Logs in as the legitimate user/entity at Users.
3. Performs actions indistinguishable from the real user (auth events look normal).

4. Damage scales with the impersonated user's permissions; admin accounts cause the most harm.

DREAD — D:7 R:6 E:9 A:5 D:9 → **Total 36/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Data-in-transit modification

STRIDE → Tampering | Component: **Vector DB** (database) | Source: rule-based

CWE: CWE-345 — Insufficient Verification of Data Authenticity

CVSS 3.1: 7.3 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Vector DB** (‘database’).

■ **Description:** Attacker on the network path alters request/response payloads.

■■ Attack scenario:

1. Attacker reaches an input vector exposed by Vector DB (request body, query string, message queue payload, persisted state).
2. Submits malformed input designed to alter the program's logic or stored data (injection, parameter tampering, deserialization gadgets).
3. Vector DB processes the input without strict validation, and the malicious change takes effect.
4. Tampered data is persisted, executed, or relayed downstream — corrupting integrity beyond the original entry point.

DREAD — D:8 R:6 E:7 A:7 D:7 → **Total 35/50**

■ How to mitigate:

[preventive] **Schema-validate and parameterize all inputs** — JSON Schema / OpenAPI validation at API edges; parameterized queries / prepared statements for SQL; ORM-level validators on persisted models.

[preventive] **Sign and verify integrity-critical messages** — For commands or state mutations, attach an HMAC or signature so downstream consumers can detect tampering in transit or at rest.

[detective] **Use append-only / write-once stores for audit-critical data** — Where data must not be silently changed, write to an append-only log (e.g., immutable S3 bucket, CloudTrail-like store) and reconcile against the mutable copy.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-345 — Insufficient Verification of Data Authenticity](#)

Config / secret tampering

STRIDE → Tampering | Component: **Vector DB** (database) | Source: rule-based

CWE: CWE-345 — Insufficient Verification of Data Authenticity

CVSS 3.1: 7.3 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Vector DB** (‘database’).

■ **Description:** Unauthorized changes to runtime config alter security behavior.

■■ Attack scenario:

1. Attacker reaches an input vector exposed by Vector DB (request body, query string, message queue payload, persisted state).
2. Submits malformed input designed to alter the program's logic or stored data (injection, parameter tampering, deserialization gadgets).
3. Vector DB processes the input without strict validation, and the malicious change takes effect.
4. Tampered data is persisted, executed, or relayed downstream — corrupting integrity beyond the original entry point.

DREAD — D:8 R:6 E:7 A:7 D:7 → **Total 35/50**

■ How to mitigate:

[preventive] **Schema-validate and parameterize all inputs** — JSON Schema / OpenAPI validation at API edges; parameterized queries / prepared statements for SQL; ORM-level validators on persisted models.

[preventive] **Sign and verify integrity-critical messages** — For commands or state mutations, attach an HMAC or signature so downstream consumers can detect tampering in transit or at rest.

[detective] **Use append-only / write-once stores for audit-critical data** — Where data must not be silently changed, write to an append-only log (e.g., immutable S3 bucket, CloudTrail-like store) and reconcile against the mutable copy.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-345 — Insufficient Verification of Data Authenticity](#)

Log tampering

STRIDE → Repudiation | Component: **Vector DB** (database) | Source: rule-based

CWE: CWE-778 — Insufficient Logging

CVSS 3.1: 7.3 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Vector DB** (‘database’).

■ **Description:** An attacker with access modifies or deletes audit trails.

■■ Attack scenario:

1. User performs a sensitive action through Vector DB (a transfer, a permission change, a deletion).
2. Audit logs are absent, incomplete, or modifiable by the same principal who performed the action.
3. User later denies having performed the action, or an attacker covers their tracks.
4. Without tamper-evident logs, neither responsible party can be identified — leading to fraud, dispute, or regulatory exposure.

DREAD — D:8 R:6 E:7 A:7 D:7 → **Total 35/50**

■ How to mitigate:

[detective] **Emit tamper-evident audit logs** — Sign log entries (HMAC chain, hash-linked) so insertion or deletion is detectable. Forward to a separate, append-only system the action’s principal cannot administer.

[detective] **Capture sufficient detail per audit event** — Who (authenticated principal, not just session ID), what (specific action and target), when (UTC, monotonic-clock-corroborated), where (source IP, request ID), why (correlated to the upstream business event).

[detective] **Periodic log integrity verification** — Schedule daily integrity checks on the audit log chain. Alert on any gap.

[A09:2021 — Security Logging and Monitoring Failures](#) | [STRIDE reference](#) | [CWE-778 — Insufficient Logging](#)

Sensitive data exposure in transit

STRIDE → Information Disclosure | Component: **Vector DB** (database) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Vector DB** (‘database’).

■ **Description:** PII, secrets, or tokens transmitted over unencrypted channels.

■■ Attack scenario:

1. Attacker reaches Vector DB via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:8 R:6 E:7 A:7 D:8 → **Total 36/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don’t rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For database, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Sensitive data exposure at rest

STRIDE → Information Disclosure | Component: **Vector DB** (database) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Vector DB** (`database`).

■ **Description:** Stored PII / secrets accessible without proper authorization.

■ ■ Attack scenario:

1. Attacker reaches Vector DB via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:8 R:6 E:7 A:7 D:8 → **Total 36/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For database, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Insecure direct object reference (IDOR)

STRIDE → Information Disclosure | Component: **Vector DB** (database) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Vector DB** (`database`).

■ **Description:** User can access another user's resources by guessing IDs.

■ ■ Attack scenario:

1. Attacker reaches Vector DB via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:8 R:8 E:7 A:7 D:8 → **Total 38/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For database, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

Application-layer DDoS

STRIDE → Denial of Service | Component: **Vector DB** (database) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.3 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:H

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **Vector DB** (`database`).

■ **Description:** Attacker floods expensive endpoints (e.g., search, login).

■■ Attack scenario:

1. Attacker identifies a costly operation in Vector DB (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Vector DB's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Vector DB; cascading failure may take down dependents.

DREAD — D:8 R:6 E:7 A:7 D:7 → **Total 35/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Privilege escalation via mass assignment

STRIDE → Elevation of Privilege | Component: **Vector DB** (database) | Source: rule-based

CWE: CWE-269 — Improper Privilege Management

CVSS 3.1: 7.6 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

CVSS 4.0: 6.3 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

■ **Location:** Within component **Vector DB** (`database`).

■ **Description:** User submits extra fields (e.g., role=admin) and they bind to the model.

■■ Attack scenario:

1. Attacker authenticates to Vector DB as a low-privilege user, or reaches an unauthenticated entry point.
2. Identifies an authorization gap — a function that doesn't re-check the caller's role, an IDOR-style URL, an admin endpoint with weak gating.
3. Issues requests to that gap, gaining access to data or actions reserved for higher-privilege roles.
4. Now operating with elevated privilege, attacker can pivot to other components, exfiltrate data, or persist access.

DREAD — D:8 R:8 E:7 A:7 D:7 → **Total 37/50**

■ How to mitigate:

[preventive] **Re-authorize on every action, not just at session start** — Every privileged endpoint checks the current principal's roles/permissions for the specific resource being acted on. No 'logged in = allowed'.

[preventive] **Apply least-privilege to service identities** — The IAM/role attached to Vector DB should grant only the actions it actually performs. Audit IAM grants quarterly.

[detective] **Detect anomalous privilege use** — Alert when a principal performs an action they have not performed in the trailing 90 days, or when admin-tier actions originate from non-admin networks.

[A01:2021 — Broken Access Control](#) | [STRIDE reference](#) | [CWE-269 — Improper Privilege Management](#)

Trust-boundary crossing without strong authn: Retrieval (RAG) → Vector DB [cross-boundary]

STRIDE → Spoofing | Component: **Vector DB** (database) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 6.8 (Medium) [CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N](#)

CVSS 4.0: 5.8 (Medium) [CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N](#)

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Retrieval (RAG) → Vector DB** (label: *Retrieval (RAG) → Vector DB*, protocol: TCP, auth: credentials, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Vector DB** (`database`).

■ **Description:** Flow 'Retrieval (RAG) → Vector DB' crosses trust boundary 'Application tier' → 'Data tier'. Caller identity must be re-verified at the boundary; existing trust does not transit.

■ ■ **Attack scenario:**

1. Attacker positions themselves on the network path between source and destination, or compromises an upstream component in the source zone.
2. Crafts requests with forged or replayed identity claims (cookies, tokens, IP-based trust).
3. Vector DB accepts the request because identity is not re-verified at the boundary, treating the upstream zone as trusted.
4. Attacker can now perform actions as the impersonated principal across the boundary.

DREAD — D:8 R:6 E:7 A:9 D:7 → **Total 37/50**

■ **How to mitigate:**

[preventive] **Re-authenticate at the boundary** — Require a fresh, audience-bound credential at the receiver. Do not infer identity from network position, source IP, or upstream session.

[preventive] **Use mutual TLS or signed tokens with audience claims** — For service-to-service calls into Vector DB, terminate mTLS at the boundary and reject any caller without a valid client cert. For human-driven flows, use OAuth2 / OIDC with audience and issuer validation.

[detective] **Log every cross-boundary auth decision** — Emit an authentication event (success and failure) tagged with both source and destination zone. Forward to SIEM with retention ≥ 90 days.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Cross-boundary input not validated: Retrieval (RAG) → Vector DB [cross-boundary]

STRIDE → Tampering | Component: **Vector DB** (database) | Source: rule-based

CWE: CWE-20 — Improper Input Validation

CVSS 3.1: 8.7 (High) [CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:N](#)

CVSS 4.0: 5.8 (Medium) [CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:L/SI:H/SA:N](#)

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Retrieval (RAG) → Vector DB** (label: *Retrieval (RAG) → Vector DB*, protocol: TCP, auth: credentials, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Vector DB** (`database`).

■ **Description:** Data crossing the trust boundary into 'Data tier' must be treated as untrusted, even if the source is internal. Implicit trust is the most common cause of injection / SSRF / deserialization bugs.

■ ■ **Attack scenario:**

1. Attacker compromises a less-trusted upstream zone or directly sends malformed input to Vector DB.
2. Sends payloads with unexpected types, lengths, or special characters (e.g. SQL meta-chars, NUL bytes, control sequences).
3. Vector DB processes the input under the assumption that the upstream zone has already validated it — but the boundary changes the security context.
4. Injection, deserialization flaws, or buffer issues trigger; attacker pivots into Vector DB or the zone behind it.

DREAD — D:8 R:6 E:7 A:9 D:7 → **Total 37/50**

■ **How to mitigate:**

[preventive] **Validate against an allow-list schema at the boundary** — Define an explicit schema (JSON Schema, Protobuf, OpenAPI) for every accepted message at this boundary. Reject anything that doesn't match — type, length, charset, enum values, nested depth.

[preventive] **Canonicalize before validation** — Decode URL-encoding, Unicode normalization (NFC), trim whitespace, and resolve relative paths before validating. Avoid double-decoding bypasses.

[preventive] **Apply context-specific output encoding** — Whatever Vector DB does with the input — SQL: parameterized queries, HTML: contextual escaping, OS commands: avoid shell, use exec arrays, LDAP: escape filter chars.

Cross-boundary data exposure risk: Retrieval (RAG) → Vector DB [cross-boundary]

STRIDE → Information Disclosure | Component: **Vector DB** (database) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 6.8 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Application tier → Data tier

■ **Location:** On the data flow **Retrieval (RAG) → Vector DB** (label: *Retrieval (RAG) → Vector DB*, protocol: TCP, auth: credentials, encrypted: yes), crossing the trust boundary from `Application tier` into `Data tier`. The receiving component is **Vector DB** (`database`).

■ **Description:** Information leaving 'Application tier' into 'Data tier' may include data the receiving zone is not authorized to see. Cross-boundary egress is a common data-leak surface.

■ ■ Attack scenario:

1. Attacker observes traffic leaving the source zone, or compromises a component in the destination zone.
2. The destination zone receives more data than it strictly needs (over-fetching, verbose error responses, full record dumps).
3. Attacker harvests sensitive fields (PII, secrets, internal IDs) that should never have left the source zone.
4. Data is exfiltrated, sold, or used to plan a deeper attack on the originating zone.

DREAD — D:8 R:6 E:7 A:9 D:8 → **Total 38/50**

■ How to mitigate:

[preventive] **Enforce minimum-data-needed at the egress point** — Whitelist exactly which fields leave the source zone. Strip everything else server-side before serialization.

[preventive] **Tokenize or redact sensitive fields** — Replace PII / PCI / PHI fields with reversible tokens or one-way hashes when the destination zone doesn't need the cleartext value.

[detective] **Log cross-boundary data flows for review** — Sample-log the field set crossing this boundary (not the values) so DLP and privacy reviews can audit what's egressing.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Authentication bypass via credential stuffing

STRIDE → Spoofing | Component: **Web Search** (external_entity) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Web Search** (`external\_entity`).

■ **Description:** Attackers reuse leaked credentials to impersonate legitimate users.

■ ■ Attack scenario:

1. Attacker obtains valid credentials via phishing, credential stuffing, or a third-party breach.
2. Logs in as the legitimate user/entity at Web Search.
3. Performs actions indistinguishable from the real user (auth events look normal).
4. Damage scales with the impersonated user's permissions; admin accounts cause the most harm.

DREAD — D:7 R:6 E:9 A:5 D:9 → **Total 36/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Session token hijacking

STRIDE → Spoofing | Component: **Web Search** (external_entity) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Web Search** (`external\_entity`).

■ **Description:** Attacker steals or predicts a session token to impersonate a user.

■ ■ Attack scenario:

1. Attacker obtains valid credentials via phishing, credential stuffing, or a third-party breach.
2. Logs in as the legitimate user/entity at Web Search.
3. Performs actions indistinguishable from the real user (auth events look normal).
4. Damage scales with the impersonated user's permissions; admin accounts cause the most harm.

DREAD — D:7 R:6 E:9 A:5 D:9 → **Total 36/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

API key / service identity spoofing

STRIDE → Spoofing | Component: **Web Search** (external_entity) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Web Search** (`external\_entity`).

■ **Description:** An attacker obtains or forges an API key to act as a trusted service.

■ ■ Attack scenario:

1. Attacker obtains valid credentials via phishing, credential stuffing, or a third-party breach.
2. Logs in as the legitimate user/entity at Web Search.
3. Performs actions indistinguishable from the real user (auth events look normal).
4. Damage scales with the impersonated user's permissions; admin accounts cause the most harm.

DREAD — D:7 R:6 E:9 A:5 D:9 → **Total 36/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Sensitive data exposure in transit

STRIDE → Information Disclosure | Component: **Web Search** (external_entity) | Source: rule-based

CWE: CWE-319 — Cleartext Transmission of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Web Search** (`external\_entity`).

■ **Description:** PII, secrets, or tokens transmitted over unencrypted channels.

■ ■ **Attack scenario:**

1. Attacker reaches Web Search via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:6 E:9 A:5 D:10 → **Total 37/50**

■ **How to mitigate:**

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For external_entity, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-319 — Cleartext Transmission of Sensitive Information](#)

Sensitive data exposure at rest

STRIDE → Information Disclosure | Component: **Web Search** (external_entity) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Web Search** (external_entity).

■ **Description:** Stored PII / secrets accessible without proper authorization.

■ ■ **Attack scenario:**

1. Attacker reaches Web Search via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:6 E:9 A:5 D:10 → **Total 37/50**

■ **How to mitigate:**

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For external_entity, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Insecure direct object reference (IDOR)

STRIDE → Information Disclosure | Component: **Web Search** (external_entity) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Web Search** (external_entity).

■ **Description:** User can access another user's resources by guessing IDs.

■ ■ **Attack scenario:**

1. Attacker reaches Web Search via a legitimate or guessable channel.

2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:8 E:9 A:5 D:10 → **Total 39/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For external_entity, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Application-layer DDoS

STRIDE → Denial of Service | Component: **Web Search** (external_entity) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **Web Search** (external_entity).

■ **Description:** Attacker floods expensive endpoints (e.g., search, login).

■ Attack scenario:

1. Attacker identifies a costly operation in Web Search (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Web Search's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Web Search; cascading failure may take down dependents.

DREAD — D:7 R:6 E:9 A:5 D:9 → **Total 36/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Trust-boundary crossing without strong authn: Agent Runtime / Orchestrator → Web Search

[cross-boundary]

STRIDE → Spoofing | Component: **Web Search** (external_entity) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Application tier → Internet

■ **Location:** On the data flow **Agent Runtime / Orchestrator → Web Search** (label: *Agent Runtime / Orchestrator → Web Search*, protocol: HTTPS, auth: api_key, encrypted: yes), crossing the trust boundary from `Application tier` into `Internet`. The receiving component is **Web Search** (external_entity).

■ **Description:** Flow 'Agent Runtime / Orchestrator → Web Search' crosses trust boundary 'Application tier' → 'Internet'. Caller identity must be re-verified at the boundary; existing trust does not transit.

■ Attack scenario:

1. Attacker positions themselves on the network path between source and destination, or compromises an upstream component in the source zone.
2. Crafts requests with forged or replayed identity claims (cookies, tokens, IP-based trust).
3. Web Search accepts the request because identity is not re-verified at the boundary, treating the upstream zone as trusted.
4. Attacker can now perform actions as the impersonated principal across the boundary.

DREAD — D:7 R:6 E:9 A:7 D:9 → **Total 38/50**

■ How to mitigate:

[preventive] **Re-authenticate at the boundary** — Require a fresh, audience-bound credential at the receiver. Do not infer identity from network position, source IP, or upstream session.

[preventive] **Use mutual TLS or signed tokens with audience claims** — For service-to-service calls into Web Search, terminate mTLS at the boundary and reject any caller without a valid client cert. For human-driven flows, use OAuth2 / OIDC with audience and issuer validation.

[detective] **Log every cross-boundary auth decision** — Emit an authentication event (success and failure) tagged with both source and destination zone. Forward to SIEM with retention ≥ 90 days.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Cross-boundary input not validated: Agent Runtime / Orchestrator → Web Search [cross-boundary]

STRIDE → Tampering | Component: **Web Search** (external_entity) | Source: rule-based

CWE: CWE-20 — Improper Input Validation

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:N/I:H/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:N/VI:H/VA:N/SC:L/SI:H/SA:N

Boundary crossing: Application tier → Internet

■ **Location:** On the data flow **Agent Runtime / Orchestrator → Web Search** (label: *Agent Runtime / Orchestrator → Web Search*, protocol: HTTPS, auth: api_key, encrypted: yes), crossing the trust boundary from `Application tier` into `Internet`. The receiving component is **Web Search** (`external\_entity`).

■ **Description:** Data crossing the trust boundary into 'Internet' must be treated as untrusted, even if the source is internal. Implicit trust is the most common cause of injection / SSRF / deserialization bugs.

■ Attack scenario:

1. Attacker compromises a less-trusted upstream zone or directly sends malformed input to Web Search.
2. Sends payloads with unexpected types, lengths, or special characters (e.g. SQL meta-chars, NUL bytes, control sequences).
3. Web Search processes the input under the assumption that the upstream zone has already validated it — but the boundary changes the security context.
4. Injection, deserialization flaws, or buffer issues trigger; attacker pivots into Web Search or the zone behind it.

DREAD — D:7 R:6 E:9 A:7 D:9 → **Total 38/50**

■ How to mitigate:

[preventive] **Validate against an allow-list schema at the boundary** — Define an explicit schema (JSON Schema, Protobuf, OpenAPI) for every accepted message at this boundary. Reject anything that doesn't match — type, length, charset, enum values, nested depth.

[preventive] **Canonicalize before validation** — Decode URL-encoding, Unicode normalization (NFC), trim whitespace, and resolve relative paths before validating. Avoid double-decoding bypasses.

[preventive] **Apply context-specific output encoding** — Whatever Web Search does with the input — SQL: parameterized queries, HTML: contextual escaping, OS commands: avoid shell, use exec arrays, LDAP: escape filter chars.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-20 — Improper Input Validation](#)

Cross-boundary data exposure risk: Agent Runtime / Orchestrator → Web Search [cross-boundary]

STRIDE → Information Disclosure | Component: **Web Search** (external_entity) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Application tier → Internet

■ **Location:** On the data flow **Agent Runtime / Orchestrator** → **Web Search** (label: *Agent Runtime / Orchestrator → Web Search*, protocol: HTTPS, auth: api_key, encrypted: yes), crossing the trust boundary from `Application tier` into `Internet`. The receiving component is **Web Search** (`external\_entity`).

■ **Description:** Information leaving 'Application tier' into 'Internet' may include data the receiving zone is not authorized to see. Cross-boundary egress is a common data-leak surface.

■ ■ **Attack scenario:**

1. Attacker observes traffic leaving the source zone, or compromises a component in the destination zone.
2. The destination zone receives more data than it strictly needs (over-fetching, verbose error responses, full record dumps).
3. Attacker harvests sensitive fields (PII, secrets, internal IDs) that should never have left the source zone.
4. Data is exfiltrated, sold, or used to plan a deeper attack on the originating zone.

DREAD — D:7 R:6 E:9 A:7 D:10 → **Total 39/50**

■ **How to mitigate:**

[preventive] **Enforce minimum-data-needed at the egress point** — Whitelist exactly which fields leave the source zone. Strip everything else server-side before serialization.

[preventive] **Tokenize or redact sensitive fields** — Replace PII / PCI / PHI fields with reversible tokens or one-way hashes when the destination zone doesn't need the cleartext value.

[detective] **Log cross-boundary data flows for review** — Sample-log the field set crossing this boundary (not the values) so DLP and privacy reviews can audit what's egressing.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Authentication bypass via credential stuffing

STRIDE → Spoofing | Component: **Wiz AI Security Lens** (external_entity) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Wiz AI Security Lens** (`external\_entity`).

■ **Description:** Attackers reuse leaked credentials to impersonate legitimate users.

■ ■ **Attack scenario:**

1. Attacker obtains valid credentials via phishing, credential stuffing, or a third-party breach.
2. Logs in as the legitimate user/entity at Wiz AI Security Lens.
3. Performs actions indistinguishable from the real user (auth events look normal).
4. Damage scales with the impersonated user's permissions; admin accounts cause the most harm.

DREAD — D:7 R:6 E:9 A:5 D:9 → **Total 36/50**

■ **How to mitigate:**

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Session token hijacking

STRIDE → Spoofing | Component: **Wiz AI Security Lens** (external_entity) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Wiz AI Security Lens** (`external\_entity`).

■ **Description:** Attacker steals or predicts a session token to impersonate a user.

■ ■ ■ Attack scenario:

1. Attacker obtains valid credentials via phishing, credential stuffing, or a third-party breach.
2. Logs in as the legitimate user/entity at Wiz AI Security Lens.
3. Performs actions indistinguishable from the real user (auth events look normal).
4. Damage scales with the impersonated user's permissions; admin accounts cause the most harm.

DREAD — D:7 R:6 E:9 A:5 D:9 → **Total 36/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

API key / service identity spoofing

STRIDE → Spoofing | Component: **Wiz AI Security Lens** (external_entity) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Wiz AI Security Lens** (`external\_entity`).

■ **Description:** An attacker obtains or forges an API key to act as a trusted service.

■ ■ ■ Attack scenario:

1. Attacker obtains valid credentials via phishing, credential stuffing, or a third-party breach.
2. Logs in as the legitimate user/entity at Wiz AI Security Lens.
3. Performs actions indistinguishable from the real user (auth events look normal).
4. Damage scales with the impersonated user's permissions; admin accounts cause the most harm.

DREAD — D:7 R:6 E:9 A:5 D:9 → **Total 36/50**

■ How to mitigate:

[preventive] **Validate token signatures with explicit algorithm allow-listing** — Reject 'alg: none', reject HS256 when expecting RS256. Use a vetted JWT library; never hand-roll verification.

[preventive] **Enforce audience and issuer claims** — Every token must declare which service it's intended for. Reject tokens whose audience doesn't match this component.

[preventive] **Bind tokens to a session or device** — Use mTLS-bound tokens, DPoP, or token binding to prevent replay if a token is captured.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Sensitive data exposure in transit

STRIDE → Information Disclosure | Component: **Wiz AI Security Lens** (external_entity) | Source: rule-based

CWE: CWE-319 — Cleartext Transmission of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Wiz AI Security Lens** (`external\_entity`).

■ **Description:** PII, secrets, or tokens transmitted over unencrypted channels.

■ ■ ■ Attack scenario:

1. Attacker reaches Wiz AI Security Lens via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:6 E:9 A:5 D:10 → **Total 37/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For external_entity, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-319 — Cleartext Transmission of Sensitive Information](#)

Sensitive data exposure at rest

STRIDE → Information Disclosure | Component: **Wiz AI Security Lens** (external_entity) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Wiz AI Security Lens** (`external\_entity`).

■ **Description:** Stored PII / secrets accessible without proper authorization.

■■ Attack scenario:

1. Attacker reaches Wiz AI Security Lens via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:6 E:9 A:5 D:10 → **Total 37/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For external_entity, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Insecure direct object reference (IDOR)

STRIDE → Information Disclosure | Component: **Wiz AI Security Lens** (external_entity) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Wiz AI Security Lens** (`external\_entity`).

■ **Description:** User can access another user's resources by guessing IDs.

■■ Attack scenario:

1. Attacker reaches Wiz AI Security Lens via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:7 R:8 E:9 A:5 D:10 → **Total 39/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For external_entity, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Application-layer DDoS

STRIDE → Denial of Service | Component: **Wiz AI Security Lens** (external_entity) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **Wiz AI Security Lens** (external_entity).

■ **Description:** Attacker floods expensive endpoints (e.g., search, login).

■ ■ Attack scenario:

1. Attacker identifies a costly operation in Wiz AI Security Lens (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Wiz AI Security Lens's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Wiz AI Security Lens; cascading failure may take down dependents.

DREAD — D:7 R:6 E:9 A:5 D:9 → **Total 36/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Trust-boundary crossing without strong authn: Core Agent Layer → Wiz AI Security Lens [cross-boundary]

STRIDE → Spoofing | Component: **Wiz AI Security Lens** (external_entity) | Source: rule-based

CWE: CWE-287 — Improper Authentication

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Application tier → Internet

■ **Location:** On the data flow **Core Agent Layer → Wiz AI Security Lens** (label: *Core Agent Layer → Wiz AI Security Lens*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `Application tier` into `Internet`. The receiving component is **Wiz AI Security Lens** (external_entity).

■ **Description:** Flow 'Core Agent Layer → Wiz AI Security Lens' crosses trust boundary 'Application tier' → 'Internet'. Caller identity must be re-verified at the boundary; existing trust does not transit.

■ ■ Attack scenario:

1. Attacker positions themselves on the network path between source and destination, or compromises an upstream component in the source zone.
2. Crafts requests with forged or replayed identity claims (cookies, tokens, IP-based trust).
3. Wiz AI Security Lens accepts the request because identity is not re-verified at the boundary, treating the upstream zone as trusted.
4. Attacker can now perform actions as the impersonated principal across the boundary.

DREAD — D:7 R:6 E:9 A:7 D:9 → **Total 38/50**

■ How to mitigate:

[preventive] **Re-authenticate at the boundary** — Require a fresh, audience-bound credential at the receiver. Do not infer identity from network position, source IP, or upstream session.

[preventive] **Use mutual TLS or signed tokens with audience claims** — For service-to-service calls into Wiz AI Security Lens, terminate mTLS at the boundary and reject any caller without a valid client cert. For human-driven flows, use OAuth2 / OIDC with audience and issuer validation.

[detective] **Log every cross-boundary auth decision** — Emit an authentication event (success and failure) tagged with both source and destination zone. Forward to SIEM with retention ≥ 90 days.

[A07:2021 — Identification and Authentication Failures](#) | [STRIDE reference](#) | [CWE-287 — Improper Authentication](#)

Cross-boundary input not validated: Core Agent Layer → Wiz AI Security Lens [cross-boundary]

STRIDE → Tampering | Component: **Wiz AI Security Lens** (external_entity) | Source: rule-based

CWE: CWE-20 — Improper Input Validation

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:N/I:H/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:N/VI:H/VA:N/SC:L/SI:H/SA:N

Boundary crossing: Application tier → Internet

■ **Location:** On the data flow **Core Agent Layer → Wiz AI Security Lens** (label: *Core Agent Layer → Wiz AI Security Lens*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `Application tier` into `Internet`. The receiving component is **Wiz AI Security Lens** (`external\_entity`).

■ **Description:** Data crossing the trust boundary into 'Internet' must be treated as untrusted, even if the source is internal. Implicit trust is the most common cause of injection / SSRF / deserialization bugs.

■■ Attack scenario:

1. Attacker compromises a less-trusted upstream zone or directly sends malformed input to Wiz AI Security Lens.
2. Sends payloads with unexpected types, lengths, or special characters (e.g. SQL meta-chars, NUL bytes, control sequences).
3. Wiz AI Security Lens processes the input under the assumption that the upstream zone has already validated it — but the boundary changes the security context.
4. Injection, deserialization flaws, or buffer issues trigger; attacker pivots into Wiz AI Security Lens or the zone behind it.

DREAD — D:7 R:6 E:9 A:7 D:9 → **Total 38/50**

■ How to mitigate:

[preventive] **Validate against an allow-list schema at the boundary** — Define an explicit schema (JSON Schema, Protobuf, OpenAPI) for every accepted message at this boundary. Reject anything that doesn't match — type, length, charset, enum values, nested depth.

[preventive] **Canonicalize before validation** — Decode URL-encoding, Unicode normalization (NFC), trim whitespace, and resolve relative paths before validating. Avoid double-decoding bypasses.

[preventive] **Apply context-specific output encoding** — Whatever Wiz AI Security Lens does with the input — SQL: parameterized queries, HTML: contextual escaping, OS commands: avoid shell, use exec arrays, LDAP: escape filter chars.

[A03:2021 — Injection](#) | [STRIDE reference](#) | [CWE-20 — Improper Input Validation](#)

Cross-boundary data exposure risk: Core Agent Layer → Wiz AI Security Lens [cross-boundary]

STRIDE → Information Disclosure | Component: **Wiz AI Security Lens** (external_entity) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.7 (High) CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N

CVSS 4.0: 5.8 (Medium) CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:L/SA:N

Boundary crossing: Application tier → Internet

■ **Location:** On the data flow **Core Agent Layer → Wiz AI Security Lens** (label: *Core Agent Layer → Wiz AI Security Lens*, protocol: HTTPS, auth: bearer, encrypted: yes), crossing the trust boundary from `Application tier` into `Internet`. The receiving component is **Wiz AI Security Lens** (`external\_entity`).

■ **Description:** Information leaving 'Application tier' into 'Internet' may include data the receiving zone is not authorized to see. Cross-boundary egress is a common data-leak surface.

■■ Attack scenario:

1. Attacker observes traffic leaving the source zone, or compromises a component in the destination zone.

2. The destination zone receives more data than it strictly needs (over-fetching, verbose error responses, full record dumps).
3. Attacker harvests sensitive fields (PII, secrets, internal IDs) that should never have left the source zone.
4. Data is exfiltrated, sold, or used to plan a deeper attack on the originating zone.

DREAD — D:7 R:6 E:9 A:7 D:10 → **Total 39/50**

■ How to mitigate:

[preventive] **Enforce minimum-data-needed at the egress point** — Whitelist exactly which fields leave the source zone. Strip everything else server-side before serialization.

[preventive] **Tokenize or redact sensitive fields** — Replace PII / PCI / PHI fields with reversible tokens or one-way hashes when the destination zone doesn't need the cleartext value.

[detective] **Log cross-boundary data flows for review** — Sample-log the field set crossing this boundary (not the values) so DLP and privacy reviews can audit what's egressing.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Medium (73)

Insufficient audit logging

STRIDE → Repudiation | Component: **AI Detection & Response (api)** | Source: rule-based

CWE: CWE-778 — Insufficient Logging

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **AI Detection & Response** (`api`).

■ **Description:** Critical user actions cannot be reliably attributed after the fact.

■■ Attack scenario:

1. User performs a sensitive action through AI Detection & Response (a transfer, a permission change, a deletion).
2. Audit logs are absent, incomplete, or modifiable by the same principal who performed the action.
3. User later denies having performed the action, or an attacker covers their tracks.
4. Without tamper-evident logs, neither responsible party can be identified — leading to fraud, dispute, or regulatory exposure.

DREAD — D:5 R:4 E:5 A:6 D:5 → **Total 25/50**

■ How to mitigate:

[detective] **Emit tamper-evident audit logs** — Sign log entries (HMAC chain, hash-linked) so insertion or deletion is detectable. Forward to a separate, append-only system the action's principal cannot administer.

[detective] **Capture sufficient detail per audit event** — Who (authenticated principal, not just session ID), what (specific action and target), when (UTC, monotonic-clock-corroborated), where (source IP, request ID), why (correlated to the upstream business event).

[detective] **Periodic log integrity verification** — Schedule daily integrity checks on the audit log chain. Alert on any gap.

[A09:2021 — Security Logging and Monitoring Failures](#) | [STRIDE reference](#) | [CWE-778 — Insufficient Logging](#)

Verbose error messages / stack traces

STRIDE → Information Disclosure | Component: **AI Detection & Response (api)** | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **AI Detection & Response** (`api`).

■ **Description:** Error responses leak implementation details to attackers.

■■ Attack scenario:

1. Attacker reaches AI Detection & Response via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).

3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:5 R:4 E:5 A:6 D:6 → **Total 26/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Resource exhaustion via unbounded input

STRIDE → Denial of Service | Component: **AI Detection & Response** (api) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **AI Detection & Response** (`api`).

■ **Description:** Large payloads or unbounded loops exhaust CPU/memory.

■ Attack scenario:

1. Attacker identifies a costly operation in AI Detection & Response (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. AI Detection & Response's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach AI Detection & Response; cascading failure may take down dependents.

DREAD — D:5 R:4 E:5 A:6 D:5 → **Total 25/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Algorithmic complexity attack

STRIDE → Denial of Service | Component: **AI Detection & Response** (api) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **AI Detection & Response** (`api`).

■ **Description:** Crafted inputs trigger worst-case algorithm behavior (e.g., regex DoS).

■ Attack scenario:

1. Attacker identifies a costly operation in AI Detection & Response (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. AI Detection & Response's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach AI Detection & Response; cascading failure may take down dependents.

DREAD — D:5 R:4 E:5 A:6 D:5 → **Total 25/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Verbose error messages / stack traces

STRIDE → Information Disclosure | Component: **Agent Evaluation** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) [CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 5.2 (Medium) [CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Agent Evaluation** (`datastore`).

■ **Description:** Error responses leak implementation details to attackers.

■■ Attack scenario:

1. Attacker reaches Agent Evaluation via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:6 R:4 E:5 A:3 D:6 → **Total 24/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Insufficient audit logging

STRIDE → Repudiation | Component: **Agent Gateway** (api) | Source: rule-based

CWE: CWE-778 — Insufficient Logging

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:H/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Agent Gateway** (`api`).

■ **Description:** Critical user actions cannot be reliably attributed after the fact.

■■ Attack scenario:

1. User performs a sensitive action through Agent Gateway (a transfer, a permission change, a deletion).
2. Audit logs are absent, incomplete, or modifiable by the same principal who performed the action.
3. User later denies having performed the action, or an attacker covers their tracks.
4. Without tamper-evident logs, neither responsible party can be identified — leading to fraud, dispute, or regulatory exposure.

DREAD — D:5 R:4 E:7 A:7 D:7 → **Total 30/50**

■ How to mitigate:

[detective] **Emit tamper-evident audit logs** — Sign log entries (HMAC chain, hash-linked) so insertion or deletion is detectable. Forward to a separate, append-only system the action's principal cannot administer.

[*detective*] **Capture sufficient detail per audit event** — Who (authenticated principal, not just session ID), what (specific action and target), when (UTC, monotonic-clock-corroborated), where (source IP, request ID), why (correlated to the upstream business event).

[*detective*] **Periodic log integrity verification** — Schedule daily integrity checks on the audit log chain. Alert on any gap.

[A09:2021 — Security Logging and Monitoring Failures](#) | [STRIDE reference](#) | [CWE-778 — Insufficient Logging](#)

Verbose error messages / stack traces

STRIDE → Information Disclosure | Component: **Agent Gateway** (api) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Gateway** (`api`).

■ **Description:** Error responses leak implementation details to attackers.

■ ■ Attack scenario:

1. Attacker reaches Agent Gateway via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:5 R:4 E:7 A:7 D:8 → **Total 31/50**

■ How to mitigate:

[*preventive*] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[*preventive*] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[*preventive*] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Resource exhaustion via unbounded input

STRIDE → Denial of Service | Component: **Agent Gateway** (api) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Gateway** (`api`).

■ **Description:** Large payloads or unbounded loops exhaust CPU/memory.

■ ■ Attack scenario:

1. Attacker identifies a costly operation in Agent Gateway (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Agent Gateway's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Agent Gateway; cascading failure may take down dependents.

DREAD — D:5 R:4 E:7 A:7 D:7 → **Total 30/50**

■ How to mitigate:

[*preventive*] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[*preventive*] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[*corrective*] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

Algorithmic complexity attack

STRIDE → Denial of Service | Component: **Agent Gateway** (api) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Gateway** (`api`).

■ **Description:** Crafted inputs trigger worst-case algorithm behavior (e.g., regex DoS).

■■ Attack scenario:

1. Attacker identifies a costly operation in Agent Gateway (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Agent Gateway's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Agent Gateway; cascading failure may take down dependents.

DREAD — D:5 R:4 E:7 A:7 D:7 → **Total 30/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Verbose error messages / stack traces

STRIDE → Information Disclosure | Component: **Agent Registry** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Registry** (`datastore`).

■ **Description:** Error responses leak implementation details to attackers.

■■ Attack scenario:

1. Attacker reaches Agent Registry via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:6 R:4 E:5 A:3 D:6 → **Total 24/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Insufficient audit logging

STRIDE → Repudiation | Component: **Agent Runtime / Orchestrator** (api) | Source: rule-based

CWE: CWE-778 — Insufficient Logging

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:H/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Agent Runtime / Orchestrator** (`api`).

■ **Description:** Critical user actions cannot be reliably attributed after the fact.

■ ■ **Attack scenario:**

1. User performs a sensitive action through Agent Runtime / Orchestrator (a transfer, a permission change, a deletion).
2. Audit logs are absent, incomplete, or modifiable by the same principal who performed the action.
3. User later denies having performed the action, or an attacker covers their tracks.
4. Without tamper-evident logs, neither responsible party can be identified — leading to fraud, dispute, or regulatory exposure.

DREAD — D:5 R:4 E:5 A:8 D:5 → **Total 27/50**

■ **How to mitigate:**

[*detective*] **Emit tamper-evident audit logs** — Sign log entries (HMAC chain, hash-linked) so insertion or deletion is detectable. Forward to a separate, append-only system the action's principal cannot administer.

[*detective*] **Capture sufficient detail per audit event** — Who (authenticated principal, not just session ID), what (specific action and target), when (UTC, monotonic-clock-corroborated), where (source IP, request ID), why (correlated to the upstream business event).

[*detective*] **Periodic log integrity verification** — Schedule daily integrity checks on the audit log chain. Alert on any gap.

[A09:2021 — Security Logging and Monitoring Failures](#) | [STRIDE reference](#) | [CWE-778 — Insufficient Logging](#)

Verbose error messages / stack traces

STRIDE → Information Disclosure | Component: **Agent Runtime / Orchestrator** (api) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Agent Runtime / Orchestrator** (`api`).

■ **Description:** Error responses leak implementation details to attackers.

■ ■ **Attack scenario:**

1. Attacker reaches Agent Runtime / Orchestrator via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:5 R:4 E:5 A:8 D:6 → **Total 28/50**

■ **How to mitigate:**

[*preventive*] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[*preventive*] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[*preventive*] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Resource exhaustion via unbounded input

STRIDE → Denial of Service | Component: **Agent Runtime / Orchestrator** (api) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Agent Runtime / Orchestrator** (`api`).

■ **Description:** Large payloads or unbounded loops exhaust CPU/memory.

■ ■ **Attack scenario:**

1. Attacker identifies a costly operation in Agent Runtime / Orchestrator (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Agent Runtime / Orchestrator's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Agent Runtime / Orchestrator; cascading failure may take down dependents.

DREAD — D:5 R:4 E:5 A:8 D:5 → **Total 27/50**

■ **How to mitigate:**

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Algorithmic complexity attack

STRIDE → Denial of Service | Component: **Agent Runtime / Orchestrator** (api) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Runtime / Orchestrator** (`api`).

■ **Description:** Crafted inputs trigger worst-case algorithm behavior (e.g., regex DoS).

■ ■ **Attack scenario:**

1. Attacker identifies a costly operation in Agent Runtime / Orchestrator (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Agent Runtime / Orchestrator's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Agent Runtime / Orchestrator; cascading failure may take down dependents.

DREAD — D:5 R:4 E:5 A:8 D:5 → **Total 27/50**

■ **How to mitigate:**

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Verbose error messages / stack traces

STRIDE → Information Disclosure | Component: **Agent Tools Catalog** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Agent Tools Catalog** (`datastore`).

■ **Description:** Error responses leak implementation details to attackers.

■ ■ **Attack scenario:**

1. Attacker reaches Agent Tools Catalog via a legitimate or guessable channel.

2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:6 R:4 E:5 A:3 D:6 → **Total 24/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Insufficient audit logging

STRIDE → Repudiation | Component: **Core Agent Layer** (api) | Source: rule-based

CWE: CWE-778 — Insufficient Logging

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Core Agent Layer** (`api`).

■ **Description:** Critical user actions cannot be reliably attributed after the fact.

■■ Attack scenario:

1. User performs a sensitive action through Core Agent Layer (a transfer, a permission change, a deletion).
2. Audit logs are absent, incomplete, or modifiable by the same principal who performed the action.
3. User later denies having performed the action, or an attacker covers their tracks.
4. Without tamper-evident logs, neither responsible party can be identified — leading to fraud, dispute, or regulatory exposure.

DREAD — D:5 R:4 E:7 A:8 D:7 → **Total 31/50**

■ How to mitigate:

[detective] **Emit tamper-evident audit logs** — Sign log entries (HMAC chain, hash-linked) so insertion or deletion is detectable. Forward to a separate, append-only system the action's principal cannot administer.

[detective] **Capture sufficient detail per audit event** — Who (authenticated principal, not just session ID), what (specific action and target), when (UTC, monotonic-clock-corroborated), where (source IP, request ID), why (correlated to the upstream business event).

[detective] **Periodic log integrity verification** — Schedule daily integrity checks on the audit log chain. Alert on any gap.

[A09:2021 — Security Logging and Monitoring Failures](#) | [STRIDE reference](#) | [CWE-778 — Insufficient Logging](#)

Verbose error messages / stack traces

STRIDE → Information Disclosure | Component: **Core Agent Layer** (api) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Core Agent Layer** (`api`).

■ **Description:** Error responses leak implementation details to attackers.

■■ Attack scenario:

1. Attacker reaches Core Agent Layer via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:5 R:4 E:7 A:8 D:8 → **Total 32/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Resource exhaustion via unbounded input

STRIDE → Denial of Service | Component: **Core Agent Layer** (api) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **Core Agent Layer** (`api`).

■ **Description:** Large payloads or unbounded loops exhaust CPU/memory.

■■ Attack scenario:

1. Attacker identifies a costly operation in Core Agent Layer (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Core Agent Layer's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Core Agent Layer; cascading failure may take down dependents.

DREAD — D:5 R:4 E:7 A:8 D:7 → **Total 31/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Algorithmic complexity attack

STRIDE → Denial of Service | Component: **Core Agent Layer** (api) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **Core Agent Layer** (`api`).

■ **Description:** Crafted inputs trigger worst-case algorithm behavior (e.g., regex DoS).

■■ Attack scenario:

1. Attacker identifies a costly operation in Core Agent Layer (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Core Agent Layer's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Core Agent Layer; cascading failure may take down dependents.

DREAD — D:5 R:4 E:7 A:8 D:7 → **Total 31/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on

saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Verbose error messages / stack traces

STRIDE → Information Disclosure | Component: **CrowdStrike AIDR** (external_entity) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **CrowdStrike AIDR** (external_entity).

■ **Description:** Error responses leak implementation details to attackers.

■ ■ Attack scenario:

1. Attacker reaches CrowdStrike AIDR via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:5 R:4 E:7 A:3 D:8 → **Total 27/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For external_entity, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Resource exhaustion via unbounded input

STRIDE → Denial of Service | Component: **CrowdStrike AIDR** (external_entity) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **CrowdStrike AIDR** (external_entity).

■ **Description:** Large payloads or unbounded loops exhaust CPU/memory.

■ ■ Attack scenario:

1. Attacker identifies a costly operation in CrowdStrike AIDR (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. CrowdStrike AIDR's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach CrowdStrike AIDR; cascading failure may take down dependents.

DREAD — D:5 R:4 E:7 A:3 D:7 → **Total 26/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Algorithmic complexity attack

STRIDE → Denial of Service | Component: **CrowdStrike AIDR** (external_entity) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **CrowdStrike AIDR** (external_entity).

■ **Description:** Crafted inputs trigger worst-case algorithm behavior (e.g., regex DoS).

■ ■ Attack scenario:

1. Attacker identifies a costly operation in CrowdStrike AIDR (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. CrowdStrike AIDR's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach CrowdStrike AIDR; cascading failure may take down dependents.

DREAD — D:5 R:4 E:7 A:3 D:7 → **Total 26/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Insufficient audit logging

STRIDE → Repudiation | Component: **Identity Provider / Okta MCP** (auth_service) | Source: rule-based

CWE: CWE-778 — Insufficient Logging

CVSS 3.1: 9.1 (Critical) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Identity Provider / Okta MCP** (auth_service).

■ **Description:** Critical user actions cannot be reliably attributed after the fact.

■ ■ Attack scenario:

1. User performs a sensitive action through Identity Provider / Okta MCP (a transfer, a permission change, a deletion).
2. Audit logs are absent, incomplete, or modifiable by the same principal who performed the action.
3. User later denies having performed the action, or an attacker covers their tracks.
4. Without tamper-evident logs, neither responsible party can be identified — leading to fraud, dispute, or regulatory exposure.

DREAD — D:6 R:4 E:7 A:6 D:7 → **Total 30/50**

■ How to mitigate:

[detective] **Emit tamper-evident audit logs** — Sign log entries (HMAC chain, hash-linked) so insertion or deletion is detectable. Forward to a separate, append-only system the action's principal cannot administer.

[detective] **Capture sufficient detail per audit event** — Who (authenticated principal, not just session ID), what (specific action and target), when (UTC, monotonic-clock-corroborated), where (source IP, request ID), why (correlated to the upstream business event).

[detective] **Periodic log integrity verification** — Schedule daily integrity checks on the audit log chain. Alert on any gap.

[A09:2021 — Security Logging and Monitoring Failures](#) | [STRIDE reference](#) | [CWE-778 — Insufficient Logging](#)

Resource exhaustion via unbounded input

STRIDE → Denial of Service | Component: **Identity Provider / Okta MCP** (auth_service) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 9.1 (Critical) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:H

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:H/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Identity Provider / Okta MCP** (`auth\_service`).

■ **Description:** Large payloads or unbounded loops exhaust CPU/memory.

■ ■ **Attack scenario:**

1. Attacker identifies a costly operation in Identity Provider / Okta MCP (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Identity Provider / Okta MCP's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Identity Provider / Okta MCP; cascading failure may take down dependents.

DREAD — D:6 R:4 E:7 A:6 D:7 → **Total 30/50**

■ **How to mitigate:**

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Algorithmic complexity attack

STRIDE → Denial of Service | Component: **Identity Provider / Okta MCP** (auth_service) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 9.1 (Critical) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:H](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:H/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Identity Provider / Okta MCP** (`auth\_service`).

■ **Description:** Crafted inputs trigger worst-case algorithm behavior (e.g., regex DoS).

■ ■ **Attack scenario:**

1. Attacker identifies a costly operation in Identity Provider / Okta MCP (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Identity Provider / Okta MCP's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Identity Provider / Okta MCP; cascading failure may take down dependents.

DREAD — D:6 R:4 E:7 A:6 D:7 → **Total 30/50**

■ **How to mitigate:**

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Verbose error messages / stack traces

STRIDE → Information Disclosure | Component: **LLM Models** (external_entity) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **LLM Models** (`external\_entity`).

■ **Description:** Error responses leak implementation details to attackers.

■ ■ **Attack scenario:**

1. Attacker reaches LLM Models via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:5 R:4 E:7 A:3 D:8 → **Total 27/50**

■ **How to mitigate:**

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For external_entity, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Resource exhaustion via unbounded input

STRIDE → Denial of Service | Component: **LLM Models** (external_entity) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **LLM Models** (external_entity).

■ **Description:** Large payloads or unbounded loops exhaust CPU/memory.

■ ■ **Attack scenario:**

1. Attacker identifies a costly operation in LLM Models (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. LLM Models's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach LLM Models; cascading failure may take down dependents.

DREAD — D:5 R:4 E:7 A:3 D:7 → **Total 26/50**

■ **How to mitigate:**

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Algorithmic complexity attack

STRIDE → Denial of Service | Component: **LLM Models** (external_entity) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **LLM Models** (external_entity).

■ **Description:** Crafted inputs trigger worst-case algorithm behavior (e.g., regex DoS).

■ ■ **Attack scenario:**

1. Attacker identifies a costly operation in LLM Models (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.

- 3. LLM Models's resources (CPU, memory, connections, disk) saturate or exhaust.
- 4. Legitimate users can no longer reach LLM Models; cascading failure may take down dependents.

DREAD — D:5 R:4 E:7 A:3 D:7 → **Total 26/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Insufficient audit logging

STRIDE → Repudiation | Component: **Lambda Functions** (api) | Source: rule-based

CWE: CWE-778 — Insufficient Logging

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Lambda Functions** (`api`).

■ **Description:** Critical user actions cannot be reliably attributed after the fact.

■■ Attack scenario:

1. User performs a sensitive action through Lambda Functions (a transfer, a permission change, a deletion).
2. Audit logs are absent, incomplete, or modifiable by the same principal who performed the action.
3. User later denies having performed the action, or an attacker covers their tracks.
4. Without tamper-evident logs, neither responsible party can be identified — leading to fraud, dispute, or regulatory exposure.

DREAD — D:5 R:4 E:5 A:5 D:5 → **Total 24/50**

■ How to mitigate:

[detective] **Emit tamper-evident audit logs** — Sign log entries (HMAC chain, hash-linked) so insertion or deletion is detectable. Forward to a separate, append-only system the action's principal cannot administer.

[detective] **Capture sufficient detail per audit event** — Who (authenticated principal, not just session ID), what (specific action and target), when (UTC, monotonic-clock-corroborated), where (source IP, request ID), why (correlated to the upstream business event).

[detective] **Periodic log integrity verification** — Schedule daily integrity checks on the audit log chain. Alert on any gap.

[A09:2021 — Security Logging and Monitoring Failures](#) | [STRIDE reference](#) | [CWE-778 — Insufficient Logging](#)

Verbose error messages / stack traces

STRIDE → Information Disclosure | Component: **Lambda Functions** (api) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Lambda Functions** (`api`).

■ **Description:** Error responses leak implementation details to attackers.

■■ Attack scenario:

1. Attacker reaches Lambda Functions via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:5 R:4 E:5 A:5 D:6 → **Total 25/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Resource exhaustion via unbounded input

STRIDE → Denial of Service | Component: **Lambda Functions** (api) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **Lambda Functions** (`api`).

■ **Description:** Large payloads or unbounded loops exhaust CPU/memory.

■■ Attack scenario:

1. Attacker identifies a costly operation in Lambda Functions (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Lambda Functions's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Lambda Functions; cascading failure may take down dependents.

DREAD — D:5 R:4 E:5 A:5 D:5 → **Total 24/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Algorithmic complexity attack

STRIDE → Denial of Service | Component: **Lambda Functions** (api) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **Lambda Functions** (`api`).

■ **Description:** Crafted inputs trigger worst-case algorithm behavior (e.g., regex DoS).

■■ Attack scenario:

1. Attacker identifies a costly operation in Lambda Functions (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Lambda Functions's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Lambda Functions; cascading failure may take down dependents.

DREAD — D:5 R:4 E:5 A:5 D:5 → **Total 24/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Verbose error messages / stack traces

STRIDE → Information Disclosure | Component: **Long Term Memory** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Long Term Memory** (`datastore`).

■ **Description:** Error responses leak implementation details to attackers.

■ ■ Attack scenario:

1. Attacker reaches Long Term Memory via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:6 R:4 E:5 A:3 D:6 → **Total 24/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Insufficient audit logging

STRIDE → Repudiation | Component: **MCP Server Hosting** (api) | Source: rule-based

CWE: CWE-778 — Insufficient Logging

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **MCP Server Hosting** (`api`).

■ **Description:** Critical user actions cannot be reliably attributed after the fact.

■ ■ Attack scenario:

1. User performs a sensitive action through MCP Server Hosting (a transfer, a permission change, a deletion).
2. Audit logs are absent, incomplete, or modifiable by the same principal who performed the action.
3. User later denies having performed the action, or an attacker covers their tracks.
4. Without tamper-evident logs, neither responsible party can be identified — leading to fraud, dispute, or regulatory exposure.

DREAD — D:5 R:4 E:5 A:6 D:5 → **Total 25/50**

■ How to mitigate:

[detective] **Emit tamper-evident audit logs** — Sign log entries (HMAC chain, hash-linked) so insertion or deletion is detectable. Forward to a separate, append-only system the action's principal cannot administer.

[detective] **Capture sufficient detail per audit event** — Who (authenticated principal, not just session ID), what (specific action and target), when (UTC, monotonic-clock-corroborated), where (source IP, request ID), why (correlated to the upstream business event).

[detective] **Periodic log integrity verification** — Schedule daily integrity checks on the audit log chain. Alert on any gap.

[A09:2021 — Security Logging and Monitoring Failures](#) | [STRIDE reference](#) | [CWE-778 — Insufficient Logging](#)

Verbose error messages / stack traces

STRIDE → Information Disclosure | Component: **MCP Server Hosting** (api) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **MCP Server Hosting** (`api`).

■ **Description:** Error responses leak implementation details to attackers.

■ ■ **Attack scenario:**

1. Attacker reaches MCP Server Hosting via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:5 R:4 E:5 A:6 D:6 → **Total 26/50**

■ **How to mitigate:**

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Resource exhaustion via unbounded input

STRIDE → Denial of Service | Component: **MCP Server Hosting** (api) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **MCP Server Hosting** (`api`).

■ **Description:** Large payloads or unbounded loops exhaust CPU/memory.

■ ■ **Attack scenario:**

1. Attacker identifies a costly operation in MCP Server Hosting (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. MCP Server Hosting's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach MCP Server Hosting; cascading failure may take down dependents.

DREAD — D:5 R:4 E:5 A:6 D:5 → **Total 25/50**

■ **How to mitigate:**

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Algorithmic complexity attack

STRIDE → Denial of Service | Component: **MCP Server Hosting** (api) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **MCP Server Hosting** (``api``).

■ **Description:** Crafted inputs trigger worst-case algorithm behavior (e.g., regex DoS).

■■ **Attack scenario:**

1. Attacker identifies a costly operation in MCP Server Hosting (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. MCP Server Hosting's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach MCP Server Hosting; cascading failure may take down dependents.

DREAD — D:5 R:4 E:5 A:6 D:5 → **Total 25/50**

■ **How to mitigate:**

[*preventive*] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[*preventive*] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[*corrective*] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Verbose error messages / stack traces

STRIDE → Information Disclosure | Component: **MCP Server Targets** (`external_entity`) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **MCP Server Targets** (`external_entity`).

■ **Description:** Error responses leak implementation details to attackers.

■■ **Attack scenario:**

1. Attacker reaches MCP Server Targets via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:5 R:4 E:7 A:3 D:8 → **Total 27/50**

■ **How to mitigate:**

[*preventive*] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[*preventive*] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[*preventive*] **Encrypt data at rest with key separation** — For `external_entity`, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Resource exhaustion via unbounded input

STRIDE → Denial of Service | Component: **MCP Server Targets** (`external_entity`) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **MCP Server Targets** (`external\_entity`).

■ **Description:** Large payloads or unbounded loops exhaust CPU/memory.

■ ■ **Attack scenario:**

1. Attacker identifies a costly operation in MCP Server Targets (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. MCP Server Targets's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach MCP Server Targets; cascading failure may take down dependents.

DREAD — D:5 R:4 E:7 A:3 D:7 → **Total 26/50**

■ **How to mitigate:**

[*preventive*] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[*preventive*] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[*corrective*] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Algorithmic complexity attack

STRIDE → Denial of Service | Component: **MCP Server Targets** (external_entity) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **MCP Server Targets** (`external\_entity`).

■ **Description:** Crafted inputs trigger worst-case algorithm behavior (e.g., regex DoS).

■ ■ **Attack scenario:**

1. Attacker identifies a costly operation in MCP Server Targets (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. MCP Server Targets's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach MCP Server Targets; cascading failure may take down dependents.

DREAD — D:5 R:4 E:7 A:3 D:7 → **Total 26/50**

■ **How to mitigate:**

[*preventive*] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[*preventive*] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[*corrective*] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Verbose error messages / stack traces

STRIDE → Information Disclosure | Component: **Metrics & Tracing** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Metrics & Tracing** (`datastore`).

■ **Description:** Error responses leak implementation details to attackers.

■ Attack scenario:

1. Attacker reaches Metrics & Tracing via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:6 R:4 E:5 A:3 D:6 → **Total 24/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Insufficient audit logging

STRIDE → Repudiation | Component: **Model Gateway** (api) | Source: rule-based

CWE: CWE-778 — Insufficient Logging

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Model Gateway** (`api`).

■ **Description:** Critical user actions cannot be reliably attributed after the fact.

■ Attack scenario:

1. User performs a sensitive action through Model Gateway (a transfer, a permission change, a deletion).
2. Audit logs are absent, incomplete, or modifiable by the same principal who performed the action.
3. User later denies having performed the action, or an attacker covers their tracks.
4. Without tamper-evident logs, neither responsible party can be identified — leading to fraud, dispute, or regulatory exposure.

DREAD — D:5 R:4 E:7 A:6 D:7 → **Total 29/50**

■ How to mitigate:

[detective] **Emit tamper-evident audit logs** — Sign log entries (HMAC chain, hash-linked) so insertion or deletion is detectable. Forward to a separate, append-only system the action's principal cannot administer.

[detective] **Capture sufficient detail per audit event** — Who (authenticated principal, not just session ID), what (specific action and target), when (UTC, monotonic-clock-corroborated), where (source IP, request ID), why (correlated to the upstream business event).

[detective] **Periodic log integrity verification** — Schedule daily integrity checks on the audit log chain. Alert on any gap.

[A09:2021 — Security Logging and Monitoring Failures](#) | [STRIDE reference](#) | [CWE-778 — Insufficient Logging](#)

Verbose error messages / stack traces

STRIDE → Information Disclosure | Component: **Model Gateway** (api) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Model Gateway** (`api`).

■ **Description:** Error responses leak implementation details to attackers.

■ Attack scenario:

1. Attacker reaches Model Gateway via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.

4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:5 R:4 E:7 A:6 D:8 → **Total 30/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Resource exhaustion via unbounded input

STRIDE → Denial of Service | Component: **Model Gateway** (api) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **Model Gateway** (`api`).

■ **Description:** Large payloads or unbounded loops exhaust CPU/memory.

■■ Attack scenario:

1. Attacker identifies a costly operation in Model Gateway (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Model Gateway's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Model Gateway; cascading failure may take down dependents.

DREAD — D:5 R:4 E:7 A:6 D:7 → **Total 29/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Algorithmic complexity attack

STRIDE → Denial of Service | Component: **Model Gateway** (api) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **Model Gateway** (`api`).

■ **Description:** Crafted inputs trigger worst-case algorithm behavior (e.g., regex DoS).

■■ Attack scenario:

1. Attacker identifies a costly operation in Model Gateway (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Model Gateway's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Model Gateway; cascading failure may take down dependents.

DREAD — D:5 R:4 E:7 A:6 D:7 → **Total 29/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Insufficient audit logging

STRIDE → Repudiation | Component: **Planning & Workflow** (api) | Source: rule-based

CWE: CWE-778 — Insufficient Logging

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Planning & Workflow** (`api`).

■ **Description:** Critical user actions cannot be reliably attributed after the fact.

■■ Attack scenario:

1. User performs a sensitive action through Planning & Workflow (a transfer, a permission change, a deletion).
2. Audit logs are absent, incomplete, or modifiable by the same principal who performed the action.
3. User later denies having performed the action, or an attacker covers their tracks.
4. Without tamper-evident logs, neither responsible party can be identified — leading to fraud, dispute, or regulatory exposure.

DREAD — D:5 R:4 E:5 A:5 D:5 → **Total 24/50**

■ How to mitigate:

[detective] **Emit tamper-evident audit logs** — Sign log entries (HMAC chain, hash-linked) so insertion or deletion is detectable. Forward to a separate, append-only system the action's principal cannot administer.

[detective] **Capture sufficient detail per audit event** — Who (authenticated principal, not just session ID), what (specific action and target), when (UTC, monotonic-clock-corroborated), where (source IP, request ID), why (correlated to the upstream business event).

[detective] **Periodic log integrity verification** — Schedule daily integrity checks on the audit log chain. Alert on any gap.

[A09:2021 — Security Logging and Monitoring Failures](#) | [STRIDE reference](#) | [CWE-778 — Insufficient Logging](#)

Verbose error messages / stack traces

STRIDE → Information Disclosure | Component: **Planning & Workflow** (api) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Planning & Workflow** (`api`).

■ **Description:** Error responses leak implementation details to attackers.

■■ Attack scenario:

1. Attacker reaches Planning & Workflow via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:5 R:4 E:5 A:5 D:6 → **Total 25/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Resource exhaustion via unbounded input

STRIDE → Denial of Service | Component: **Planning & Workflow** (api) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **Planning & Workflow** (`api`).

■ **Description:** Large payloads or unbounded loops exhaust CPU/memory.

■ ■ Attack scenario:

1. Attacker identifies a costly operation in Planning & Workflow (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Planning & Workflow's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Planning & Workflow; cascading failure may take down dependents.

DREAD — D:5 R:4 E:5 A:5 D:5 → **Total 24/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Algorithmic complexity attack

STRIDE → Denial of Service | Component: **Planning & Workflow** (api) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **Planning & Workflow** (`api`).

■ **Description:** Crafted inputs trigger worst-case algorithm behavior (e.g., regex DoS).

■ ■ Attack scenario:

1. Attacker identifies a costly operation in Planning & Workflow (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Planning & Workflow's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Planning & Workflow; cascading failure may take down dependents.

DREAD — D:5 R:4 E:5 A:5 D:5 → **Total 24/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

Verbose error messages / stack traces

STRIDE → Information Disclosure | Component: **Prompt & Cyber Logging** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Prompt & Cyber Logging** (`datastore`).

■ **Description:** Error responses leak implementation details to attackers.

■ ■ ■ Attack scenario:

1. Attacker reaches Prompt & Cyber Logging via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:6 R:4 E:5 A:3 D:6 → **Total 24/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Insufficient audit logging

STRIDE → Repudiation | Component: **Retrieval (RAG)** (api) | Source: rule-based

CWE: CWE-778 — Insufficient Logging

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Retrieval (RAG)** (`api`).

■ **Description:** Critical user actions cannot be reliably attributed after the fact.

■ ■ ■ Attack scenario:

1. User performs a sensitive action through Retrieval (RAG) (a transfer, a permission change, a deletion).
2. Audit logs are absent, incomplete, or modifiable by the same principal who performed the action.
3. User later denies having performed the action, or an attacker covers their tracks.
4. Without tamper-evident logs, neither responsible party can be identified — leading to fraud, dispute, or regulatory exposure.

DREAD — D:5 R:4 E:5 A:6 D:5 → **Total 25/50**

■ How to mitigate:

[detective] **Emit tamper-evident audit logs** — Sign log entries (HMAC chain, hash-linked) so insertion or deletion is detectable. Forward to a separate, append-only system the action's principal cannot administer.

[detective] **Capture sufficient detail per audit event** — Who (authenticated principal, not just session ID), what (specific action and target), when (UTC, monotonic-clock-corroborated), where (source IP, request ID), why (correlated to the upstream business event).

[detective] **Periodic log integrity verification** — Schedule daily integrity checks on the audit log chain. Alert on any gap.

[A09:2021 — Security Logging and Monitoring Failures](#) | [STRIDE reference](#) | [CWE-778 — Insufficient Logging](#)

Verbose error messages / stack traces

STRIDE → Information Disclosure | Component: **Retrieval (RAG)** (api) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Retrieval (RAG)** (`api`).

■ **Description:** Error responses leak implementation details to attackers.

■ ■ **Attack scenario:**

1. Attacker reaches Retrieval (RAG) via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:5 R:4 E:5 A:6 D:6 → **Total 26/50**

■ **How to mitigate:**

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For api, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Resource exhaustion via unbounded input

[STRIDE](#) → Denial of Service | Component: **Retrieval (RAG)** (api) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Retrieval (RAG)** (`api`).

■ **Description:** Large payloads or unbounded loops exhaust CPU/memory.

■ ■ **Attack scenario:**

1. Attacker identifies a costly operation in Retrieval (RAG) (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Retrieval (RAG)'s resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Retrieval (RAG); cascading failure may take down dependents.

DREAD — D:5 R:4 E:5 A:6 D:5 → **Total 25/50**

■ **How to mitigate:**

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Algorithmic complexity attack

[STRIDE](#) → Denial of Service | Component: **Retrieval (RAG)** (api) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Retrieval (RAG)** (`api`).

■ **Description:** Crafted inputs trigger worst-case algorithm behavior (e.g., regex DoS).

■ ■ **Attack scenario:**

1. Attacker identifies a costly operation in Retrieval (RAG) (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Retrieval (RAG)'s resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Retrieval (RAG); cascading failure may take down dependents.

DREAD — D:5 R:4 E:5 A:6 D:5 → **Total 25/50**

■ **How to mitigate:**

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Insufficient audit logging

STRIDE → Repudiation | Component: **SailPoint Agent Identity** (auth_service) | Source: rule-based

CWE: CWE-778 — Insufficient Logging

CVSS 3.1: 9.1 (Critical) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **SailPoint Agent Identity** (auth_service).

■ **Description:** Critical user actions cannot be reliably attributed after the fact.

■ ■ **Attack scenario:**

1. User performs a sensitive action through SailPoint Agent Identity (a transfer, a permission change, a deletion).
2. Audit logs are absent, incomplete, or modifiable by the same principal who performed the action.
3. User later denies having performed the action, or an attacker covers their tracks.
4. Without tamper-evident logs, neither responsible party can be identified — leading to fraud, dispute, or regulatory exposure.

DREAD — D:6 R:4 E:7 A:5 D:7 → **Total 29/50**

■ **How to mitigate:**

[detective] **Emit tamper-evident audit logs** — Sign log entries (HMAC chain, hash-linked) so insertion or deletion is detectable. Forward to a separate, append-only system the action's principal cannot administer.

[detective] **Capture sufficient detail per audit event** — Who (authenticated principal, not just session ID), what (specific action and target), when (UTC, monotonic-clock-corroborated), where (source IP, request ID), why (correlated to the upstream business event).

[detective] **Periodic log integrity verification** — Schedule daily integrity checks on the audit log chain. Alert on any gap.

[A09:2021 — Security Logging and Monitoring Failures](#) | [STRIDE reference](#) | [CWE-778 — Insufficient Logging](#)

Resource exhaustion via unbounded input

STRIDE → Denial of Service | Component: **SailPoint Agent Identity** (auth_service) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 9.1 (Critical) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **SailPoint Agent Identity** (auth_service).

■ **Description:** Large payloads or unbounded loops exhaust CPU/memory.

■ ■ **Attack scenario:**

1. Attacker identifies a costly operation in SailPoint Agent Identity (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. SailPoint Agent Identity's resources (CPU, memory, connections, disk) saturate or exhaust.

4. Legitimate users can no longer reach SailPoint Agent Identity; cascading failure may take down dependents.

DREAD — D:6 R:4 E:7 A:5 D:7 → **Total 29/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Algorithmic complexity attack

STRIDE → Denial of Service | Component: **SailPoint Agent Identity** (auth_service) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 9.1 (Critical) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **SailPoint Agent Identity** (`auth\_service`).

■ **Description:** Crafted inputs trigger worst-case algorithm behavior (e.g., regex DoS).

■■ Attack scenario:

1. Attacker identifies a costly operation in SailPoint Agent Identity (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. SailPoint Agent Identity's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach SailPoint Agent Identity; cascading failure may take down dependents.

DREAD — D:6 R:4 E:7 A:5 D:7 → **Total 29/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Verbose error messages / stack traces

STRIDE → Information Disclosure | Component: **Session Memory** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Session Memory** (`datastore`).

■ **Description:** Error responses leak implementation details to attackers.

■■ Attack scenario:

1. Attacker reaches Session Memory via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:6 R:4 E:5 A:3 D:6 → **Total 24/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Verbose error messages / stack traces

STRIDE → Information Disclosure | Component: **Short Term Memory** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Short Term Memory** (datastore).

■ **Description:** Error responses leak implementation details to attackers.

■■ Attack scenario:

1. Attacker reaches Short Term Memory via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:6 R:4 E:5 A:3 D:6 → **Total 24/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Verbose error messages / stack traces

STRIDE → Information Disclosure | Component: **State Tracking Services** (datastore) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **State Tracking Services** (datastore).

■ **Description:** Error responses leak implementation details to attackers.

■■ Attack scenario:

1. Attacker reaches State Tracking Services via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:6 R:4 E:5 A:3 D:6 → **Total 24/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For datastore, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Verbose error messages / stack traces

STRIDE → Information Disclosure | Component: **Tools (Internal & External)** (external_entity) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Tools (Internal & External)** (external_entity).

■ **Description:** Error responses leak implementation details to attackers.

■ ■ ■ Attack scenario:

1. Attacker reaches Tools (Internal & External) via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:5 R:4 E:7 A:3 D:8 → **Total 27/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For external_entity, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Resource exhaustion via unbounded input

STRIDE → Denial of Service | Component: **Tools (Internal & External)** (external_entity) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **Tools (Internal & External)** (external_entity).

■ **Description:** Large payloads or unbounded loops exhaust CPU/memory.

■ ■ ■ Attack scenario:

1. Attacker identifies a costly operation in Tools (Internal & External) (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Tools (Internal & External)'s resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Tools (Internal & External); cascading failure may take down dependents.

DREAD — D:5 R:4 E:7 A:3 D:7 → **Total 26/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Algorithmic complexity attack

STRIDE → Denial of Service | Component: **Tools (Internal & External)** (external_entity) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **Tools (Internal & External)** (external_entity).

■ **Description:** Crafted inputs trigger worst-case algorithm behavior (e.g., regex DoS).

■ ■ ■ Attack scenario:

1. Attacker identifies a costly operation in Tools (Internal & External) (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Tools (Internal & External)'s resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Tools (Internal & External); cascading failure may take down dependents.

DREAD — D:5 R:4 E:7 A:3 D:7 → **Total 26/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Insufficient audit logging

STRIDE → Repudiation | Component: **Vector DB** (database) | Source: rule-based

CWE: CWE-778 — Insufficient Logging

CVSS 3.1: 7.3 (High) CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

CVSS 4.0: 5.2 (Medium) CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Vector DB** (database).

■ **Description:** Critical user actions cannot be reliably attributed after the fact.

■ ■ ■ Attack scenario:

1. User performs a sensitive action through Vector DB (a transfer, a permission change, a deletion).
2. Audit logs are absent, incomplete, or modifiable by the same principal who performed the action.
3. User later denies having performed the action, or an attacker covers their tracks.
4. Without tamper-evident logs, neither responsible party can be identified — leading to fraud, dispute, or regulatory exposure.

DREAD — D:6 R:4 E:5 A:5 D:5 → **Total 25/50**

■ How to mitigate:

[detective] **Emit tamper-evident audit logs** — Sign log entries (HMAC chain, hash-linked) so insertion or deletion is detectable. Forward to a separate, append-only system the action's principal cannot administer.

[detective] **Capture sufficient detail per audit event** — Who (authenticated principal, not just session ID), what (specific action and target), when (UTC, monotonic-clock-corroborated), where (source IP, request ID), why (correlated to the upstream business event).

[detective] **Periodic log integrity verification** — Schedule daily integrity checks on the audit log chain. Alert on any gap.

[A09:2021 — Security Logging and Monitoring Failures](#) | [STRIDE reference](#) | [CWE-778 — Insufficient Logging](#)

Verbose error messages / stack traces

STRIDE → Information Disclosure | Component: **Vector DB** (database) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 5.7 (Medium) [CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 5.2 (Medium) [CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Vector DB** (`database`).

■ **Description:** Error responses leak implementation details to attackers.

■ ■ Attack scenario:

1. Attacker reaches Vector DB via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:6 R:4 E:5 A:5 D:6 → **Total 26/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For database, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Resource exhaustion via unbounded input

STRIDE → Denial of Service | Component: **Vector DB** (database) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.3 (High) [CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:H](#)

CVSS 4.0: 5.2 (Medium) [CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:N/VA:H/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Vector DB** (`database`).

■ **Description:** Large payloads or unbounded loops exhaust CPU/memory.

■ ■ Attack scenario:

1. Attacker identifies a costly operation in Vector DB (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Vector DB's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Vector DB; cascading failure may take down dependents.

DREAD — D:6 R:4 E:5 A:5 D:5 → **Total 25/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Algorithmic complexity attack

STRIDE → Denial of Service | Component: **Vector DB** (database) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.3 (High) [CVSS:3.1/AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:H](#)

CVSS 4.0: 5.2 (Medium) [CVSS:4.0/AV:A/AC:L/AT:N/PR:L/UI:N/VC:L/VI:N/VA:H/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Vector DB** (`database`).

■ **Description:** Crafted inputs trigger worst-case algorithm behavior (e.g., regex DoS).

■■ **Attack scenario:**

1. Attacker identifies a costly operation in Vector DB (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Vector DB's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Vector DB; cascading failure may take down dependents.

DREAD — D:6 R:4 E:5 A:5 D:5 → **Total 25/50**

■ **How to mitigate:**

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Verbose error messages / stack traces

STRIDE → Information Disclosure | Component: **Web Search** (external_entity) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Web Search** (`external\_entity`).

■ **Description:** Error responses leak implementation details to attackers.

■■ **Attack scenario:**

1. Attacker reaches Web Search via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).
3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:5 R:4 E:7 A:3 D:8 → **Total 27/50**

■ **How to mitigate:**

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For external_entity, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Resource exhaustion via unbounded input

STRIDE → Denial of Service | Component: **Web Search** (external_entity) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H](#)

CVSS 4.0: 7.0 (High) [CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N](#)

■ **Location:** Within component **Web Search** (`external\_entity`).

■ **Description:** Large payloads or unbounded loops exhaust CPU/memory.

■ ■ **Attack scenario:**

1. Attacker identifies a costly operation in Web Search (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Web Search's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Web Search; cascading failure may take down dependents.

DREAD — D:5 R:4 E:7 A:3 D:7 → **Total 26/50**

■ **How to mitigate:**

[*preventive*] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[*preventive*] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[*corrective*] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Algorithmic complexity attack

STRIDE → Denial of Service | Component: **Web Search** (external_entity) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **Web Search** (external_entity).

■ **Description:** Crafted inputs trigger worst-case algorithm behavior (e.g., regex DoS).

■ ■ **Attack scenario:**

1. Attacker identifies a costly operation in Web Search (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Web Search's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Web Search; cascading failure may take down dependents.

DREAD — D:5 R:4 E:7 A:3 D:7 → **Total 26/50**

■ **How to mitigate:**

[*preventive*] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[*preventive*] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[*corrective*] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Verbose error messages / stack traces

STRIDE → Information Disclosure | Component: **Wiz AI Security Lens** (external_entity) | Source: rule-based

CWE: CWE-200 — Exposure of Sensitive Information

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

■ **Location:** Within component **Wiz AI Security Lens** (external_entity).

■ **Description:** Error responses leak implementation details to attackers.

■ ■ **Attack scenario:**

1. Attacker reaches Wiz AI Security Lens via a legitimate or guessable channel.
2. Triggers an error, edge case, or verbose endpoint that returns more information than necessary (stack traces, internal IDs, debug data, full PII fields).

3. Aggregates leaked information from multiple requests to build a profile of the system or its users.
4. Uses the harvested information to plan a targeted attack, reset accounts, or commit fraud.

DREAD — D:5 R:4 E:7 A:3 D:8 → **Total 27/50**

■ How to mitigate:

[preventive] **Apply field-level access control on responses** — The data layer enforces what the calling principal is allowed to see. Don't rely on the UI to hide fields.

[preventive] **Strip debug detail from error responses** — Production responses return a stable error code and a correlation ID. Stack traces, SQL errors, and internal IDs go to logs only.

[preventive] **Encrypt data at rest with key separation** — For external_entity, enable storage-level encryption with a CMK distinct from the encryption-at-rest key for adjacent stores. Rotate annually.

[A02:2021 — Cryptographic Failures](#) | [STRIDE reference](#) | [CWE-200 — Exposure of Sensitive Information](#)

Resource exhaustion via unbounded input

STRIDE → Denial of Service | Component: **Wiz AI Security Lens** (external_entity) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **Wiz AI Security Lens** (`external\_entity`).

■ **Description:** Large payloads or unbounded loops exhaust CPU/memory.

■■ Attack scenario:

1. Attacker identifies a costly operation in Wiz AI Security Lens (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Wiz AI Security Lens's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Wiz AI Security Lens; cascading failure may take down dependents.

DREAD — D:5 R:4 E:7 A:3 D:7 → **Total 26/50**

■ How to mitigate:

[preventive] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[preventive] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[corrective] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)

Algorithmic complexity attack

STRIDE → Denial of Service | Component: **Wiz AI Security Lens** (external_entity) | Source: rule-based

CWE: CWE-400 — Uncontrolled Resource Consumption (DoS)

CVSS 3.1: 7.5 (High) CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

CVSS 4.0: 7.0 (High) CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N

■ **Location:** Within component **Wiz AI Security Lens** (`external\_entity`).

■ **Description:** Crafted inputs trigger worst-case algorithm behavior (e.g., regex DoS).

■■ Attack scenario:

1. Attacker identifies a costly operation in Wiz AI Security Lens (regex matching, file generation, expensive query, large allocation).
2. Issues many concurrent requests targeting that operation, or a single request with pathological input.
3. Wiz AI Security Lens's resources (CPU, memory, connections, disk) saturate or exhaust.
4. Legitimate users can no longer reach Wiz AI Security Lens; cascading failure may take down dependents.

DREAD — D:5 R:4 E:7 A:3 D:7 → **Total 26/50**

■ **How to mitigate:**

[*preventive*] **Apply input limits at the edge** — Maximum request size, body depth, array length, and string length. Reject early — before parsing or business logic runs.

[*preventive*] **Rate-limit per principal and per resource** — Token bucket per authenticated user AND per costly endpoint. Adaptive throttling on saturation.

[*corrective*] **Set hard timeouts on outbound calls** — No call (DB, downstream service, third-party API) should be able to hang the request handler indefinitely. Use circuit breakers and bulkheads.

[A05:2021 — Security Misconfiguration](#) | [STRIDE reference](#) | [CWE-400 — Uncontrolled Resource Consumption \(DoS\)](#)